Applications: Statistical Modelling, Master in Data Science, University of Milano-Bicocca

Prof. Fulvia Pennoni, email: fulvia.pennoni@unimib.it

29 Apr 2026

Contents

Learning objectives **5**

Introduction to R **5**
Basic syntax for R 6
Hints on logical operators and functions 12
Help 14
Programming 15
Functions 17
Graphical representations 19
Cleaning environment 22
Libraries 22

Learning objectives **25**

Introduction to R `Markdown` **25**
Tips 26
Usage 27
File 27
Code 27
The chunk 27
Echo and Eval 28
Example 28
Code extraction 29

Learning objectives **30**

Case study: Credit rating **30**

Exploratory analyses **31**
Univariate summaries 31
Variability indices 34
Visualization 35

Learning objectives **42**

Generating values from random variables **42**
Realisations from the univariate Gaussian distribution 42

Learning objectives **51**

Measures of association **51**
Correlations 51
Partial correlations 52
Corplot 53

Learning objectives **56**

Bivariate Normal distribution **56**
Generate values from the bivariate Normal distribution 56
Scatterplot 59
3D Density plot 60
Contour plots 62
Bivariate distribution with low linear correlation 64
Bivariate distribution with high lienar correlation 66

Variance-covariance matrix **69**
Review on matrix multiplication 69
Eigenvalue decomposition 70

Example of likelihood function **72**

Learning objectives **75**

Nonparametric bootstrap **75**
Data scores 75
Skewness index 77
Bootstrap 78
Use of the boostrap library 81

Bootstrap confidence intervals **81**
Percentile method 81

Learning objectives **85**

Multiple linear regression: Profits of the firms **85**

Estimation of the model parameters **85**
Output of the function lm 86
Goodness of fit 88

Learning objectives **91**

Estimated multiple linear regression model **91**
Fitted values 91
Residuals 92
Decting unusual and influential observations 93
Leverage points 100
Cook's distance 102
Removing one record from the sample 103

Learning objectives **105**

Estimation of the variance **105**
Comparison of full and null model 106
t-Test 107
Confidence intervals for the regression parameters 109

Learning objectives **111**

Multicollinearity: Variance inflation factor **111**

Model selection: Data-driven approach **111**
Example: Percentage of body fat 112

Prediction **122**
Prediction of mean response 124

Learning objectives **125**

Dummy covariate **125**
Summary by the categorical variable 127
Plot 128

Interaction terms **133**

Categorical covariate with more than two levels **135**

Learning objectives **142**

Logistic regression model for a binary response **142**
Relevel of the response 144
Explanatory analyses 144
Estimation of the model with glm function 150
Fitted values 152
Model selection 153
Adding the place of residence 155

Prediction: classification table and ROC curve **158**

Learning objectives **163**

Estimation of a Gaussian mixture model for cholesterol data **163**
Function Mclust 164
Classification 167
Model selection 169
Bootstrap standard errors and confidence interval 171

© *Fulvia Pennoni\ All rights reserved. Students are not allowed to reproduce this material.*

Learning objectives

This is an *introduction* to basic syntax of R to illustrate the main features and functions of this programming environment.

Introduction to R

Statistics is currently developing more and more in its computational aspect, which was inaugurated in the 1950s with the first programming languages (in 1952 Grace Hopper proposed the first symbolic programming language, which later developed into the programming language known as FORTRAN). In this respect, it is referred to as *computational statistics or statistical computing*.

The basic idea of current statistical systems such as SAS and R is to enable data processing with a specific user and programming language.

In particular, the programming language of R is one of the languages referred to as *programming languages* or matrix languages that make use of algebraic tools to define objects of two or more dimensions (arrays, consisting of vectors and matrices).
Some advantages of using R are as follows: it is open-source, it is efficient for sophisticated programming structures, it allows other programming languages to be integrated, and it has a good graphical interface. Among the environments developed for R (Integrated Development Environments) is `RStudio` https://www.rstudio.com/ which has a text editor; data editor; etc..

Currently `RStudio` (Rstudio, 2020) has the possibility of using a markup language called Markdown. Markdown makes it possible to make data analyses *reproducible*: the same file contains the code (script), the results of the processing and the corresponding commentary. The library called `knitr` (Yihui Xie, 2013) makes it possible to produce HTML documents in which codes and descriptions can be integrated. It starts with a source file that has the extension `.Rmd`, which is evaluated by the function `knit` and returns an integrated file of text and graphics as output.

R can be defined as a statistical analysis system, which is simultaneously a language and a software or rather an environment. It is an object-oriented language. In fact, each object (e.g. vector, graph, etc.) is treated by the functions of R with a specific method (i.e. a programme code to manage and manipulate data structures) and new methods can be implemented to extend the possibilities of the functions themselves.

One feature of `R` is that it has been *Open Source* since 1995. This characteristic implies among others, that one has the possibility of accessing and modifying the source code; it has source code and to modify it; it has an extensive manual (in language) that can be consulted and downloaded from the Internet. Access via the Internet to a wide range of very detailed libraries for statistical analysis, developed

and made available for other users.

It is possible to count on the support and assistance from the R Development Core Team and all users (a real worldwide community) thanks to the Internet site and the discussion lists, through which one can the help of experts in statistics and computing.

It is distributed free of charge under the constraints of the GPL (General Public License), which is a licence for free software that allows users freedom of use, copying, modification and distribution.

It is based on the language developed by John Chambers and others at Bell Laboratories in 1976 and is available for various hardware architectures and operating systems: Unix, Linux, Windows, MacOS. It can be downloaded from `http://cran.r-project.org` where `CRAN` stands for "Comprehensive R Archive Network". The version to be installed for most common use is the binary version.

Please note that if you encounter programming difficulties, you can ask questions or seek answers at http://stackoverflow.com/questions.

See also http://rseek.org/ for information and examples.

https://pragprog.com/about for guidance on how to program and http://www.r-bloggers.com/. An introduction into basic statistics with R is Dalgaard's Introductory Statistics with R (2008).

Basic syntax for R

The working directory can be chosen by the user. The current working directory is checked with the command `getwd` while with `setwd` one moves between directories. With `dir()` you check the contents of the directory in use.

```
getwd()
setwd("C:/text")
dir
```

Arithmetic expressions can be developed when the symbol > appears on the *console* of R.

To close the session, `q()` is used. If you type enter you are asked whether or not to save the workspace. This workspace contains a record of the commands that were typed during the session and may contain saved results. Generally this is not saved saved, what is saved is the final script, i.e. the text file containing all the instructions to be executed. The editor for creating a script appears from the File menu. The script commands are executed in R with the Ctrl-R command.

Some calculations with R:

```
3^4
## [1] 81
7 * 4 + 2
## [1] 30
31/7
## [1] 4.428571
31%/%7
## [1] 4
```

R has a workspace known as the *global enviroment* which is used to contain the results of the calculations you perform and the objects you generate. For example, if you want to save the result of the calculation 1.0025^{30} you need to assign the value to an object with a name e.g. `int.30`

```
1.0025^30
## [1] 1.077783
int.30<-1.0025^30
```

the assignment is made with an arrow pointing to the left e.g. using the minus symbol followed by the dash. The equal sign is also supported by R but it is better to use the arrow to specify that an action is required instead of establishing a relation. The result of the assignment is not visible except by typing the name of the object, remembering that R is case sensitive: the following code gives an error

```
X <- 1:10
mean(x)
```

When the object for which calculation is required is not defined in the workspace, the following error message is given: `Error in mean(x) : object "x" not found`.

The function `mean()` implemented in R allows the calculation of the arithmetic mean and `:` create a vector

```r
X <- 1:10
mean(X)
## [1] 5.5
```

The command `objects()` or `ls()` displays all the objects of R created objects that are present in the workspace. The `c()` command is used to concatenate the elements that are given as the argument

```r
c(0, 7, 8)
## [1] 0 7 8
x <- c(0, 7, 8)
```

The command is also used to concatenate vectors as follows

“vspace{0.7cm}

```r
num <- 5:20
a.group <- c(num, x)
a.group
##  [1]  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20  0  7  8
```

To display some elements of the vector, square brackets are used, for example the 20th entry of vector `a.group`

```r
a.group[20]
## [1] NA
```

The use of negative numbers serves to avoid considering certain elements, e.g. the second element of the object `a.group`

```
a.group[-2]
##  [1]  5  7  8  9 10 11 12 13 14 15 16 17 18 19 20  0  7  8
```

Operations between vectors are done element by element

“vspace{0.7cm}

```
x * 3
## [1]  0 21 24
y <- x - 5
```

“vspace{0.7cm}

The power-elevation of the type $(y_1^{x_1}, y_2^{x_2}, ...)$ occurs with

```
 x^3
## [1]   0 343 512
```

```
 y^x
## [1]    1  128 6561
```

There are a number of operators that make it possible to generate sequences of integers: `seq` and `rep`. The first command allows the step of the sequence to be specified

```
seq(1, 21, by = 2)
##  [1]  1  3  5  7  9 11 13 15 17 19 21
```

```
rep(3, 12)
##  [1] 3 3 3 3 3 3 3 3 3 3 3 3
```

For example, note the use of the functions `rep` and `seq` to create the following vector

1 2 3 4 5 2 3 4 5 6 3 4 5 6 7 4 5 6 7 8 5 6 7 8 9

```
rep(seq(1, 5), times = 5) + rep(0:4, each = 5)
##  [1] 1 2 3 4 5 2 3 4 5 6 3 4 5 6 7 4 5 6 7 8 5 6 7 8 9
```

To arrange values in a matrix we use the function `matrix()` to create a multidimensional array in which each element has the same mode

```
m <- matrix(1:6, nrow = 2, ncol = 3)
m[1, 2]
## [1] 3
m[4]
## [1] 4
m[1, ]
## [1] 1 3 5
```

Other useful functions for matrix are the following: gives the dimensions of matrix, extracts the diagonal of matrix, extract row 2, column 3 entry of matrix, row sums of matrix A, extract eigenvalues and eigenvectors of a squared matrix eigen , inverse of a matrix for squared matrice

```
dim(m)
## [1] 2 3
diag(m)
## [1] 1 4
m[2,3]
## [1] 6
```

```
rowSums(m)
## [1]  9 12
# eigen(m)
# solve(m)
```

The function `array()` is

```
array(1:24, c(3, 4, 2))
## , , 1
##
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
##
## , , 2
##
##      [,1] [,2] [,3] [,4]
## [1,]   13   16   19   22
## [2,]   14   17   20   23
## [3,]   15   18   21   24
```

The `dump` function is used to save a data file in .R format and `source` to retrieve it.

```
a <- c(2, 3, 4)
dump("a", "use.R")
source("use.R")
```

A data frame corresponds to what is defined as a data matrix or data set. That is, it is a list of vectors or factors of the same length that are related since they correspond to the same experimental unit and there is therefore only one set of row names.

It is possible to extract objects from the data frame in the following way

```
 x <- c(1, 2, 3)
 y <- c(1, 2, 3)
 my <- data.frame(x, y)
my$x
## [1] 1 2 3
```

When the data is structured as a matrix in which each row contains the observations relating to each individual and the columns represent the observed variables resemble the data frame structure of R. The first row may contain the names of the variables and the first column the labels assigned to the observations. For instance, if the data set is written in text format and is located in the test.dat file in the C directory then it is read by R as

```
test.df <- read.table("c:/test.dat", header = T)
```

If file contains header this is specified with `header=TRUE` if it does not contain it `header=FALSE`. To work directly with the columns of the loaded data using `attach`. The loaded data can also be viewed with the editor in the Edit window.

Hints on logical operators and functions

The TRUE and FALSE symbols are the results of the logical condition being set. For example
relational operators are

```
a <- c(3, 6, 9)
a > 4
## [1] FALSE  TRUE  TRUE
```

Note that the logical condition returns a true or false vector which passed to the function returns the following, and `which` allows to select as well indices of `x` such that entry exceeds 10

```
x <- c(3, 6, 9, 10, 20, 30, 21, 2, 1, 4)
x > 10
##  [1] FALSE FALSE FALSE FALSE  TRUE  TRUE  TRUE FALSE FALSE FALSE
which(x > 10)
## [1] 5 6 7
```

That is, `which` returns the indices of a vector vector whose elements contain TRUE or FALSE.

Other uses of `which` are as follows

```
x <- c(3, 6, 9, 10, 20, 30, 21, 2, 1, 4)
which((x > 2) & (x < 20))
## [1]  1  2  3  4 10
z <- which((x < 2) | (x > 7))
z
## [1] 3 4 5 6 7 9
x[z]
## [1]  9 10 20 30 21  1
```

Then the elements of the vector that satisfy the condition.

Note also the other oprators

```
a == 4
## [1] FALSE FALSE FALSE
a >= 4
## [1] FALSE  TRUE  TRUE
a != 4
## [1] TRUE TRUE TRUE
a[a > 4]
## [1] 6 9
```

the first == selects indices of `a` such that entry equals 4.

```
b <- c(4, 6, 8)
a < b
## [1]  TRUE FALSE FALSE
a[a < b]
## [1] 3
```

means {not a}

```
a <- c(TRUE, FALSE, TRUE)
!a
## [1] FALSE  TRUE FALSE
```

Help

To access the help of R there are many options. When you know the name of the function for which you want to consult help, just type

```
`?`(mean)
 help(mean)
```

a new window is opened which contains the full description of the command and at the bottom a number of useful examples of how to use the function are given.

Another possibility is the `example()` function:

```
 example(mean)
##
## mean> x <- c(0:10, 50)
##
## mean> xm <- mean(x)
##
## mean> c(xm, mean(x, trim = 0.10))
## [1] 8.75 5.50
```

all possible uses of the function `mean` are shown.

The option allows us to calculate the truncated mean, i.e. calculated without taking into account the extreme values of the distribution. The value of `trim` is between 0 and 0.5 and consists of the proportion of observations that R discards before averaging in each tail of the distribution.

When the name of the function is not known, it is convenient to use `help.start()`. This opens a browser page in which there is a menu with some options. Another function is `help.search()`, for example, `help.search("mean")` will open a list with various documentation (including help pages) which contain the term in either the

name, alias, title, concept or keyword entries. The `RSiteSearch()` open a brower window with search results.

Programming

Programming consists of writing more or less complex instructions with a well-defined syntax. The two main ways of programming are the imperative one, used for example by R, which consists of stringing together instructions that specify the things to be done by the computer; the declarative one, used for instance for WEB pages such as HTML, which consists of writing a description of the final result without specifying how this is to be achieved. The imperative method in R is called procedural, i.e. it involves describing each step required to achieve a certain goal. Among the basic constructs of a programme are the following tests that allow the verification of certain conditions in the flow of the algorithms.

- Some general programming rules are as follows:
- [1] understand the argument well;
- [2] make the idea of solving the problem explicit;
- [3] translate the idea into implementation;
- [4] check: if it works, if it is efficient, if not go back to the second point.
- The loop `if()` is a conditional construct that allows you to control which operations are executed. The syntax is as follows

if (condition) {commands when the condition is true} or *if (condition)* {commands when condition is true} *else* {commands when it is false}}.

For example

```
x <- 3
if (x > 2) y <- 2 * x else y <- 3 * x
 y
## [1] 6
```

Logical values are converted to numeric values according to the rule that FALSE becomes 0 and TRUE becomes 1.

- The iterative loop `for()` allows us to specify that a certain operation is to be repeated a predefined number of times. The syntax is as follows

`for(variable to values in a set) {execute commands}`.

For example

```
for (i in 1:9) print(1:i)
## [1] 1
## [1] 1 2
## [1] 1 2 3
## [1] 1 2 3 4
## [1] 1 2 3 4 5
## [1] 1 2 3 4 5 6
## [1] 1 2 3 4 5 6 7
## [1] 1 2 3 4 5 6 7 8
## [1] 1 2 3 4 5 6 7 8 9
```

In the example, it was printed what the for loop deals with: to consider the numbers from 1 to i with i varying in the set $[1, 9]$.

Example

Consider the Fibonacci succession which is a well-known sequence of natural integers natural integers in which the first two elements are defined as $[1, 1]$ and the subsequent are the sum of the previous two elements. For example, the third element is 2 (= 1+1), the fourth element is 3 (= 1+2). In R to obtain the first 12 Fibonacci numbers we use the following syntax

```
Fibonacci <- numeric(12)
 Fibonacci[1] <- Fibonacci[2] <- 1
 for (i in 3:12) Fibonacci[i] <- Fibonacci[i - 2] Fibonacci[i -1]
```

The null vector `Fibonacci` is created. The numbers 1 are inserted in the first two positions of the vector. With the loop, the elements from the third onwards are determined. As can be seen, it is necessary to initialise the vector containing the results of the operation.

- The `while()` loop enables certain operations to be repeated as long as the set condition holds. When the condition is no longer valid, the loop is exited: if the condition is evaluated as false no action is no action is performed.
  The syntax is as follows

\texttt{while (condition) execute commands }.
If it is evaluated as true the commands are executed and the process is repeated by re-evaluating the condition each time.

If you want to write a Fibonacci sequence up to 300 numbers you can use the `while` loop to set this condition. For example

```
Fib1 <- 1
Fib2 <- 1
 Fibonacci <- 1
 while (Fib2 < 300) {
     Fibonacci <- c(Fibonacci, Fib2)
     oldFib2 <- Fib2
     Fib2 <- Fib1 + Fib2
     Fib1 <- oldFib2
 }
 Fibonacci
##  [1]   1   1   2   3   5   8  13  21  34  55  89 144 233
```

As long as `Fib2` is less than 300 the vector `Fibonacci` is increased. `Fib2` is updated by adding `Fib1` which in turn is updated with the previous value of `Fib2`.

Functions

Functions are units constructed from elementary commands. In general, they are objects that receive input, perform calculations and produce a result.

Functions are objects that can be modified. In `R` the function has the following structure:

`function` (arguments) {execute commands }.

The word function is followed by round brackets enclosing the list of arguments and then the curly brackets that contain a command or a sequence of commands. Each function has its own name by which is called.

The function in the following example has the name `Eratostene`. This function takes as its argument `n` which specifies the upper limit upper limit of the sequence of numbers of the vector `description`. The arguments of the function may be multiple. It is also possible to assign default values to some arguments such that if when using the function, you do not specify their value is considered the default value.

Example

Eratosthenes' sieve is an ancient procedure for calculating tables of prime numbers up to to a certain fixed number n. It is still used today as an algorithm for calculating prime numbers by many computer programmes; although it is not an extraordinarily efficient algorithm in fact, it is on the other hand rather simple to translate into any programming language. One writes down all the naturals from 2 up to n in a list. Then you delete (sieve) all multiples of the first number in the sieve (excluding itself). E.g. starting with 2 you delete the multiples of 2. You move to the next 3 and delete its multiples. One continues in this way until one reaches the bottom. The numbers that remain are the prime numbers less than or equal to n.

```
Eratosthenes <- function(n) {
     if (n >= 2) {
        sieve <- seq(2, n)
         prime <- c()
         for (i in seq(2, n)) {
             if (any(sieve == i)) {
                 prime <- c(prime, i)
                 sieve <- c(sieve[(sieve%%i) != 0], i)
             }
         }
         return(prime)
     }
     else {
         stop("the initial value must be at least 2")
     }
 }
 Eratosthenes(50)
##  [1]  2  3  5  7 11 13 17 19 23 29 31 37 41 43 47
```

The sieve vector contains the list of numbers to be evaluated i.e. the numbers integers

from 2 to n. The object `prime` is an empty object created to contain the vector of the prime numbers to be found.

Each integer `i` in the sequence is checked to see if it is back in the vector via the condition `if`. The function `any()` gives a TRUE result if at least 1 of the logical elements is true. If the element is true, it means that that element is still in the vector `direction`.

In the body of the function there may be a command `return(value)` that specifies and prints the result of the function. In `R` each function produces only one output. If the return is not specified then the value of the last command in the body of the function is printed.

In the example above, internal local objects were created to the function used as: `n`, `sieve` and `primes` which are local objects internal to the function and then called the functions: `seq`, `c`, `any`, `return` and `stop` that are part of the environment of R.

To correct errors that may occur when writing the function, use the command `fix()`. This command opens the text of the function in a text file. function.

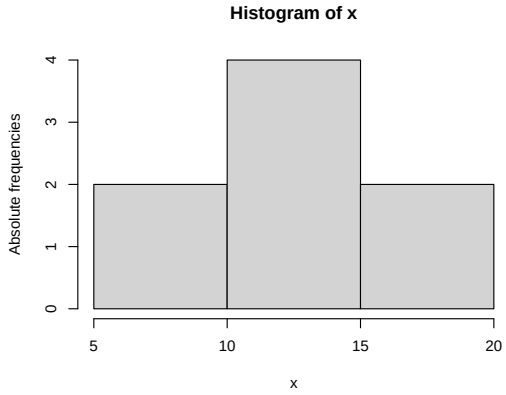
To insert comments and descriptions in the function, use `#` so that `R` ignores all text that follows (when using the .R syntax file) deletion to the end of the line.

Graphical representations

The histogram is a tool for graphically representing data of continuous quantitative phenomena.

```
 x <- c(12, 15, 20, 14, 16, 10, 8, 15)

a <- hist(x,
     main = 'Histogram of x',
     ylab = "Absolute frequencies")
```



```
a
## $breaks
## [1]  5 10 15 20
##
## $counts
## [1] 2 4 2
##
## $density
## [1] 0.05 0.10 0.05
##
## $mids
## [1]  7.5 12.5 17.5
##
## $xname
## [1] "x"
##
## $equidist
## [1] TRUE
##
## attr(,"class")
## [1] "histogram"
```

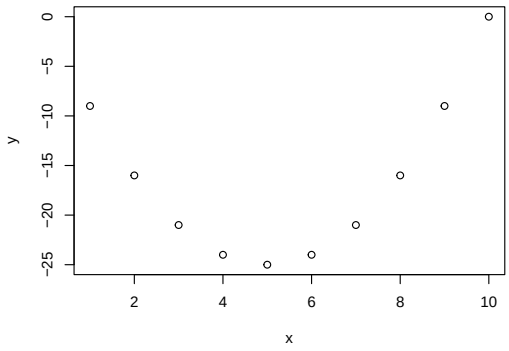
Histograms have bases proportional to class extremes and height of the rectangles proportional to the frequency density (height/base). With the command `hist()` R draws the histogram in histogram automatically. The object named `a` contains the objects `breaks` which represent the extremes of the intervals, `counts` are the absolute frequencies, `mids` are the midpoints of the intervals, `xname` is the variable name, `equidist` is a logical variable that is only TRUE if all intervals have the same amplitude. The `intensities` represent the frequency densities associated to each point in the corresponding interval. The default options can be modified, e.g. the number of classes with the following syntax

`hist(W, break=11, main= "11 classes")`.

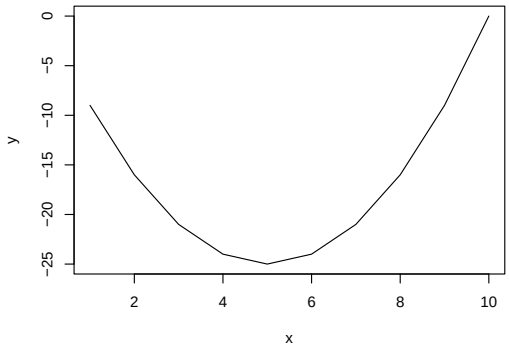
There are various rules of thumb for determining the number of intervals named after the authors who proposed them. R implements the Sturges' rule that having n values of x divides the range of variation of the data by $\log_2(n)+1$ intervals, in practice we have $\frac{x_{(n)}-x_{(1)}}{\log_2(n)+1}$.
If n is large, the method involves a number very large number of classes. Other options are `breaks = "Scott"` and `breaks = Freedman-Diaconis`.

The scatter plot or scatter diagram is generated as follows.

```r
x <- seq(1, 10)
y <- x^2 - 10 * x
plot(x, y)
```

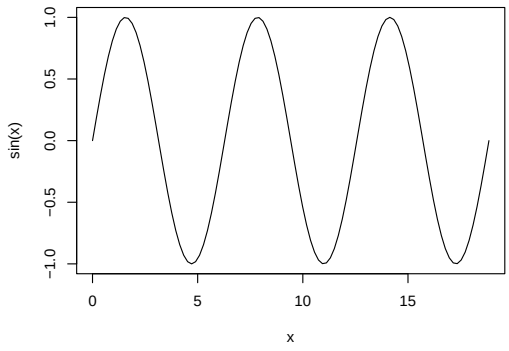



```r
plot(x, y, type = "l")
```



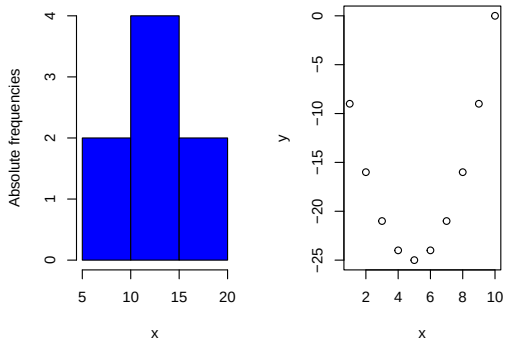
Another important function is `curve` which is used to plot the graph of a univariate function over a certain interval. The initial and final points of the interval are specified by the arguments `from` and `to`. For example, if we wish to graph the sine function x between $[0, 6\pi]$

```r
curve(sin, from = 0, to = 6 * pi)
```



To draw several graphs together in the same graphics window, use the following option which allows you to to choose the number of graphs to insert, for example with `par` you tell R you want 2 side-by-side plots (a 1 by 2 array of plots)

```
 par(mfrow = c(1, 2))
x <- c(12, 15, 20, 14, 16, 10, 8, 15)
hist(x,
     main = "",
     ylab = 'Absolute frequencies', col = "blue")
x <- seq(1, 10)
y = x^2 - 10*x
plot(x, y)
```



Other useful command for plots are the following: add the points (x_i, y_i) to the plot, adds line segments through the (x_i, y_i) to the plot, adds the line with intercept a , slope b to the plot and tells R not to clear the palette when we make our next plot

```
points(x,y)
lines(x,y)
abline(a,b)
par(new=TRUE)
```

Cleaning environment

The following creates an empty environment

```
rm(list = ls())
```

Libraries

Libraries consist of a set of functions already implemented in R. They allow common operations to be performed within the of the application for which they were created. From the prompt of R you can request the installation of additional libraries.

Accessing the Internet checks the contents of CRAN and update the available packages. Once installed, simply from the prompt use the following command to make it available in R

```
library(boot)
library(LMest)
```

To obtain instructions on the contents of the library, one uses

```
library(help = boot)
library(help = LMest)
```

The function `apropos` allows you to look up information about an object of R or a function using certain letters of the word.

```
data(Orange)
apropos('Oran')
## [1] "Orange" "Orange"
head(Orange)
##   Tree  age circumference
## 1    1  118            30
## 2    1  484            58
## 3    1  664            87
## 4    1 1004           115
## 5    1 1231           120
## 6    1 1372           142
```

To quote R you can write

```
citation()
## To cite R in publications use:
##
##   R Core Team (2025). _R: A Language and Environment for Statistical
##   Computing_. R Foundation for Statistical Computing, Vienna, Austria.
##   <https://www.R-project.org/>.
##
## A BibTeX entry for LaTeX users is
##
##   @Manual{,
##     title = {R: A Language and Environment for Statistical Computing},
##     author = {{R Core Team}},
```

```
##     organization = {R Foundation for Statistical Computing},
##     address = {Vienna, Austria},
##     year = {2025},
##     url = {https://www.R-project.org/},
##   }
##
## We have invested a lot of time and effort in creating R, please cite it
## when using it for data analysis. See also 'citation("pkgname")' for
## citing R packages.
```

The libraries can be viewed from the Internet at http://rm.mirror.garr.it/mirrors/CRAN/ following Packages, Table of available packages.

For more on R syntax, see *An Introduction to R*. https://cran.r-project.org/doc/manuals/r-release/R-intro.pdf.

Learning objectives

This is an *introduction* to the use of the interface R `Rmarkdown`. You should be able to:

- Create a new file with extension `Rmd` where you write the text and the code in specific blocks;
- Compile the document in different formats and store all the files in a unique folder;
- Extract the R code in a file with extension .R containing only the R code.

Introduction to R `Markdown`

The R `Markdown` workflow is an authoring framework for data science. The team at RStudio developed it to connect data and run code and generate reproducible reports of your work directly in the R environment.

R `Markdown` serves as a tool to facilitate the seamless integration of operations, where one file contains R syntax, output (statistics, tables, and figures), and text associated with the final report. The advantage of this approach is transparency and reproducibility. For more information see: https://rstudio.github.io/cheatsheets/html/rmarkdown.html.

A package (library) is just a collection of functions, data sets and help pages. The following libraries are required

```
install.packages(c("knitr", "rmarkdown"))
library("knitr")
library("rmarkdown")
```

Citation of a library can be obtained with `citation` and name of the library between inverted commas:

```
citation("knitr")
## To cite package 'knitr' in publications use:
##
##   Xie Y (2025). _knitr: A General-Purpose Package for Dynamic Report
##   Generation in R_. R package version 1.50, <https://yihui.org/knitr/>.
```

```
##
##   Yihui Xie (2015) Dynamic Documents with R and knitr. 2nd edition.
##   Chapman and Hall/CRC. ISBN 978-1498716963
##
##   Yihui Xie (2014) knitr: A Comprehensive Tool for Reproducible
##   Research in R. In Victoria Stodden, Friedrich Leisch and Roger D.
##   Peng, editors, Implementing Reproducible Computational Research.
##   Chapman and Hall/CRC. ISBN 978-1466561595
##
## To see these entries in BibTeX format, use 'print(<citation>,
## bibtex=TRUE)', 'toBibtex(.)', or set
## 'options(citation.bibtex.max=999)'.
```

Here there is a simple plain text file with three types of contents: chunks to run, text to display and metadata to guide the R `Markdown` build process. The code can be written in R or other languages such as `Python`.

To render the document, use the `render("filename.Rmd")` function in the console with the filename as the argument, or click the `Knit` bottom with the ball of wool in the `Rstudio`, and the R `Markdown` export your file into the chosen format, running your code and showing the results. The file remains a reproducible record of your work. There are many formats available, such as HTML, PDF, and Word.

Tips

Some recommendations are the following:

- Source files must be allocated in the same directory.
- A default **working directory** should be chosen and used where the **source document** is located at the start of the job and maintained throughout the job.
- The data files to be used must be in the same folder as the source file, and the working directory must be located in this folder.
- Always start by restarting a new session of `Rstudio` and compiling the source document after opening the new session of R.
- Even if the code or document is not perfect, it is better to share it with the instructor than not to share it at all!

Usage

Check that the source file has the extension `.Rmd`.

- From the menu window select File > New File > R Markdown
- Choose PDF, Word, or HTML format and click OK.

File

The file created is a text-only file (perfectly readable by other programmes) where some basic text formatting are indicated by special characters.

When you open the file in the RStudio IDE, it becomes a notebook interface for R. You can run each code chunk by clicking the icon. RStudio executes the code and display the results in line with your file.

For example the following syntax produce:

```
# Title

## Subtitle

### Sub-subtitle

**text in bold**

*text in italics*

- Bulleted list
- Bulleted list
  + with others
  + nested elements
```

Code

The R code and only this code must be included in **special blocks** called **'chunks'**.

While the **text** comprising the **comments** to the code, the description of the chuck, of the data and of the results should be included *outside* the chunk.

The chunk

The chunk is delimited by `{ }`.

The part between the curly brackets, `{r echo = TRUE}`.

indicates the R-language code to be shown (`echo`).

To show the code without executing it, use `eval = FALSE`.

The possible options for code chunks are as follows:

- `include = FALSE` prevents code and results from appearing in the finished file. R Markdown still runs the code in the chunk, and the results can be used by other chunks.
- `echo = FALSE` prevents code, but not the results from appearing in the finished file. This is a useful way to embed figures.
- `message = FALSE` prevents messages that are generated by code from appearing in the finished file.
- `warning = FALSE` prevents warnings that are generated by code from appearing in the finished. fig.cap = "..." adds a caption to graphical results.

For additional details see the following website: https://rmarkdown.rstudio.com/lesson-1.html and the files *MarkdownCheatsheet.pdf* and *rmarkdown-reference.pdf* provided in the elearning page of the course.

Use the cheatsheets that are built into the RStudio IDE to jog your memory about R Markdown: help > cheat sheet > R markdown cheatsheets. Go to File > Help > Markdown Quick Reference to open the Markdown Quick Reference in your help pane.

Note that `knitr` can execute code in many languages besides R.

Echo and Eval

One of the things that make Markdown particularly convenient is the ability to include graphics generated by the code. These are reported following the code itself.

Example

In the following example, the chunk contains the code where a pseudo random number is drawn from a Uniform distribution after fixing the seed. Through the conditional construct we print the result like a random experiment of tossing a coin

```
set.seed(234)
uniforme <- runif(1); uniforme
## [1] 0.74562
if(uniforme > 0.5){
```

```
  print("The result of the coin toss is head")
  } else {
  print("The result of the coin toss is cross")
  }
## [1] "The result of the coin toss is head"
```

Automatically R executes the code and enters the results in the report, saves the file in the working directory. Sometimes the preview file is opened automatically.

Code extraction

It is possible to obtain a file with extension .R reporting only the code contained in the Markdown file with the following `purl` function

```
knitr::purl("Stat1.Rmd",
            "Stat1.R",
            documentation = 1)
```

Further documentation can be found at: https://rmarkdown.rstudio.com/ and at `blog.rstudio.com`..

Please also refer to the tutorial available at: https://www.rstudio.com/resources/webinars/programming-part-3-package-writing-in-rstudio/.

Learning objectives

- Generate reproducible documents showing data, code, and results of exploratory data analyses.
- Read the data into R and define the data format.
- Calculate, report, and comment on descriptive statistics for quantitative variables.
- Create graphical representations of the data and comment on each figure.
- Calculate and comment on measures of association between variables.
- Learn to describe, in concise, precise, and comprehensive written English, the procedures implemented and the results obtained.

Case study: Credit rating

Standard and Poor's is among the three biggest American credit rating agencies along with Moody's Investors Service and Fitch Ratings. S&P has its origins in 1860, but it was formed by the merger of Standard Statistics and Poor's Publishing Company in 1941. An analyst assesses firms, and many factors are combined into **ratings**. The Standard and Poor's rating framework is divided into business analysis and financial analysis. For financial analysis financial risk is splitted into different types of risks, such as Sovereign risk and business risk.

The following sample data (*simulated data from real data of the credit rating agencies*) are related to financial risk. Credit rating of a sample of firms of the same sector is considered (`rating`) assigned by an analyst on a certain scale.

We are interested in *understanding* if:

- The rating is associated with other variables, defined as covariates (see the theory notes) when estimating a statistical model.
- The expected variability of the ratings changes according to a one-unit change in the covariates for the reference population of firms; that is, estimate the regression parameters in a linear model.
- The rating of a firm can be predicted out of sample using the estimated statistical model given a set of covariate values.

The covariates are:

- level of profitability (`profit`),
- capital structure (economic capital referring to additional assets or cash flows to cover unexpected events, `capital`),
- financial flexibility (`f_flex`) relates to the extent to which a firm can alter its funding sources in times of financial stress.

Exploratory analyses

Characteristics of the observations:

- 30 firms are assigned a rating: *sample units*
- firms of the same industry: *reference population*
- *observational study* referred to values collected in a certain reference period (e.g., last operating year - cross-sectional data)
- the variables are quantitatively measured on a different scale, they ara continous variables
- inference:
- understanding the main determinants of the rating for the whole population;
- explain the association with the covariates for the entire population;
- provide a point and interval estimation of the effects of covariates;
- make a prediction and associate uncertainty to this prediction measuring on the possible ratings obtained for a new firm (out of sample) according to certain values of profitability, capital structure, and financial flexibility.

Univariate summaries

The first task is to *explore the data*:

- examining the response and covariates through graphical and numerical summaries
- This is an essential task before any model fitting. First, we examine each variable separately.

Consider that later on, we are interested in considering rating as a response and the other as covariates in order to estimate a *multiple linear regression model*. However, at this stage, we examine each variable separately.

It is recommended to create a folder dedicated only to data and markdown files.

Set the working directory to the folder where the files are located.

Data are loaded in the current workspace

```
load("ratings.Rdata")
```

Dimensions are

```
dim(ratings)
## [1] 30  4
```

The first six rows of our data set are printed

```
head(ratings)
##   rating profit capital f_flex
## 1   12.3  4.543   3.135   0.86
## 2   20.9  5.159   5.043   1.53
## 3   39.0  5.366   5.438   1.57
## 4   47.9  5.759   7.496   1.81
## 5    5.6  4.663   3.807   0.99
## 6   25.9  5.697   7.601   1.09
```

It is a dataframe (data matrix) of 30 rows representing the different companies and columns representing the measured variables.

The observation in the first row and first column represents the score assigned by the rating agency to the first company. The company's characteristics are as follows: profitability equal to 4.5, capital structure 3.1, and financial flexibility 0.86.

Using the `skimr::skim` function of the `skimr` library, it is possible to explore the data. Continuous variables are summarized using statistics indicating the central tendency of the data and the spread or variability of the data around a chosen center

```
require(skimr)
skim_without_charts(ratings)
```

Table 1: Data summary

Name	ratings
Number of rows	30
Number of columns	4
Column type frequency:	
numeric	4
Group variables	None

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100
rating	0	1	24.53	16.26	0.70	13.55	20.95	36.70	57.20
profit	0	1	5.50	0.57	4.48	5.24	5.42	5.88	6.46
capital	0	1	5.94	2.13	3.00	3.98	5.33	7.57	10.20
f_flex	0	1	1.44	0.30	0.86	1.25	1.45	1.67	2.01

Note this function avoids histograms to be depicted.

We see that there are no missing values.

The observed **average** score is 24.5. *Remember* when n is small the arithmetic mean can be highly influenced by observations that fall well above or well below the rest of the data (extreme values).

The **range** of variation of the score is wide, the minimum score being 0.7 and the maximum score 57.2.

The **median** shows that half of the evaluated cheeses received a score less than or equal to 21.

Remember that it is the middle value being larger than 50% of the other values and smaller than 50%. If the distribution is highly skewed this is to be preferred as summary statistics of the center.

A quarter of the companies scored above 36.7 (**third quartile**).

The three **explanatory variables** have a rather different range of variation, with the largest one referring to capital structure, which in the sample varies from 3 to 10.

The profitability found ranges from 4.5 to a maximum of 6.5. The average capital structure is about 6, while on average, the companies have financial flexibility of 1.4.

It is interesting to note the values of the company that was rated as most reliable

```
which.max(ratings$rating)
## [1] 12
ratings[12,]
##    rating profit capital f_flex
## 12   57.2  6.446   7.908    1.9
```

It is the one with above-average values of the covariates profitability, capital structure, and flexibility.

Variability indices

The difference between the largest and the smallest observations is called **range**. It is sensitive to extreme observations.

The mean square deviation, said **standard deviation** measures the average distance between the sample values from the arithmetic mean. **It is always a non negative value and the larger the standard deviation the greater the spread of the data**.

Its magnitude reflects the shape of the frequency distribution: if the distribution is almost bell-shaped then we have the following rules according to the arithmetic mean $\bar{x}$ and the standard deviation σ

- about 68% of the observations fall between $\bar{x} - \sigma$ and $\bar{x} + \sigma$
- about 95% of the observations fall between $\bar{x} - 2\sigma$ and $\bar{x} + 2\sigma$
- all or nearly all observations fall between $\bar{x} - 3\sigma$ and $\bar{x} + 3\sigma$

Chebyshev in 1867 proved that for any $k \geq 1$ the proportions of observations that fall at least k standard deviations from the mean can be no greater than $1/k^2$ (Chebyshev's inequality).

The **sample standard deviation** of the response variable is equal to 16, i.e. the average variability of the assigned score around the average score (of 24.5) is about 16 points.

To describe the variability we also use **percentiles**: the $(100\text{p})th$ percentile, $0 < p < 1$ is a point π_p such that

$$P(X \leq \pi_p) = p \quad and \quad P(X > \pi_p) = 1 - p.$$

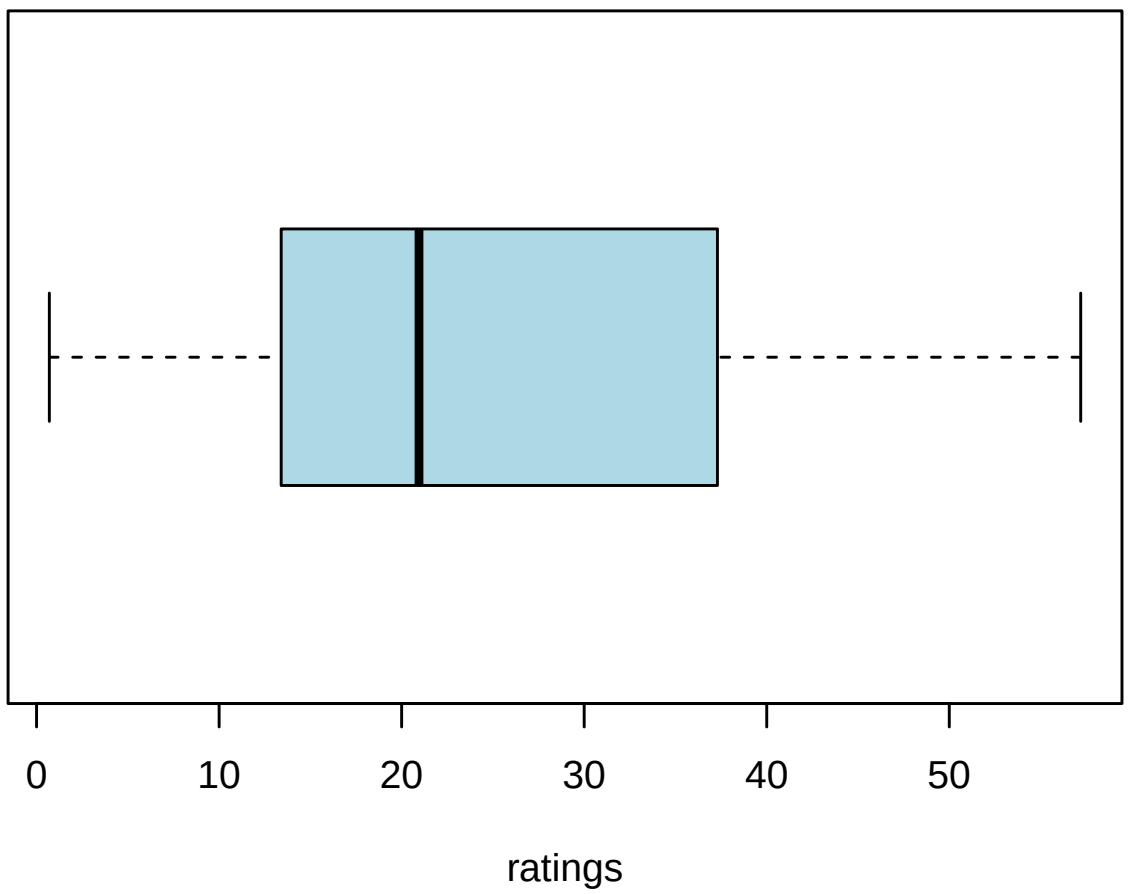
The most important percentiles are the **median** a point such that at most, half the population have values less than the median, and at most half have values greater than the median: Q_2, and Q_1, Q_3 are called the first and the third quartiles, respectively.

Visualization

Boxplot

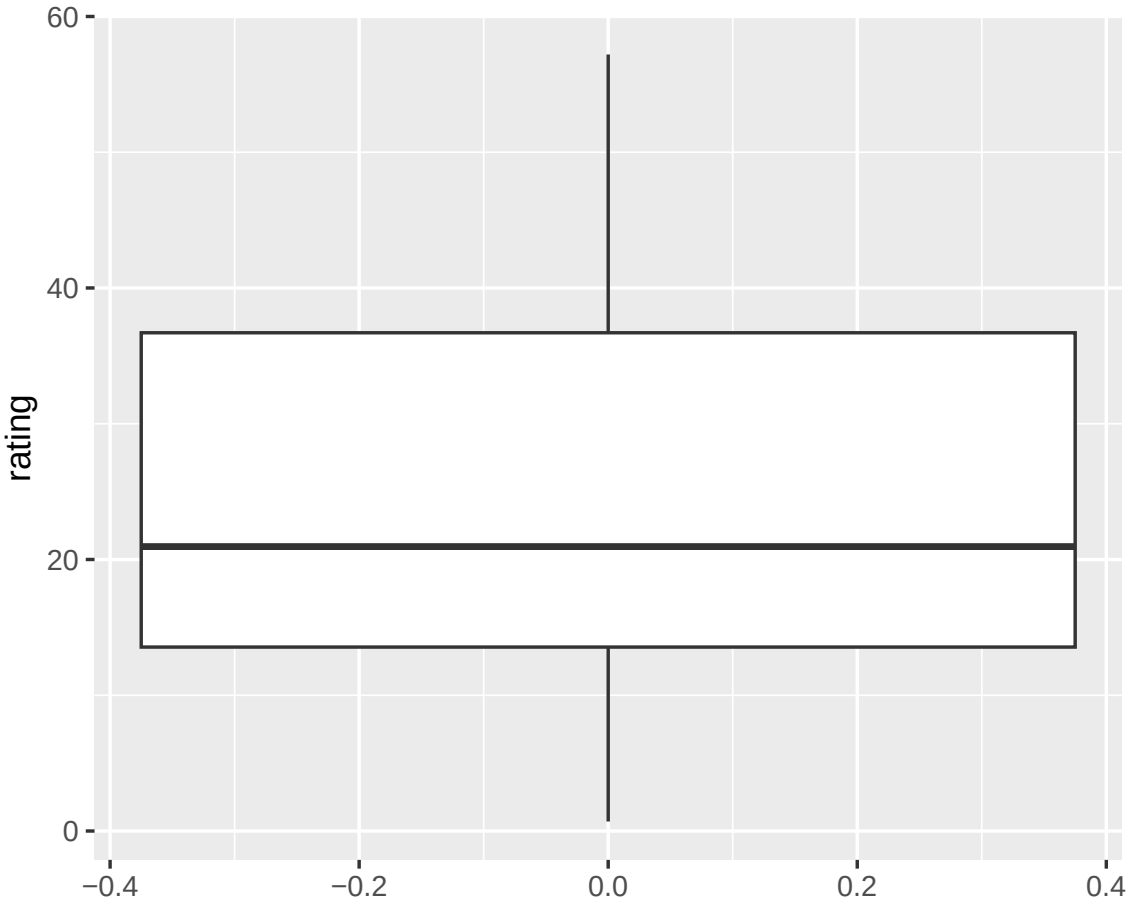
Tukey boxplots provide another way of looking at the distributions of continuous variables.

```
boxplot(ratings$rating,
        xlab = "ratings",
        horizontal = TRUE,
        col = "lightblue",
        main = "Box plot")
```



Or with the `ggplot` function

```
require(ggplot2)
ggplot(data = ratings) +
geom_boxplot(aes(y = rating))
```



- the box contains the central 50% of the data from the lower quartile to the upper quartile.
- the dashed lines are the whiskers containing the outer 50% of the data
- the length of the whole central box is the interquartile range that is $Q_3 - Q_1$ (difference between upper and lower quartile)
- observation falling 1.5 interquantile range are marked separately.

We notice that the box on the right is larger than the one on the left, indicating positive asymmetry (right-skewed distribution). This means that a few firms have particularly high ratings, pulling the distribution to the right. However, most companies have relatively low or moderate ratings, with only a few exhibiting very high scores.

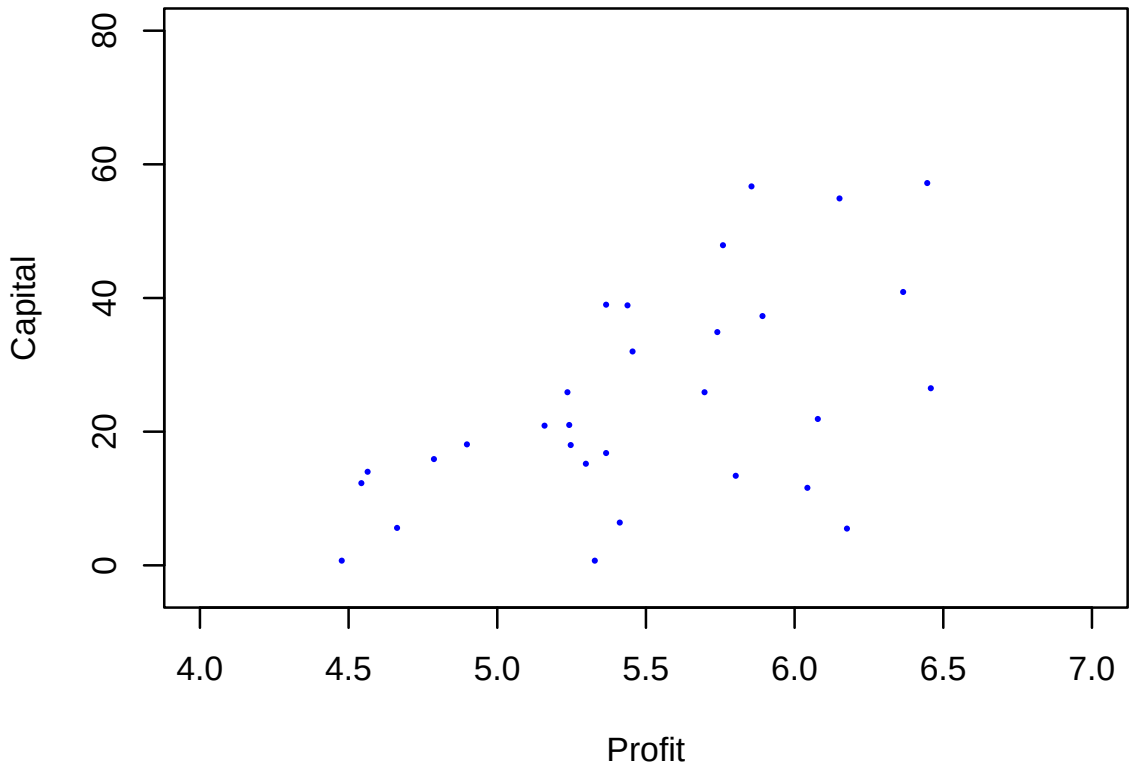
Scatterplot

The scatterplot for two variables is obtained as follows with the `plot` function

Then fix:

- the labels of the axes with `xlab` and `ylab`
- the axis lengths with `xlim = c(0,80)` and `ylim = c(3,7)`
- the colour and shape of the points with `type = "p"` and 'col = "blue"

```
plot(ratings$profit, ratings$rating,
     xlab = "Profit",
     ylab = "Capital",
     ylim = c(-3,80),
     xlim = c(4,7),
     type = "p",
     pch = 16, cex = 0.4,
     col = "blue"
     )
```

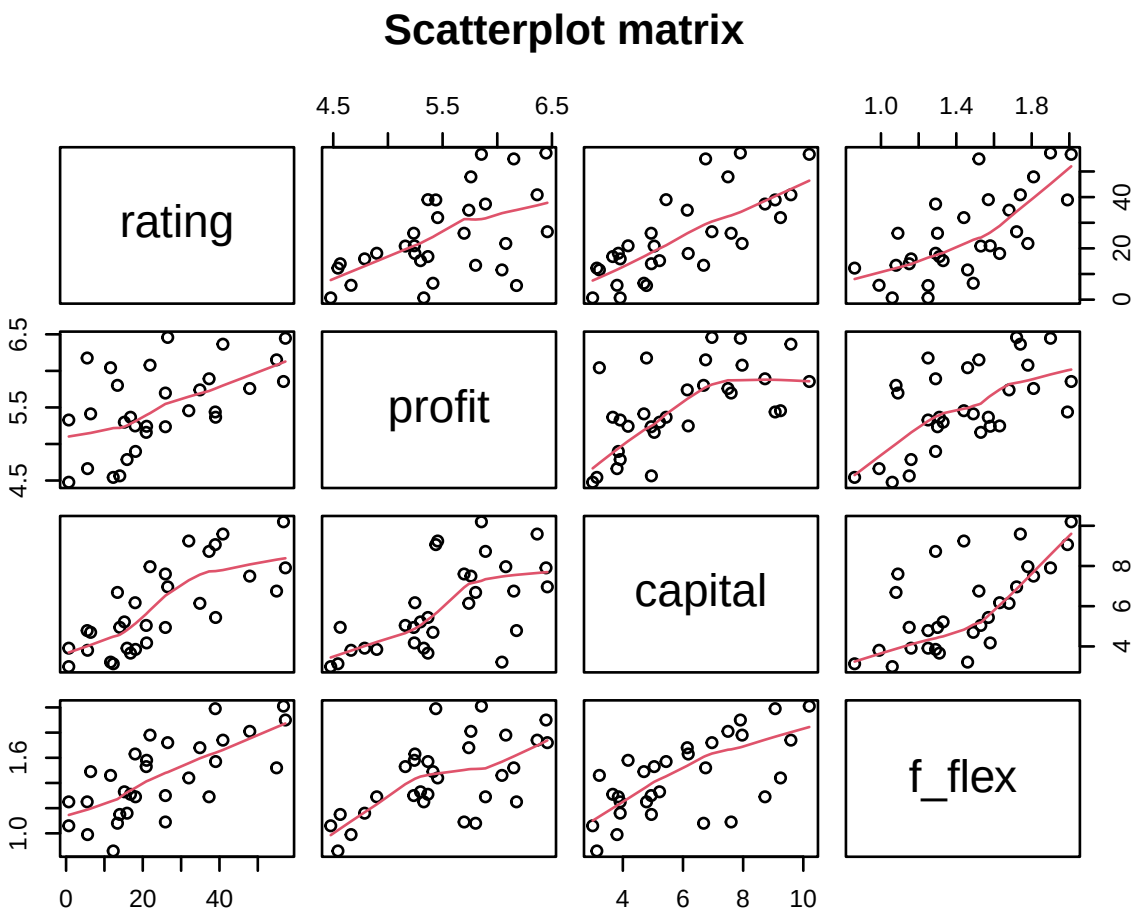


Scatterplot matrix

The scatter plot matrix is obtained with the function **pairs** which requires:

- the name of the dataset
- the **panel.smooth** option to add the display of the estimated trend line (red line)
- **main** to assign the title to the graph

```
pairs(ratings,
      panel = panel.smooth,
      main = "Scatterplot matrix ")
```



Comments may be reffered by raw: for example, the first row of the matrix is related to the response variable.

In the **first row, second column** of the matrix, we can see that the trend line (obtained according to a smooth regression) is linearly increasing and that the range of variation of the score (y-axis) is large. There is great variability of profit since points are more spread over the plane compared to those of the other two variables.

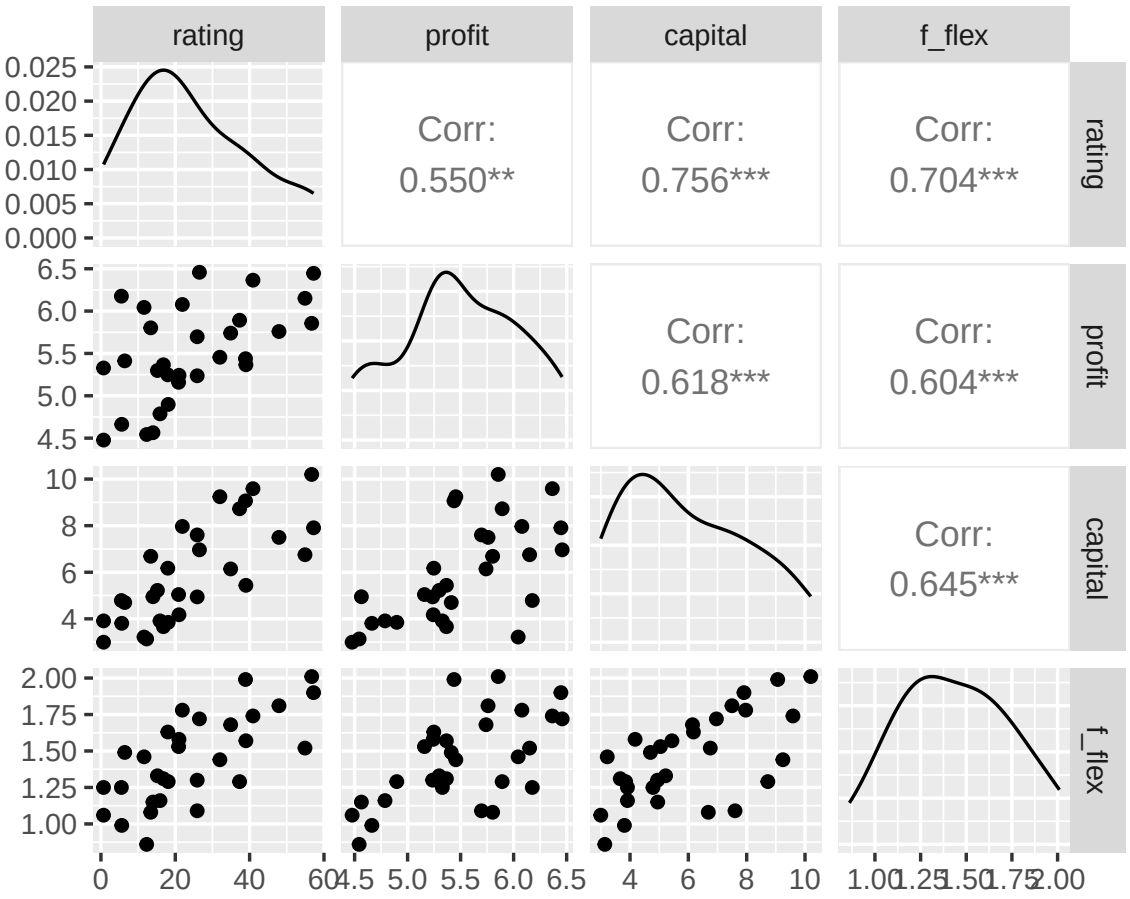
Points in the graphs of the first row and third and fourth column (response variable and capital and financial flexibility) are arranged in a linearly increasing manner, the points are scattered over the range of variation and no points particularly far from the trend line can be observed.

The trend of the points in the graph in the first row and third column is always increasing although it is less linear than in the other two graphs above. Here again, no particularly distant points from the trend line are noticeable.

An increasing linear trend can be seen between the pairs of explanatory variables.

Note also the use of the plot in the library `gamlss`

```
library(GGally)
ggpairs(ratings)
```



The lower triangular cells show scatterplots of the variables, while the upper triangular cells show their correlation coefficients. The diagonal cells display density estimates.

Learning objectives

- Generate values from a standard univariate Gaussian distribution and transform realization into a general normal distribution
- Summarize and describe the observations using the empirical distribution function and the quantiles making comparisons among different shapes.

Generating values from random variables

For each distribution, specified by its parameter values there are four functions implemented in `R`. There is a prefixes for each of the following quantities:

`d` calculate the density at a point;

`p` calculate the cumulative distribution function at a point

`q` calculate a quantile

`r` generate pseudo-determinations from the distribution

For the standard Normal (Gaussian) distribution in the `stats` library we have:

- `dnorm`
- `pnorm`
- `qnorm`
- `rnorm`

Note that this is a two-parameter distribution having *location* (mean) and *scale* (parameter)

The R functions are:

- Normal: base name `norm`, parameters μ and $sigma$.
- Student-t: base name `t`, parameter df;

In the following we consider

Realisations from the univariate Gaussian distribution

In the following, we simulate realizations from the univariate Normal distribution. Simulated values can be generated from this random variable with the function `rnorm`, which requires as input:

- the number of observations to be generated,
- the values of the two parameters referred to the location and scale: mean and standard deviation.

In addition, the option to fix the **seed** can also be always added to generate the same sequence of numbers to ensure reproducibility, it determines which deterministic random sequences start. Use an integer not from 3 to 10 digits long.

The following code simulates one thousand realizations from the Standard Normal distribution.

```
set.seed(7369)
n <- 1000
mu <- 0
sigma <- 1
z <- rnorm(n, mean = mu, sd = sigma)
```

Descriptive statistics

```
require(skimr)
skimr::skim_without_charts(z)
```

Table 3: Data summary

Name	z
Number of rows	1000
Number of columns	1
Column type frequency:	
numeric	1
Group variables	None

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100
data	0	1	-0.05	1.03	-3.4	-0.74	-0.03	0.6	3.9

Linear transformation

The following model is called the **null model** and establishes that: the response variable Y depends linearly only on a constant term (i.e. the mean μ) defined as *signal* and an error term defined as *disturbance*:

$$Y_i = \mu + \sigma Z_i,$$

$$Y_i = \mu + \epsilon_i,$$

with $\epsilon_i = \sigma Z_i$, $i = 1, \dots, n$.

In practice, this is a linear transformation of the variable of a standard normal distribution Z. In this way, the resulting variable Y has a normal distribution with μ and variance σ^2.

Therefore starting from the realisations generated above by a standard normal distribution we can obtain realisations from the following Gaussian distribution $Y \sim N(140, 36)$.

```
y <- 140 + 6*z
skimr::skim_without_charts(y)
```

Table 5: Data summary

Name	y
Number of rows	1000
Number of columns	1
Column type frequency:	
numeric	1
Group variables	None

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100
data	0	1	139.71	6.15	119.58	135.58	139.83	143.58	163.38

Another choice can be $Y \sim N(140, 100)$.

```
y1 = 140 + 10*z
```

```
skimr::skim_without_charts(y1)
```

Table 7: Data summary

Name	y1
Number of rows	1000
Number of columns	1
_____	
Column type frequency:	
numeric	1
_____	
Group variables	None

Variable type: numeric

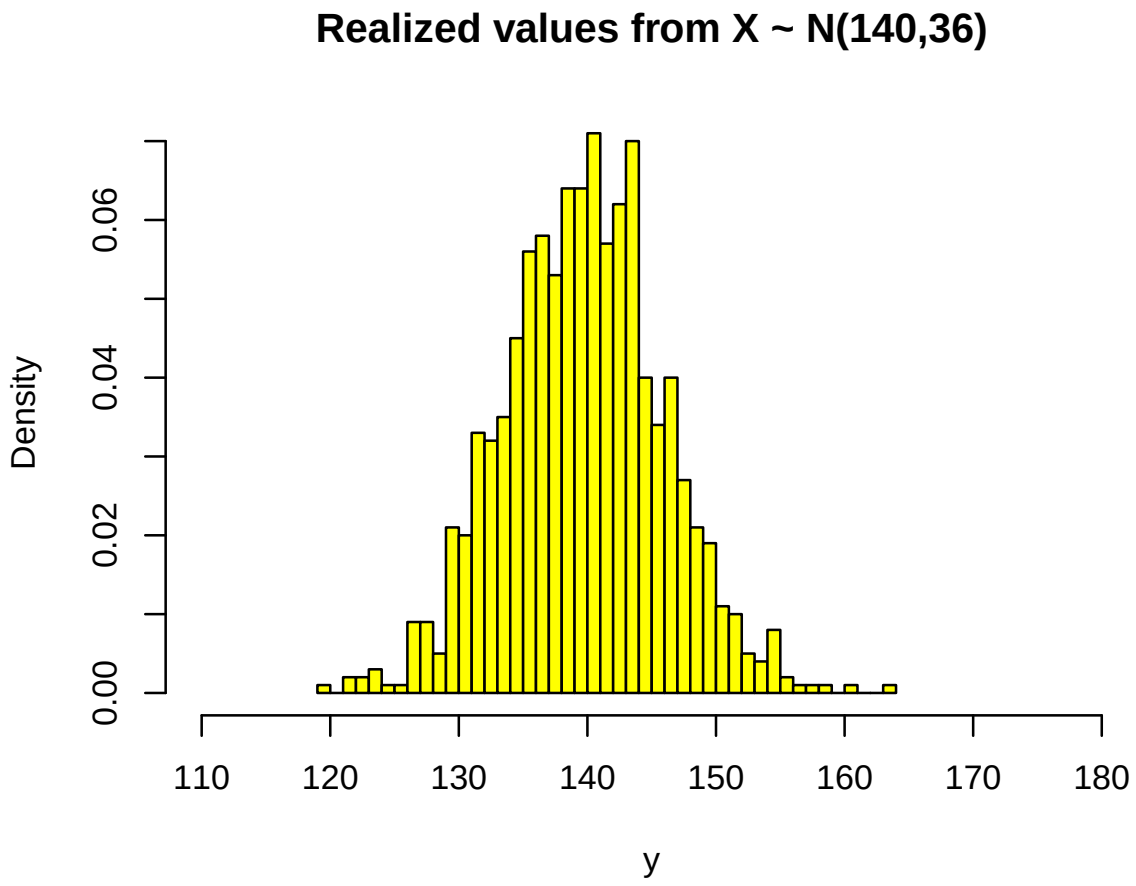
skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100
data	0	1	139.51	10.25	105.97	132.63	139.71	145.97	178.97

Data visualization

The realized values can be represented graphically through an histogram. The `hist` function allow us to change the number of classes and their width. The y-axes reports

the relative frequencies with the option `prob=TRUE`. Note that the default rule to decide the number of classes is called Sturges and classes are of equal length. The number of classes can be changed with the option `breaks` as in the following chunk

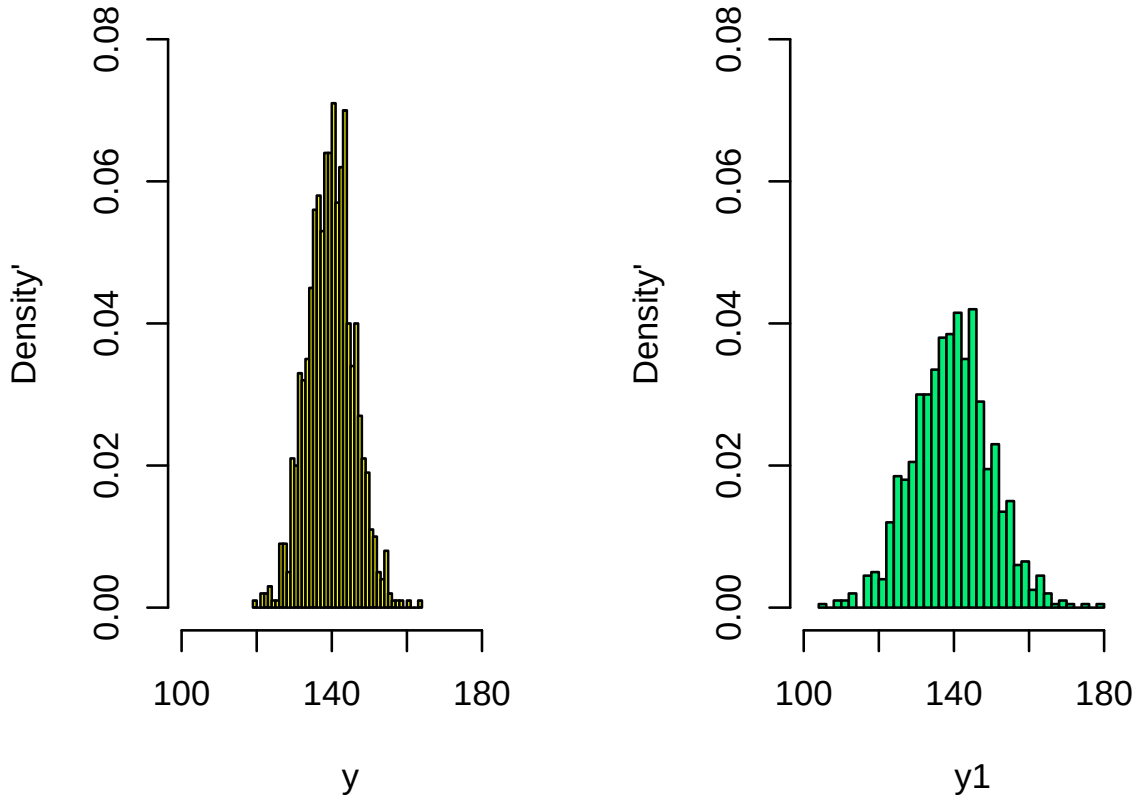
```
hist(y,
     breaks = 50,
     main= "Realized values from X ~ N(140,36)",
     prob=TRUE,
     xlim=c(110, 180),
     ylab = "Density",
     col = "yellow1")
```



The bell shape of the frequencies is informative.

To compare the histograms of the two generated series, they can be arranged in the same graphic window, taking care to fix the range of variation of the y-axis in the same way with the `ylim` option.

```
par(mfrow=c(1,2))
hist(y,50,
main= " ",
prob=TRUE,
ylab = "Density'",
col = "yellow1",
ylim = c(0,0.08),
xlim = c(100,190))
#
hist(y1,50, main= " ",
prob=TRUE,
ylab = "Density'",
col = "springgreen2",
ylim = c(0,0.08),
xlim = c(100,190))
```



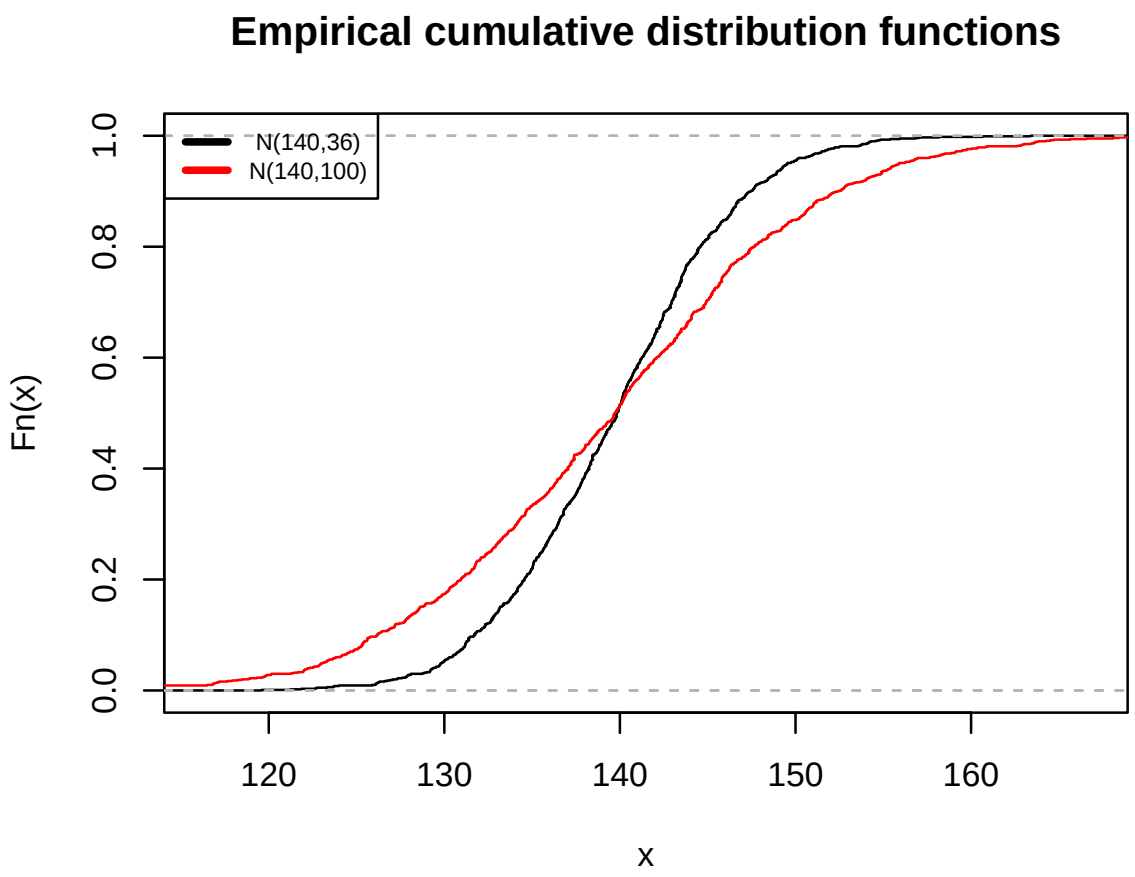
Note that the density around the mean is greater in the left-hand graph (y), while the right-hand distribution has greater variance (y1).

Empirical cumulative distribution function

The sample proportion of the n observations that fall at or below a certain value x is represented graphically through the *empirical cumulative distribution function*.

Note that the **legend** has been added to the graph to illustrate the difference of the two curves appearing in the graphical window

```
plot(ecdf(y), main = "Empirical cumulative distribution functions")
plot(ecdf(y1), add=TRUE, col="red")
legend("topleft",
       c(" N(140,36)","N(140,100)"),
       lty=c(1,1),
       lwd=c(3,3),
       col=c("black", "red" ),
       cex= 0.7)
```



The function Φ represents the cumulative distribution function of the standard Gaussian random variable and the corresponding function is `pnorm`.

Note that if $X \sim N(\mu, \sigma^2)$ then

$$P(a < X < b) = \Phi\left(\frac{b-\mu}{\sigma}\right) - \Phi\left(\frac{a-\mu}{\sigma}\right).$$

Probability and quantiles of a probability distribution

Function `pnorm` provides the cumulative probability: $P(Y \leq x = 130)$ requiring as input the value x and the values of the parameters of the distribution

```
round(pnorm(130, mean= 140, sd=6),4)
## [1] 0.0478
```

The pth quantile is the point q at which $F(p) = p$.

The quantile 0.975 is the point q at which

$$P(Y \leq q) = 0.975$$

.

The point q is determined by $q = F^{-1}(p)$ where F^{-1} is the inverse function that maps for the possible p values between 0 and 1 back to the real values that Y can take.

The function is `qnorm`

```
round(qnorm(0.975, mean=140, sd=6),4)
## [1] 151.7598
```

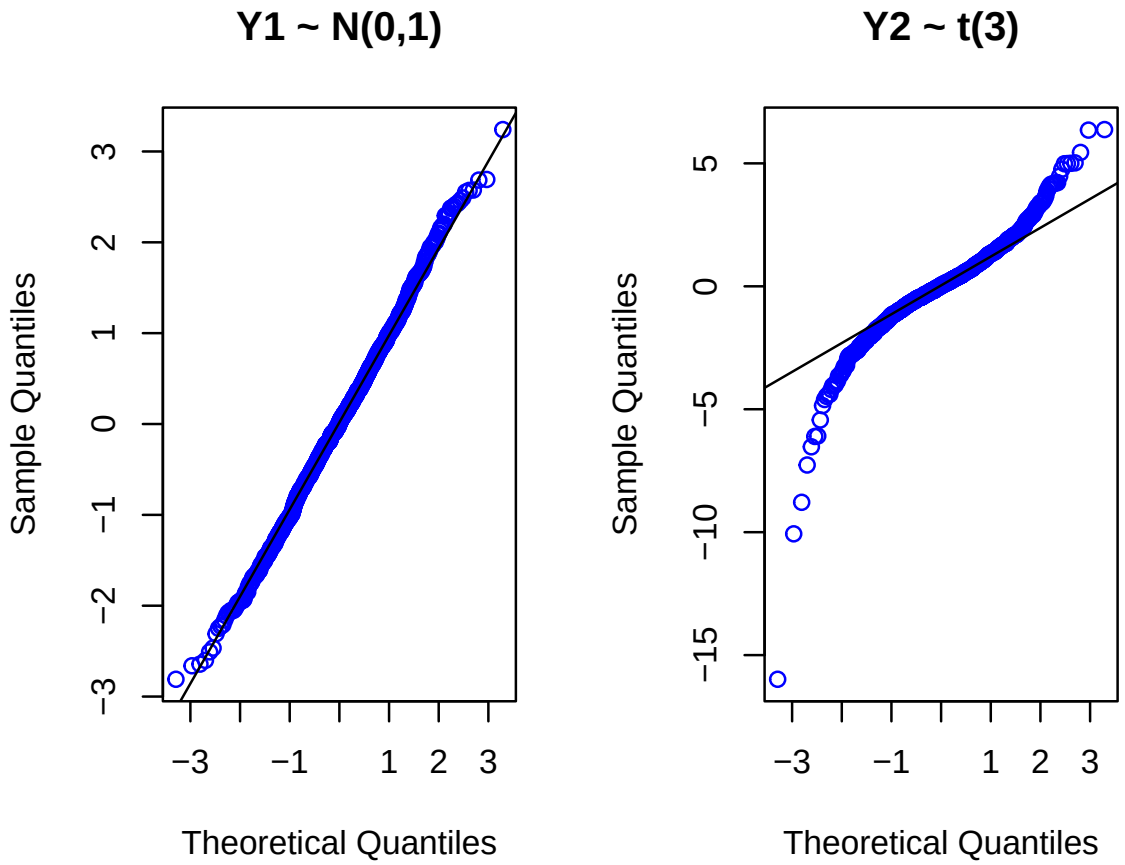
that is that 97.5 per cent of the values of the distribution of $X \sim N(\mu = 140, \sigma^2 = 36)$ are less than or equal to 152.

Q-Q plot

We generate random samples of size 1000 from the two following distributions:

- $Y_1 \sim N(0,1)$ a standard normal
- $Y_2 \sim t_3$ a Student-t with 3 degrees of freedom

```
set.seed(123)
Y1 <- rnorm(1000);
set.seed(234)
Y2 <- rt(1000, df=3)
par(mfrow=c(1, 2))
qqnorm(Y1, col='blue', main='Y1 ~ N(0,1)'); qqline(Y1)
qqnorm(Y2, col='blue', main='Y2 ~ t(3)'); qqline(Y2)
```



- As expected in Figure 1 the sample quantiles referred to Y_1 are in line with the normal quantiles with the points falling close to the straight line y = x.
- The plot of the sample quantiles of the data generated from the Student-t distribution indicates that more observations occur well out in the tails than expected in a standard normal distribution.

Learning objectives

- Describe the associations among variables and summarize the observed values of quantitative variables
- Use a scatterplot matrix to visualize the data

Measures of association

Consider the *credit rating* data illustrated previously

```
load("ratings.Rdata")
```

Correlations

The function `cor` returns the values of the linear correlation calculated as a square matrix.

```
C <- cor(ratings)
round(C,3)
##         rating profit capital f_flex
## rating   1.000  0.550   0.756  0.704
## profit   0.550  1.000   0.618  0.604
## capital  0.756  0.618   1.000  0.645
## f_flex   0.704  0.604   0.645  1.000
```

Note the use of `round` to print only a limited number of decimal places (equal to 3 in the following).

Let denote as Y the variable referred to the *ratings* and X_1, X_2, and X_3 those referred to profit, capital and financial flexibility, respectively. The variable Y is identified as a response variable whose variability could be explained by the covariates or explicative variables X_1, X_2, and X_3.

The values of the **coefficients of raw linear correlation** (r_{yx_1}, r_{yx_2} and r_{yx_3}) between the score assigned by the rating agency and profit, capital and financial flexibility, respectively are 0.55, 0.76 and 0.70.

The raw correlation of the rating is always positive and quite high. Thus, it seems that **an increasing linear association is observed** between rating and the other variables: rating increases linearly with profit, capital and financial flexibility.

The raw correlations between the explanatory variables are all positive: the highest value of 0.64 is obsered between the capital and financial flexibility.

The **sample variance-covariance matrix** is obtained with the `cov` function

```
S1 <- cov(ratings)
round(S1,3)
##          rating profit capital f_flex
## rating  264.237  5.100  26.129  3.474
## profit    5.100  0.326   0.750  0.105
## capital  26.129  0.750   4.524  0.416
## f_flex    3.474  0.105   0.416  0.092
```

The values on the diagonal are the variances. As previously stated standard deviations are high.

Partial correlations

The **partial correction matrix** is calculated with the function found in the Graphical Gaussian Models library `ggm::parcor` which requires the variance-covariance matrix as an argument.

(*The library should be installed, note that it is required also BiocManager use the following commented code*)

```
# if (!require("BiocManager", quietly = TRUE))
#     install.packages("BiocManager")

# install.packages("ggm")

BiocManager::install("graph")
require(ggm)

C1<-parcor(S1)
round(C1,3)
##         rating profit capital f_flex
## rating   1.000  0.014   0.524  0.408
## profit   0.014  1.000   0.312  0.306
## capital  0.524  0.312   1.000  0.124
## f_flex   0.408  0.306   0.124  1.000
```

The following comment on the values of the partial correlations is with respect to the raw correlations:

- there are no changes of the signs of the observed correlations
- it can be seen that $r_{yx_1} = 0.55$ while $r_{yx_1x_2x_3} = 0.01$ that is the correlation between rating and profit *net of the values of the other two variables* is almost zero.

All values of the partial correlations are considerably lower than the values of the raw correlations. In this case the influence of the other variables on the previously calculated bivariate associations is noticeable.

From these observations, we expect that in the multiple linear regression model, the magnitude of the estimate of the **partial regression coefficient** referred to profit is smaller than the other estimates.

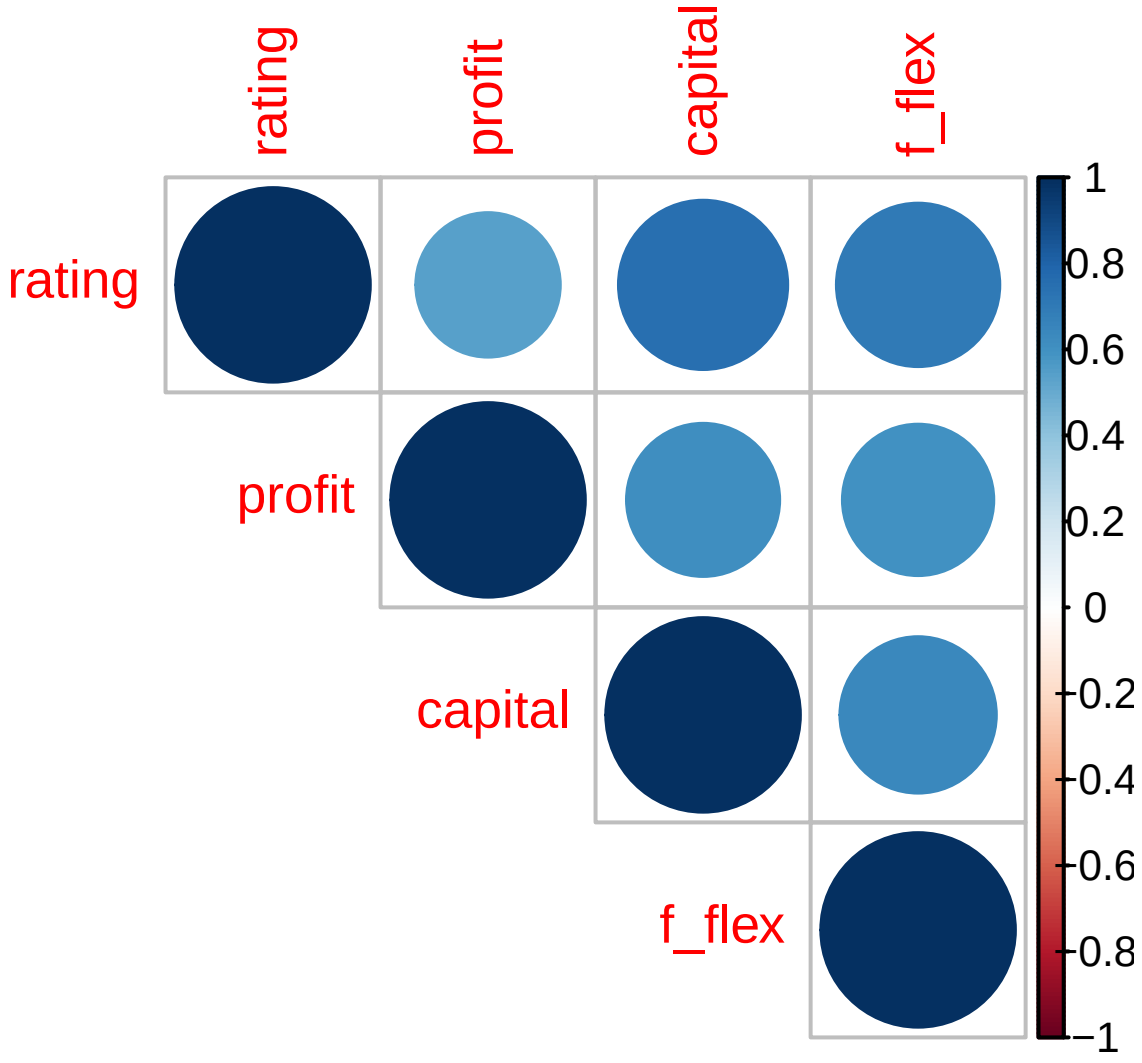
Any notable trends should be checked through formal modelling by which we can establish if these linear associations are statistically significant.

Note that high linear correlations reveal that the observations on two highly correlated variables are concentrated in only a small region of the two-dimensional space. This can cause troubles when estimating the parameters of a statistical model.

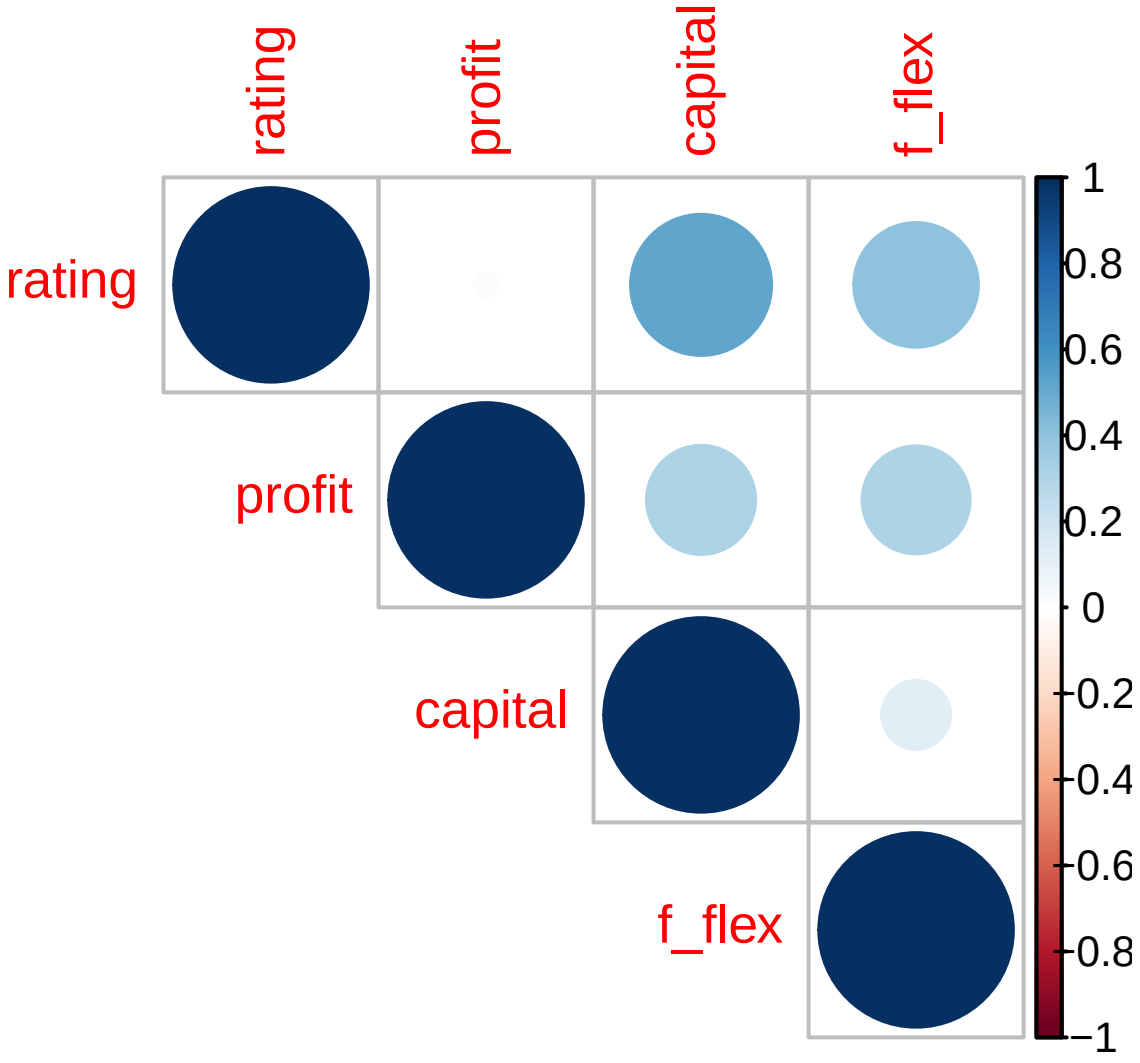
Corplot

For a graphical comparison, the following correlation graphs called `corplots` can be produced

```
require(corrplot)
corrplot(C,type = c("upper"))
```




```
corrplot(C1, type = c("upper"))
```



Note that the default method is **circle**, but the function has many other features.

The **areas of circles** show the absolute values of the corresponding correlation coefficients or partial correlations.

Each cell is shaded blue or red depending on the sign of the correlation, and the color intensity is scaled from 0 to 100% in proportion to the magnitude of the observed raw or partial correlation.

Learning objectives

- Generate data from a bivarite Normal distribution
- Describe through plots the bivariate association
- Depict the a 3D density plot and contour plots commenting on the ellipses
- Use matrix algebra and perform eigenvector decomposition

Bivariate Normal distribution

Generate values from the bivariate Normal distribution

Function `mvtnorm::rmvnorm` (of the package `mvtnorm`) allows the generation of realizations from the bivariate Normal distribution.

The function requires as input the mean vector and the variance-covariance matrix. According to the notation introduced in the teaching notes the parameters of this distribution are expressed as:

$$(X_1, X_2) \sim N(\mu_1, \mu_2, \sigma_1^2, \sigma_2^2, \rho_{12}).$$

Note that the variance-covariance matrix requires the `matrix` function:

```
require(mvtnorm)
sigma1 <- matrix(c(3,2,2,3), ncol=2); sigma1
##      [,1] [,2]
## [1,]    3    2
## [2,]    2    3
```

We fix the seed to reproduce each time the same series of data. First we generate 10 values as follows considering the values of the means stored in the vector

```
set.seed(1234)
n <- 10
mu <- c(0,0)
XY <- rmvnorm(n,
```

```
             mean = mu,
             sigma = sigma1)
round(XY,3)
##          [,1]   [,2]
##  [1,] -1.782 -0.297
##  [2,]  0.305 -3.125
##  [3,]  1.007  1.084
##  [4,] -1.268 -1.240
##  [5,] -1.463 -1.789
##  [6,] -1.389 -1.910
##  [7,] -1.216 -0.375
##  [8,]  1.484  0.415
##  [9,] -1.390 -1.790
## [10,]  0.138  3.392
```

In **output** we have 10 values from (X_1, X_2) where the first column is referred to the realizations from X_1 and the second from X_2.

The empirical *variance-covariance* and *correlation* matrices are

```
round(cov(XY),3)
##       [,1]  [,2]
## [1,] 1.387 0.933
## [2,] 0.933 3.473
round(cor(XY),3)
##       [,1]  [,2]
## [1,] 1.000 0.425
## [2,] 0.425 1.000
```

Note that the theoretical correlation is positive and quite high

```
round(2/sqrt(3*3),3)
## [1] 0.667
```

In the following we generate 2000 realization for the two variables fixing the means equal to zero and variance-covariance matrix as the previous one

```
set.seed(1234)
n <- 2000
XY <- rmvnorm(n, mean = mu, sigma = sigma1)
```

Empirical covariance and correlation

```
round(cov(XY),3)
##       [,1]  [,2]
## [1,] 3.067 2.044
## [2,] 2.044 2.966
round(cor(XY),3)
##       [,1]  [,2]
## [1,] 1.000 0.678
## [2,] 0.678 1.000
```

Descriptive statistics are the following

```
require(skimr)
```

```
skim_without_charts(XY)
```

Table 9: Data summary

Name	XY
Number of rows	2000
Number of columns	2
Column type frequency:	
numeric	2
Group variables	None

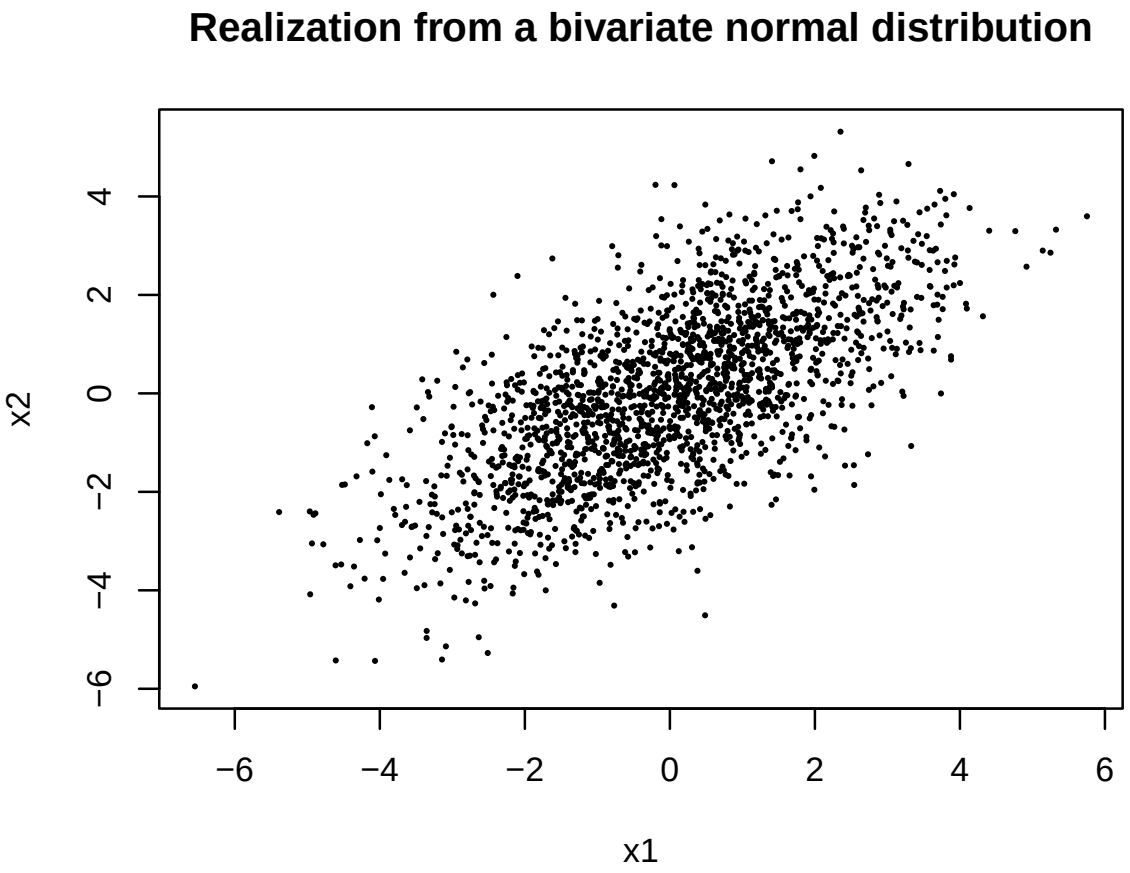
Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100
V1	0	1	0.00	1.75	-6.55	-1.22	0.06	1.16	5.75
V2	0	1	0.01	1.72	-5.95	-1.15	0.01	1.19	5.32

We note that the realized values are in line with the theoretical distribution.

Scatterplot

```
plot(XY[,1],
     XY[,2], main = "Realization from a bivariate normal distribution",
     pch=16,cex=0.4, xlab = "x1", ylab = "x2")
```



Note the scatterplot helps to identify patterns in the realized data. It shows the association between the two variables and indicates that the it can be *linear*.

The shape of an **ellipse** that is centered on the means of the two realizations. The ellipse has the main axes pointing in the directions of correlation.

3D Density plot

Function `scatterplot3d` can be used in the following way

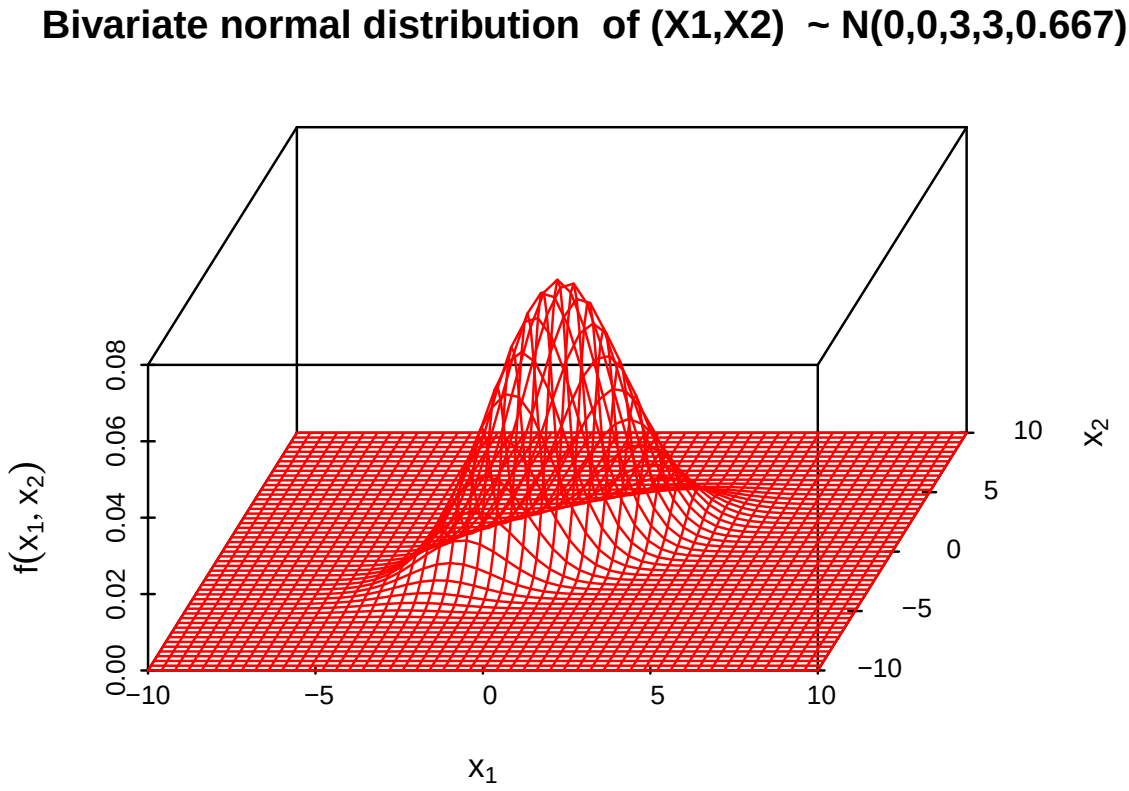
```
x1 <- x2 <- seq(-10, 10, length = 51)

dens <- matrix(dmvnorm(expand.grid(x1, x2),
                sigma = sigma1),
                ncol = length(x1))

require("scatterplot3d")
```

```r
s3d <- scatterplot3d(x1,
                     x2,
                     seq(min(dens),
                     max(dens),
                     length = length(x1)),
                     type = "n",
                     grid = FALSE,
                     angle = 70,
                     zlab = expression(f(x[1], x[2])),
                     xlab = expression(x[1]),
                     ylab = expression(x[2]),
                     main = "Bivariate normal distribution  of (X1,X2)  ~ N(0,0,3


for(i in length(x1):1){
      s3d$points3d(rep(x1[i],
                   length(x2)),
                   x2,
                   dens[i,],
                   type = "l",
                   col="red")
  }
for(i in length(x2):1){
      s3d$points3d(x1,
                    rep(x2[i],
                   length(x1)),
                   dens[,i],
                   type = "l",
                   col="red")
}
```



We notice a bell shape, orientation is positive, as expected from the correlation coefficient, the shape is not flat over the mean, as expected due to low variability of both components.

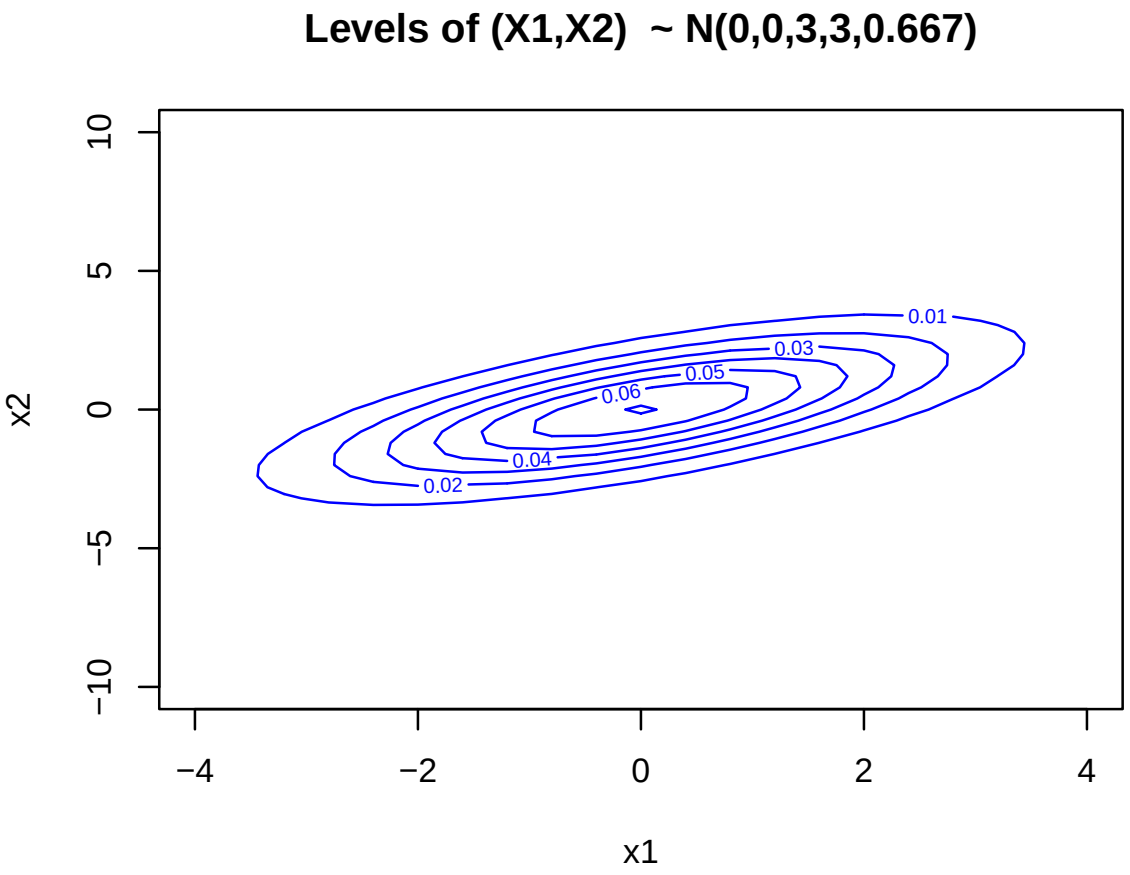
Contour plots

Contour plots are referred to the points of the plane with respect to the equation $f(x_1, x_2) = k$ at level k. They are related to the estimated density of the bivariate variables.

Function `contour` sets the number of levels according to the density. The orthogonal projections are obtained intersecting the scatterplot in 3D with a plane. For increasing values of k circles are nearest to the center.

```
contour(x1,
        x2,
        dens,
        main = "Levels of (X1,X2)  ~ N(0,0,3,3,0.667)",
        col="blue",
```

```
        xlim = c(-4,4),
        xlab = "x1",
        ylab = "x2")
```



These ellipses represent constant density levels, and their *shape*, *size*, and *orientation* provide insights into the underlying covariance structure of the data.

- The *shape* is determined by the eigenvalues and eigenvectors of Σ
- eigenvalues determine the direction (orientation) and eigenvalues the length of the axes
- larger ellipse correspond to higher variance
- ellipses oriented from bottom left to bottom right indicate a negative correlation between two random variables

Note that geometrically, the orientation of the ellipses makes it possible to determine whether the association between the two random variables is positive or negative. The centre and length of the semi-axes are also of interest.

Bivariate distribution with low linear correlation

Let's generate from $N(0, 0, 10, 10, 0.1)$ where $\rho_{12} = 0.1$.

```
sigma2 <- matrix(c(10,1,1,10), ncol=2); sigma2
##      [,1] [,2]
## [1,]   10    1
## [2,]    1   10
```

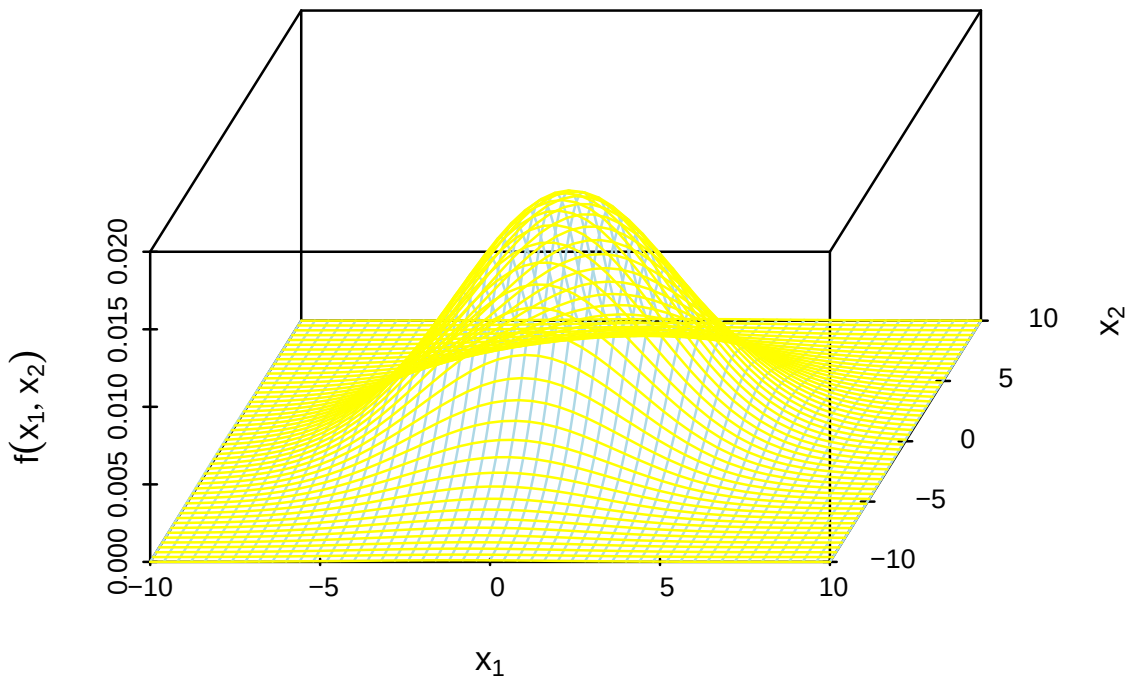
Representation in 3D

```
dens <- matrix(dmvnorm(expand.grid(x1, x2),
                       sigma = sigma2),
                       ncol = length(x1))

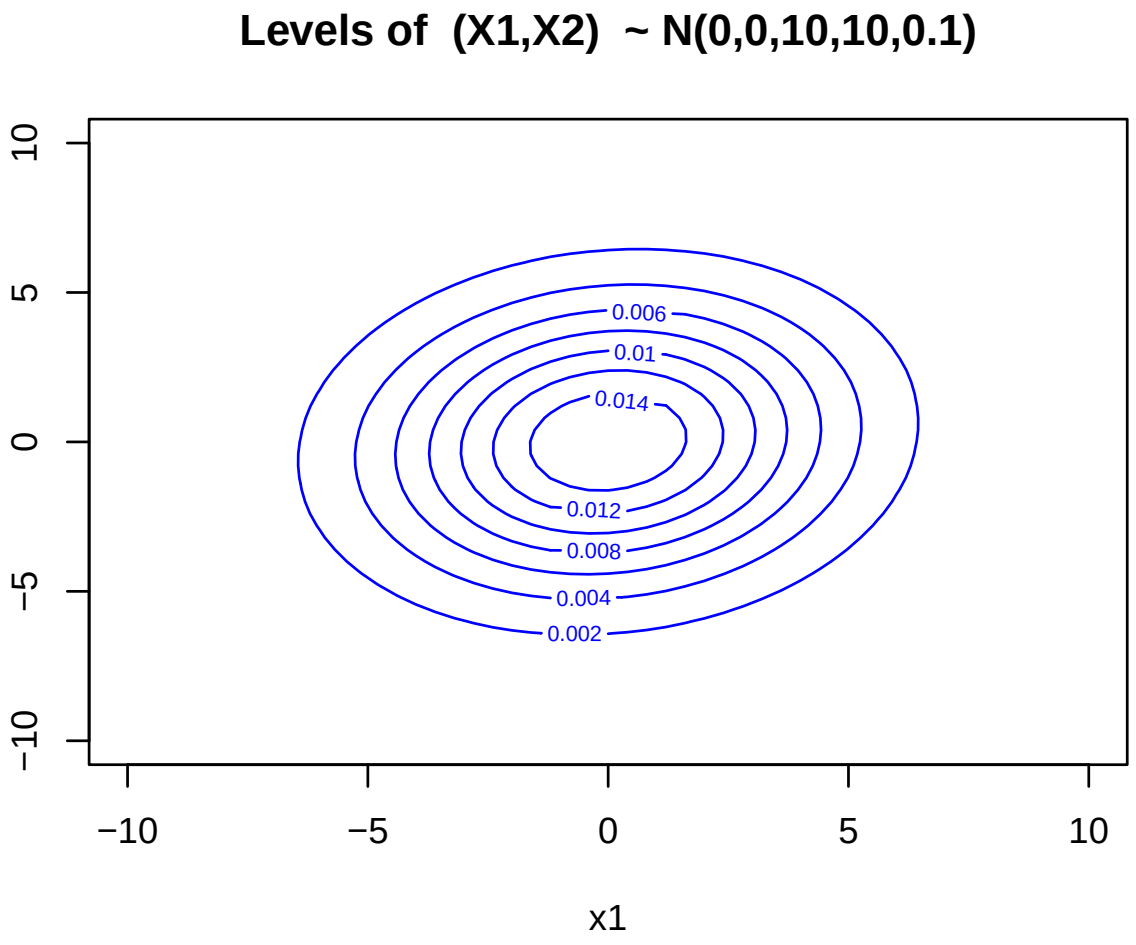
s3d <- scatterplot3d(x1,
                     x2,
                     seq(min(dens),
                     max(dens),
                     length = length(x1)),
                     type = "n",
                     grid = FALSE,
                     angle = 70,
                     zlab = expression(f(x[1], x[2])),
                     xlab = expression(x[1]),
                     ylab = expression(x[2]),
                     main = "Bivariate normal distribution of (X1, X2)~N(0,0,10,1


for(i in length(x1):1){
      s3d$points3d(rep(x1[i],
                   length(x2)),
                   x2,
                   dens[i,],
                   type = "l",
                   col = "lightblue")
  }
for(i in length(x2):1){
```

```
      s3d$points3d(x1,
                   rep(x2[i],
                  length(x1)),
                  dens[,i],
                  type = "l",
                  col="yellow")
}
```




```
contour(x1,
        x2,
        dens,
        main = "Levels of  (X1,X2)  ~ N(0,0,10,10,0.1)",
        col="blue",
        xlab = "x1",
        ylab = "x2")
```



Contour plots are ellipse not much oriented. They will be circles when the correlation is almost zero. Otherwise they will be flat.

Bivariate distribution with high lienar correlation

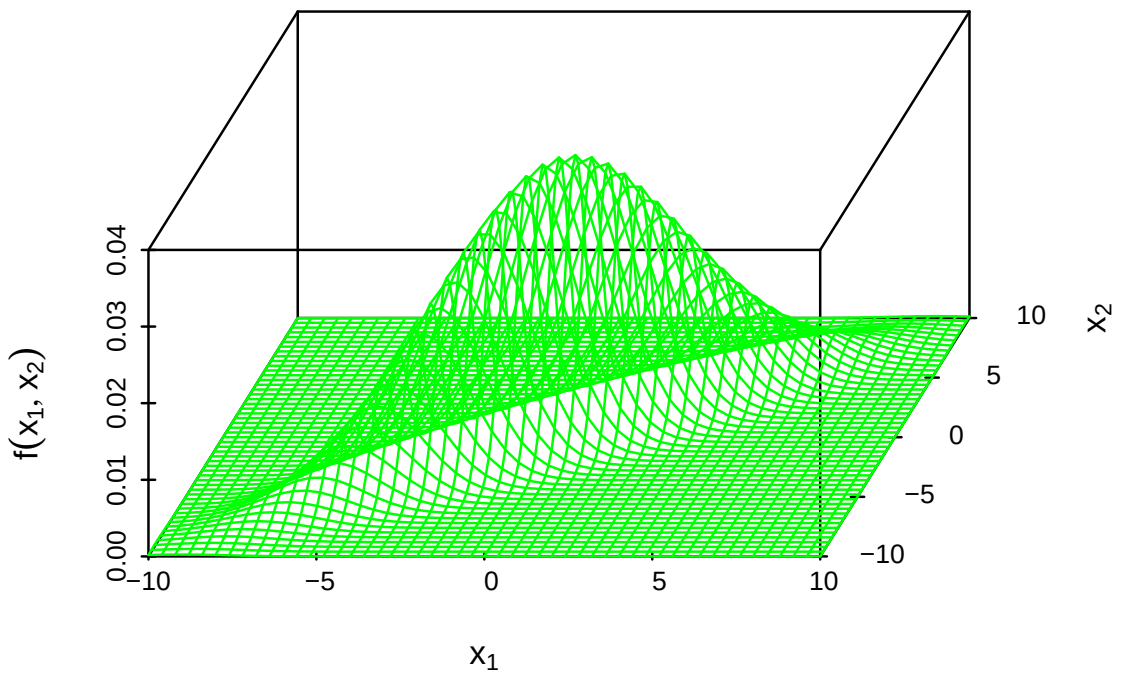
Let's generate from $N(0, 0, 10, 10, 0.9)$ where $\sigma_{12} = 9$

```
sigma3 <- matrix(c(10,9,9,10), ncol=2); sigma3
##      [,1] [,2]
## [1,]   10    9
## [2,]    9   10
```

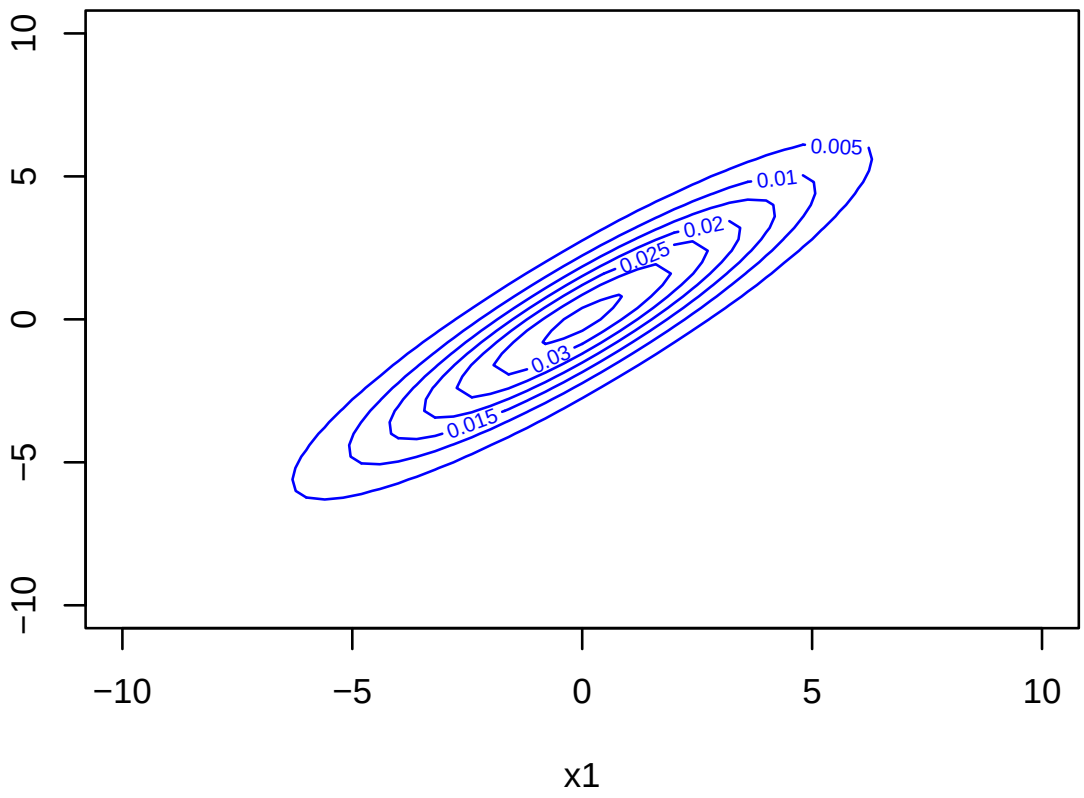
```
dens <- matrix(dmvnorm(expand.grid(x1, x2),
                       sigma = sigma3),
                       ncol = length(x1))

s3d <- scatterplot3d(x1,
                     x2,
                     seq(min(dens),
                     max(dens),
                     length = length(x1)),
                     type = "n",
                     grid = FALSE,
                     angle = 70,
                     zlab = expression(f(x[1], x[2])),
                     xlab = expression(x[1]),
                     ylab = expression(x[2]),
                     main = "Bivariate normal distribution of (X1, X2) ~ N(0,0,10


for(i in length(x1):1){
      s3d$points3d(rep(x1[i],
                   length(x2)),
                   x2,
                   dens[i,],
                   type = "l",
                   col = "green")
  }
for(i in length(x2):1){
      s3d$points3d(x1,
                 rep(x2[i],
                   length(x1)),
                   dens[,i],
                   type = "l",
                   col = "green")
}
```




```
contour(x1,
        x2,
        dens,
         main = "Bivariate normal distribution  of (X1,X2) ~ N(0,0,10,10,9)",
        col="blue",
        xlab = "x1",
        ylab = "x2")
```



Variance-covariance matrix

Review on matrix multiplication

The following code returns a matrix-like object

```
x1 = 1:5; x1
## [1] 1 2 3 4 5
X1 = cbind(1,x1); X1
##      x1
## [1,] 1  1
## [2,] 1  2
## [3,] 1  3
## [4,] 1  4
```

```
## [5,] 1  5
X1%*%c(2,3)
##      [,1]
## [1,]    5
## [2,]    8
## [3,]   11
## [4,]   14
## [5,]   17
```

Eigenvalue decomposition

We consider the computation of the eigenvalues and eigenvectors of a symmetric matrix (see also the webpage https://online.stat.psu.edu/stat505/lesson/4/4.5). Watch the following video for an explanation for the relation of the ellipse and eigenvalues and eigenvectors https://www.youtube.com/watch?v=V5sxHXHZvjE).

Function `eigen` allow to extract the following quantities

```
EV<-eigen(sigma1)
EV
## eigen() decomposition
## $values
## [1] 5 1
##
## $vectors
##           [,1]       [,2]
## [1,] 0.7071068 -0.7071068
## [2,] 0.7071068  0.7071068
```

eigenvalues are 5 and 1 so they are different from zero and there is the matrix of the eignevectors.

Recall that:

- the number of non-zero eigenvalues is equal to the *rank* of the matrix
- the eigenvalues of an *idempotent* matrix are zero or 1

- the sum of the eigenvalues is equal to the *trace*

Recall that:

- a variance-covariance matrix is always semi-positive defined
- a matrix A is semipositive defined if and only if it can be written as $A = XX'$ with whatever X
- the determinant should be non-negative

$$det(\Sigma) = \sigma_1^2\sigma_2^2 - \sigma_{12} \geq 0$$

which simplifies to $\sigma_{12}^2 \leq \sigma_1^2\sigma_2^2$

To decompose the variance-covariance matrix

```
L <- EV$values
L
## [1] 5 1
V <- EV$vectors
V
##            [,1]       [,2]
## [1,] 0.7071068 -0.7071068
## [2,] 0.7071068  0.7071068
```

The matrix of eigenvectors governs the orientation and the diagonal matrix of scaled eigenvalues determines its shape.

Decomposition can be written as $A = Q\lambda Q'$ where we consider the variance-covariance matrix

```
V %*% diag(L) %*% t(V)
##      [,1] [,2]
## [1,]    3    2
## [2,]    2    3
```

For the first component the eigenvalue decomposition is obtained as follows

```
A1 = L[1] * V[,1] %*% t(V[,1])
A1
##      [,1] [,2]
## [1,]  2.5  2.5
## [2,]  2.5  2.5
```

for the second component we have

```
A2 = L[2] * V[,2] %*% t(V[,2])
A2
##      [,1] [,2]
## [1,]  0.5 -0.5
## [2,] -0.5  0.5
```

sum of both returs the original matrix

```
A1 + A2
##      [,1] [,2]
## [1,]    3    2
## [2,]    2    3
```

Example of likelihood function

In the following we use the example presented in the teaching notes to illustrate the likelihood and the log-likelihood.

Suppose a random variable X representing a success event, such as the event that the return of a bond in a given portfolio is greater than or equal to 3%. Considering three assets, what is the proportion of bonds with returns higher than 3%?

Suppose we have a random sample in which we observe

$$x_1 = 1, \quad x_2 = 0, \quad x_3 = 1,$$

the maximum likelihood estimation is the following

```r
x <- c(1, 0, 1)

p_hat <- mean(x)
p_hat
## [1] 0.6666667
```

We compute the values of the likelihood function for different probability values and determine the value that maximizes it.

```r
likelihood <- function(p, data) {
  prod(p^data * (1 - p)^(1 - data))
}

p_vals <- seq(0.01, 0.99, length.out = 100)

lik_vals <- sapply(p_vals, likelihood, data = x)

p_mle_numeric <- p_vals[which.max(lik_vals)]

p_mle_numeric
## [1] 0.6633333
```

The same is done with the log-likelihood

```r
logLik <- function(p, data) {
  sum(data * log(p) + (1 - data) * log(1 - p))
}

loglik_vals <- sapply(p_vals, logLik, data = x)

p_mle_log <- p_vals[which.max(loglik_vals)]

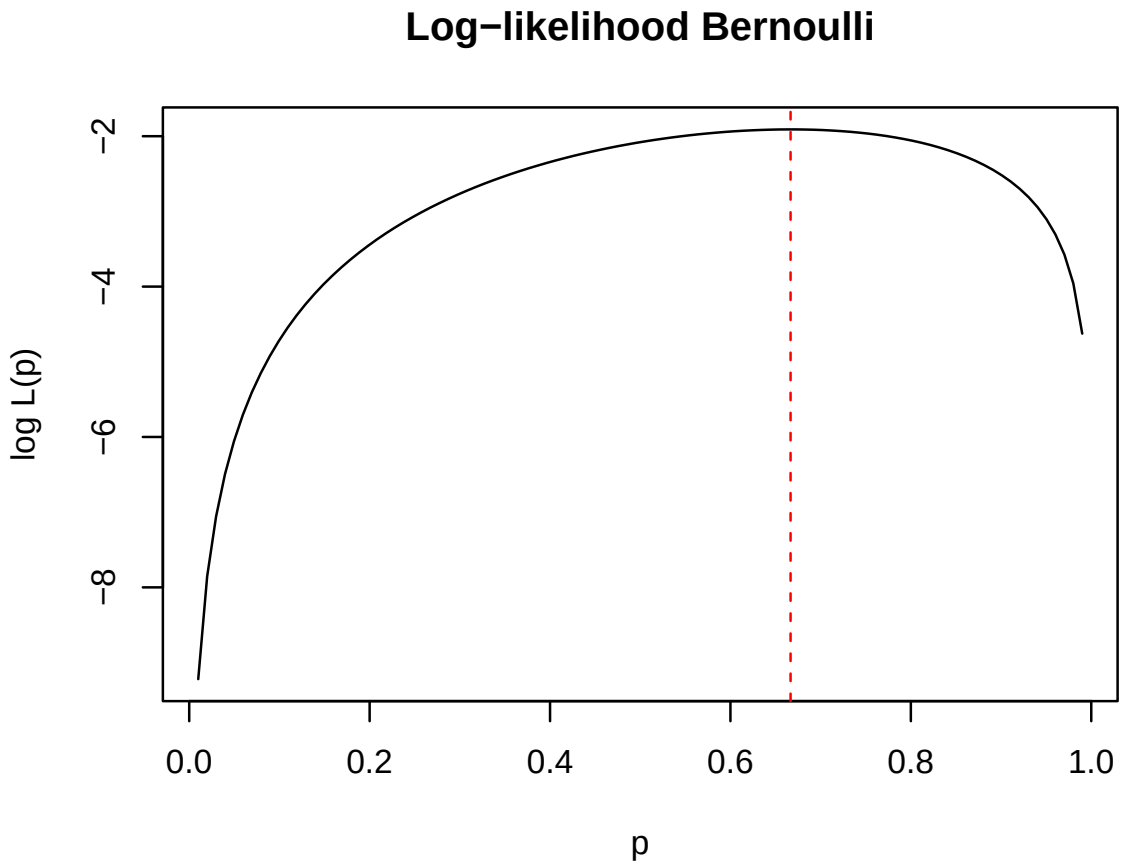
p_mle_log
## [1] 0.6633333
```

We plot the log-likelihood function

```r
plot(p_vals, loglik_vals, type = "l",
     main = "Log-likelihood Bernoulli",
     xlab = "p",
```

```
     ylab = "log L(p)")

abline(v = mean(x), col = "red", lty = 2)
```



Learning objectives

- Apply the nonparametric bootstrap for assessing estimation accuracy of any estimator
- Calculate confidence intervals using the percentile method

Nonparametric bootstrap

Data scores

Data of 799 students to a cognitive test (for example to evaluate vocabulary knowledge)

```
load("testScore.Rdata")
head(score)
## [1] 21 18  2 15 15 24
require(skimr)
skim_without_charts(score)
```

Table 11: Data summary

Name	score
Number of rows	799
Number of columns	1
Column type frequency:	
numeric	1
Group variables	None

Variable type: numeric

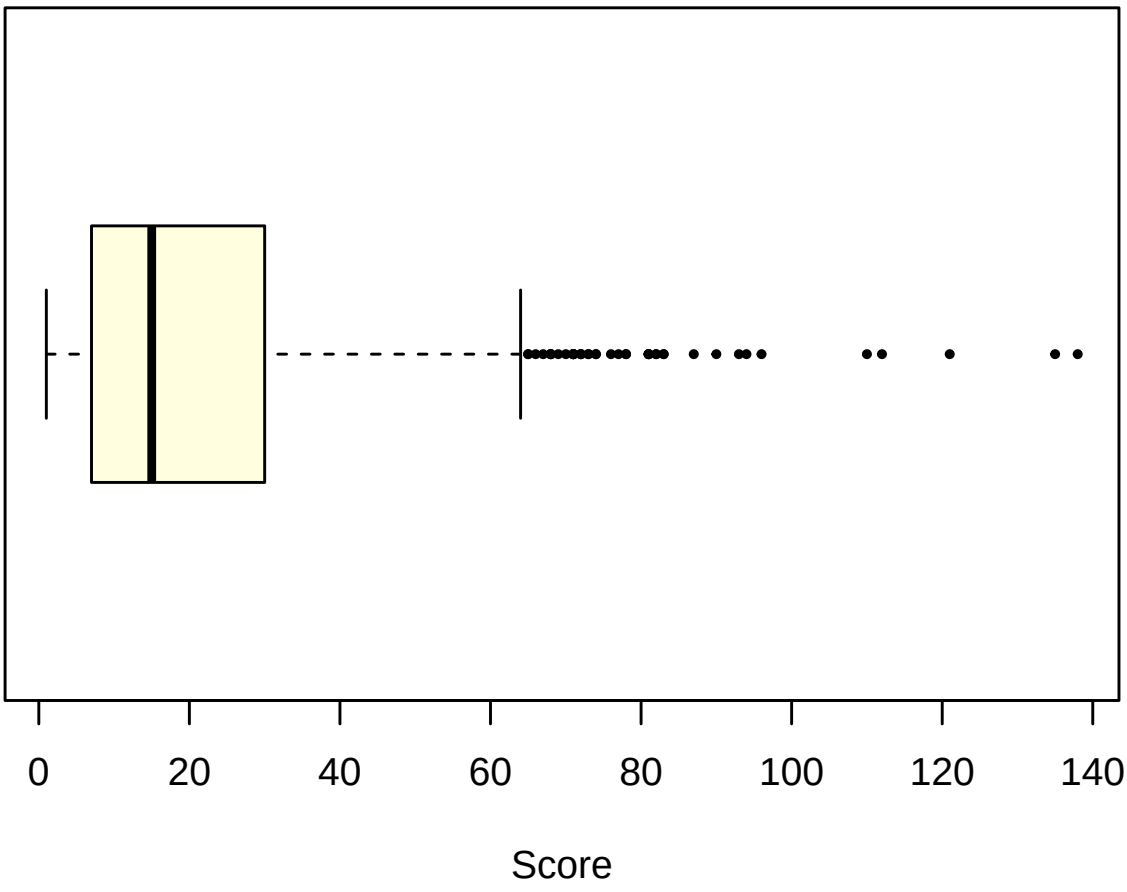
skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100
data	0	1	21.86	20.92	1	7	15	30	138

From the boxplot and the histogram we note the skewness in the empirical distribution. Tail towards lower values is more heavy than that towards higher level. Median and mean are different and there is quite huge distance between the third quantile

and the maximum.

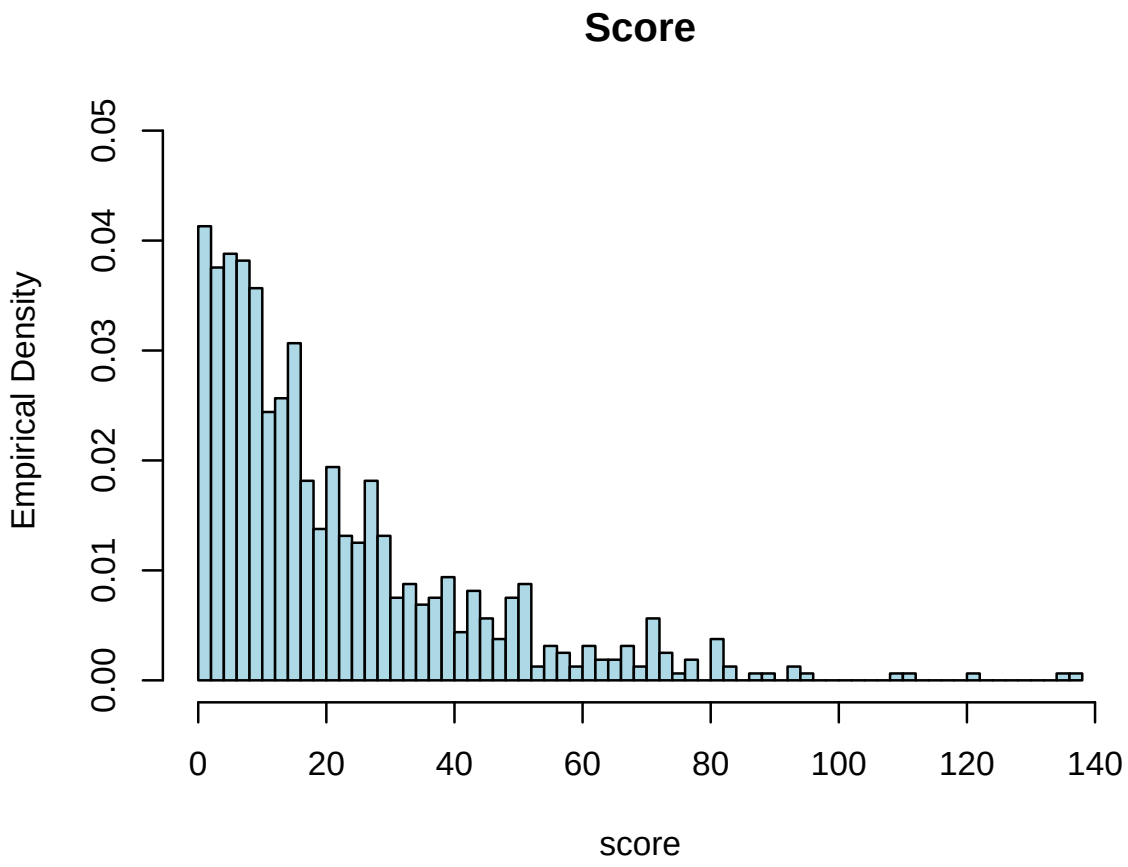
Also the following figures show that the empirical distribution of the data is not symmetric and skewed to the right.

```
boxplot(score, xlab="Score", horizontal=TRUE,
        col = "lightyellow", pch=20, cex = 0.6)
```




```
hist(score,
     breaks = 50,
     ylim = c(0, 0.05),
     main = "Score",
     ylab=" Empirical Density",
```

```
    freq =FALSE,
    col = "lightblue")
```



Skewness index

If we consider a random variable X with expected value μ and variance σ^2. The skewness is defined as:

$$\kappa = E\left[\frac{(X-\mu)^3}{\sigma^3}\right] = \frac{\int (x-\mu)^3 dF(x)}{\left(\int (x-\mu)^2 dF(x)\right)^{3/2}},$$

and $\kappa = 0$ if there is symmetry around the mean. Therefore $\kappa > 0$ if there is positive skewness.

Skewness (κ) is the **parameter** of interest in this example. We choose an estimator that is the **plug-in** estimator which is the following

$$\hat{\kappa} = T(\hat{F}_n) = \frac{\int (x-\mu)^3 d\hat{F}_n(x)}{\hat{\sigma}^3} = \frac{1/n \sum_i^n (X_i - \bar{X}_n)^3}{\hat{\sigma}^3}.$$

We use the function in the library `e1071` to calculate the point estimate for the reference population

```
require(e1071)
sk <- e1071::skewness(score)
sk
## [1] 1.757943
```

The skewness value of 1.76 indicates that the distribution of the scores is positively skewed (right-skewed). This means that the tail on the right side of the distribution is longer or heavier than the left side. Most of the observations are concentrated at lower values, with a few relatively high values pulling the distribution to the right.

As a benckmark remember that a skewness equal to zero means a perfectly symmetric distribution (e.g. like the normal distribution). A skewness above +1 is generally considered highly skewed, suggesting that the data deviate notably from normality.

Bootstrap

The distribution of the random variable referred to the test scores is unknown and the distribution of κ is unknown.

The nonparametric bootstrap is as follows:

1. We generate by resampling the bootstrap replications $X_1^*, \dots, X_n^*$;
2. For each bootstrap sample we calculate the bootstrap replication (the value of the skewness in each bootstrap sample) $T_n^* = g(X_1^*, \dots, X_n^*)$;
3. We obtain $T_{n,1}^*, \dots, T_{n,B}^*$;
4. We calculate the standard deviation as follows

$$sd_{boot} = \sqrt{\frac{1}{B-1}\sum_{r=1}^{B}\left(T_{n,b}^* - \frac{1}{B}\sum_{r=1}^{B}(T_{n,r}^*)\right)^2}.$$

Implementing the code:

- STEP 1: we sample from the data to obtain the **first replicated sample** from the original sample.

Function `sample` can be used to obtain a sample of size `n` *with replacement* once the option `{replace=TRUE}` is considered

```
n <- length(score)
B1 <- sample(score, n, replace = TRUE)
summary(B1)
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.00    7.00   14.00   20.15   27.00  138.00
```

- We generate a **second bootstrap sample**

```
B2 <- sample(score, n, replace = TRUE)
summary(B2)
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.00    7.00   16.00   22.85   33.00  138.00
```

- and the third **bootstrap sample**

```
B3 <- sample(score, n, replace = TRUE)
summary(B3)
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.00    7.00   15.00   21.87   30.00  121.00
```

- STEP 2: for each bootstrap sample we calculate the corresponding value of the skewness and we save the realized values in an object `s`

```
s1<-skewness(B1)
s2<-skewness(B2)
s3<-skewness(B3)
s <- c(s1,s2,s3); s
## [1] 1.855666 1.609749 1.582186
```

see also the way we can use function **apply**

```
BB<-cbind(B1,B2,B3)
apply(BB,2,skewness)
##       B1       B2       B3
## 1.855666 1.609749 1.582186
```

- STEP 3: we calculate the **standard deviation** of the bootstrap replications. This is the standard error we can associate to the point estimate obtained for the swknesss in the original sample.

Using only three bootstrap samples in not enough. We get the following standard error

```
sd(s)
## [1] 0.1505691
```

We need at least $B = 200$ bootstrap samples.

We first illustrate the use of the cycle for to generate many sample. Then we show the use of the function `bootstrap` within the `bootstrap` library.

```
 B <- 1000
 n <- length(score)
 Tboot <- rep(0, B)
 set.seed(16253)
 for (i in 1:B) {
       Xstar <- sample(score,
                       n,
                       replace = TRUE)
       Tboot[i] <- e1071::skewness(Xstar)
 }

head(Tboot)
## [1] 1.608102 1.532254 1.751126 1.847124 1.608976 1.736952
summary(Tboot)
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.223   1.630   1.742   1.743   1.850   2.444

seTboot <- sd(Tboot); seTboot
## [1] 0.1694565
```

We employed 1000 bootstrap replications to obtain a standard error of 0.16 (less than the previous one based only on $B = 3$ replications).

To apply the nonparametric bootstrap we require data to be **informative** on the object of inference.

Use of the boostrap library

```
require(bootstrap)
set.seed(162534)
bootM <- bootstrap(score,
          nboot= B,
          e1071::skewness)
```

The object in output is named `thetastar`

```
sd(bootM$thetastar)
## [1] 0.1599339
```

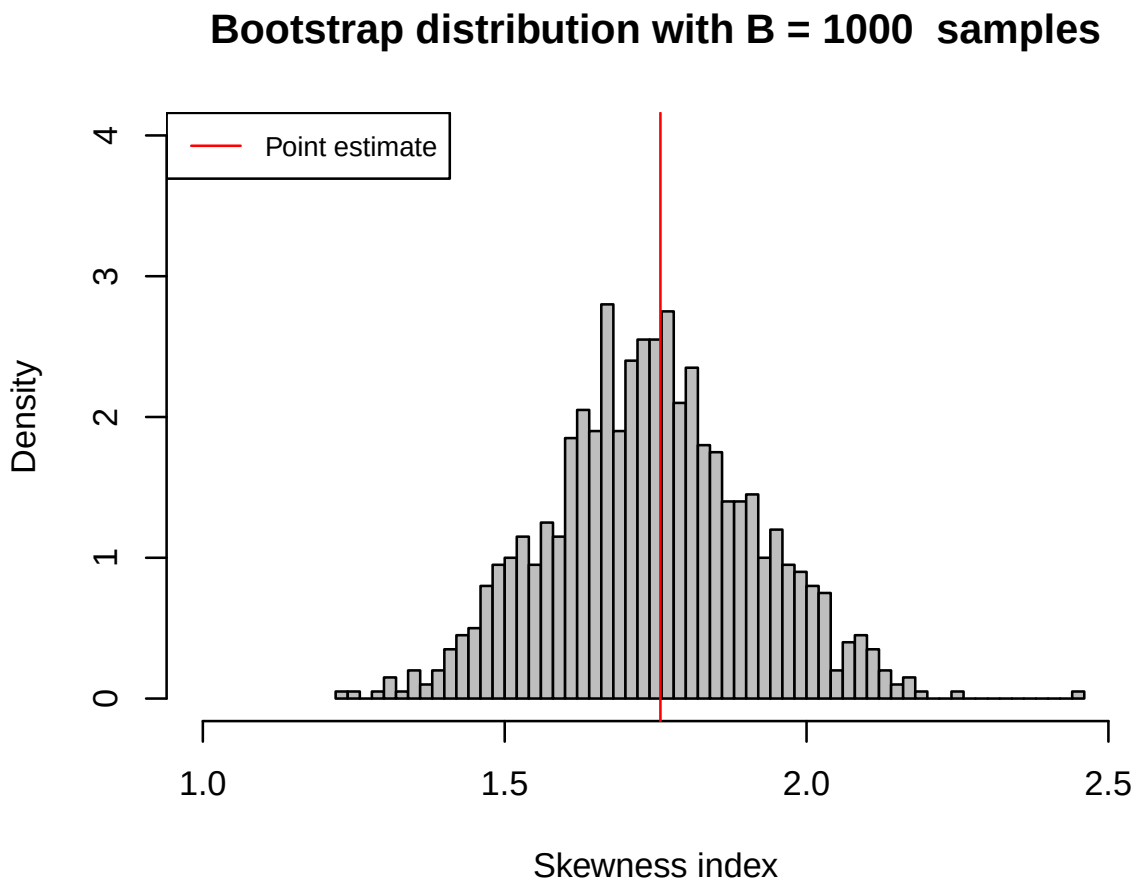
Bootstrap confidence intervals

The bootstrap method provides a direct computational way of assessing uncertainty, by sampling from the data.

Percentile method

```
hist(Tboot,
      breaks=50,
      freq=FALSE,
      main = "Bootstrap distribution with B = 1000  samples",
```

```
      xlab = "Skewness index",
      ylim = c(0,4),
      col= "gray",
      ylab = "Density",
      xlim = c(1,2.5))
abline(v=sk, col="red")
legend("topleft", 2,
       c("Point estimate"),
       col = "red",
       lty= 1,
       cex = 0.8)
```



The bootstrap confidence interval obtained from the percentile method is calculated using the empirical distribution of bootstrap replications.

When using the percentile method as extremes of the confidence interval, one takes the quantiles of the empirical distribution corresponding to the chosen confidence level, e.g. 0.95.

Therefore this is a 95% confidence interval for the skewness index.

Recall that:

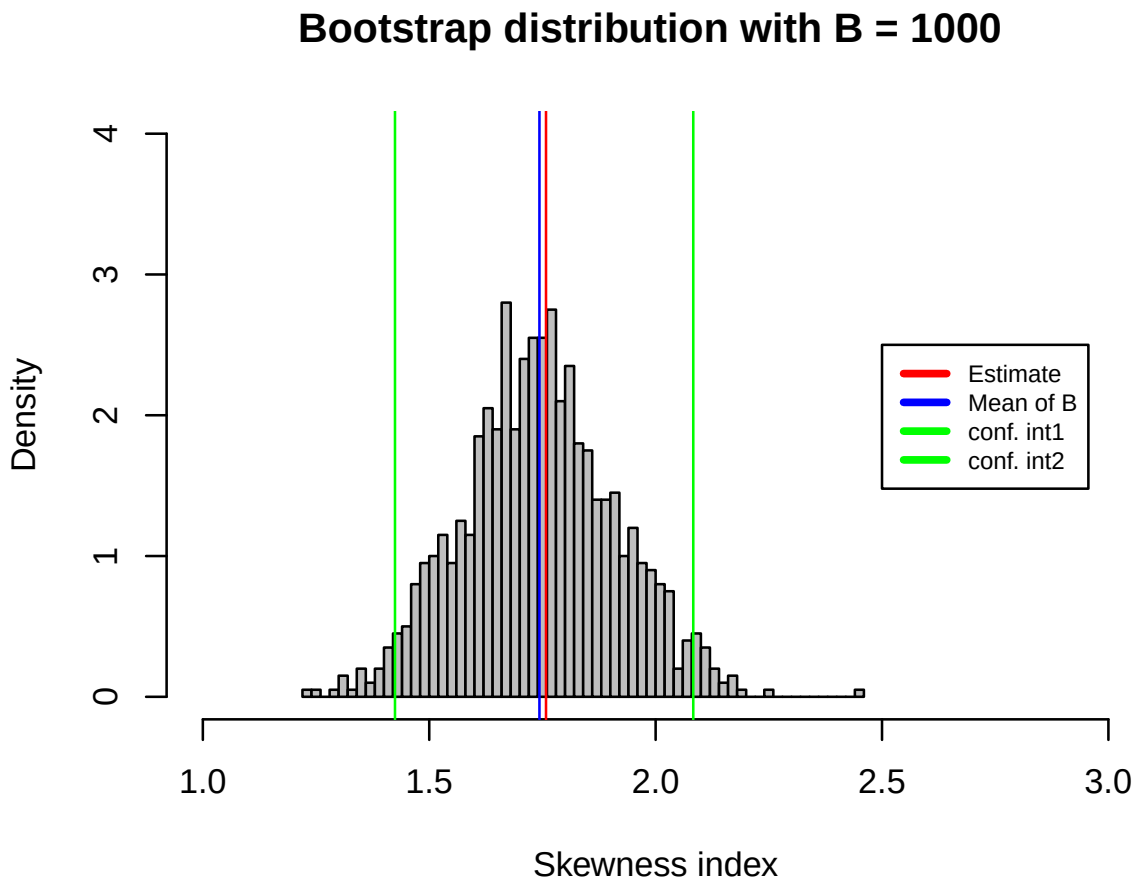
- The confidence interval is a range of values around the point estimate of the parameter within which it is considered plausible that the parameter is predicted to fall.
- The **confidence level** is a probability referred to as a confidence interval with random extremes in repeated sampling.
- The value 0.95 on the other hand **is not** the probability that κ belongs to the observed interval.
- We have confidence that κ belongs to the observed interval because the procedure includes it in 95 per cent of the cases.

The histogram of the bootstrap distribution is depicted by adding the bar indicating the point estimate and the minimum and maximum values of the realized confidence intervals.

```
sB<- mean(Tboot)
Q <- quantile(Tboot, c(0.025, 0.975))

Q[1]; Q[2]
##     2.5%
## 1.424408
##    97.5%
## 2.083085
#
hist(Tboot,
      breaks = 60,
      freq=FALSE,
      main = "Bootstrap distribution with B = 1000 ",
      xlab = "Skewness index",
     ylim = c(0,4),
      col = "gray",
       ylab = "Density",
     xlim = c(1, 3))
#
abline( v = c(sk,sB,Q[1], Q[2]),
       col = c("red", "blue", "green", "green"))
```

```
#
legend(2.5,2.5,
       c("Estimate", "Mean of B",
         "conf. int1", "conf. int2"),
      col = c("red", "blue", "green", "green"),
       lty = c(1,1,1,1),
       lwd = c(3,3,3,3),
       cex = 0.7)
```



Learning objectives

- Formulate the research questions to be answered using a multiple linear regression model
- Estimate the linear model and comment on the results
- Interpret the estimated parameters in the context of interest

Multiple linear regression: Profits of the firms

As an example we consider the profit margins of different firms within the same sector (y), their net revenues per deposit dollar (x_1), and the share of investments in other important sector-specific activities (x_2). We aim to explain the variability in profits in terms of net revenues and investments.

We also to assess whether and to what extent profits increase with net revenues and investments.

Explanatory data analyses and descriptive statistics, including correlations, partial correlations, and scatter plots, are required, as in Exercise 2.

Estimation of the model parameters

The function `stats::lm` is used to estimate the parameters of the multiple linear regression model

```
load("profits.Rdata")
head(profits)
##              y        x1       x2
## 1 -0.03229359 1.4498836 2.835682
## 2  0.82817699 0.8164275 1.315232
## 3  4.01284020 1.5060157 0.965941
## 4  4.82411236 2.2585572 2.755820
## 5  2.48486540 1.0347815 2.088001
## 6  1.28942814 1.4005885 2.074641
```

The model is specified symbolically as a **response** (numeric vector) followed by a **tilde**.

The **explanatory variables** are specified sequentially using the + symbol. The data frame name is provided as the last argument of the function.

The model is then estimated as follows, with the output stored in the object `lm1`

```
lm1 <- lm(y ~ x1 + x2, profits)
```

Similarly, when all variables in the dataset are to be used as explanatory variables, the model can be specified using a dot `.`, as follows:

```
lm1 <- lm(y ~ . , profits)
```

Output of the function lm

lm1 contains all the information about the fitted model:

- the *residuals* of the estimated model,
- the parameter *estimates* of the multiple regression model, along with their standard errors and t-tests
- the fitted values
- the degrees of freedom
- the R^2 and adjusted R^2
- the F-statistic

Using `summary` function, the previous mentioned are displayed

```
summary(lm1, digits=3)
##
## Call:
## lm(formula = y ~ ., data = profits)
```

```
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.2286 -0.6205  0.0173  0.6531  2.5808
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.82453    0.10657  -7.737 5.72e-14 ***
## x1           3.04361    0.04506  67.551  < 2e-16 ***
## x2          -0.59457    0.06229  -9.545  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9639 on 497 degrees of freedom
## Multiple R-squared:  0.9391, Adjusted R-squared:  0.9389
## F-statistic:  3835 on 2 and 497 DF,  p-value: < 2.2e-16
```

where:

- `Call`: recalls the formula of the estimated model,
- `Residuals`: provides descriptive statistics summarising the residuals,
- `Coefficients`: is a $(p+1) \times 4$ matrix that reports:
  - the estimates of the model parameters and related measures (standard errors, t-values, and p-values)
  - `df` (degrees of freedom): the number of degrees of freedom in the model,
  - `fstatistic`: the value of the F-statistic used to test the null model,
  - `R.squared`: the value of the index (R^2 in the theoretical handouts) and the corresponding adjusted R^2, measuring the goodness of fit.

From the output we notice:

- the **residuals** vary between -3.2 and 2.6; their variability lies within the expected range ± 3,

- based on the estimated **coefficients**, the equation of the fitted plane for the i-th unit, $i = 1, \ldots, n$, is as follows

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \hat{\beta}_2 x_{i2}$$

that is (*report only 4 decimal digits*)

$$\hat{y}_i = -0.8245 + 3.0436 x_{i1} - 0.5945 x_{i2}$$

The **intercept** β_0 represents the expected value of the response variable when all the explanatory variables are equal to zero. Its estimate is negative; specifically, -0.8245 represents the expected profit of a firm when revenues and investment are zero.

The estimate of the **partial regression coefficient** β_1 indicates that profits increases by 3.0436 points for each unit increase in revenues, while holding investments fixed.

The estimate of the **partial regression coefficient** β_2 indicates that, adjusting for revenues, the predicted profits decrease by 0.6 points for each unit increase in investments.

Remember: *association does not imply causation.*

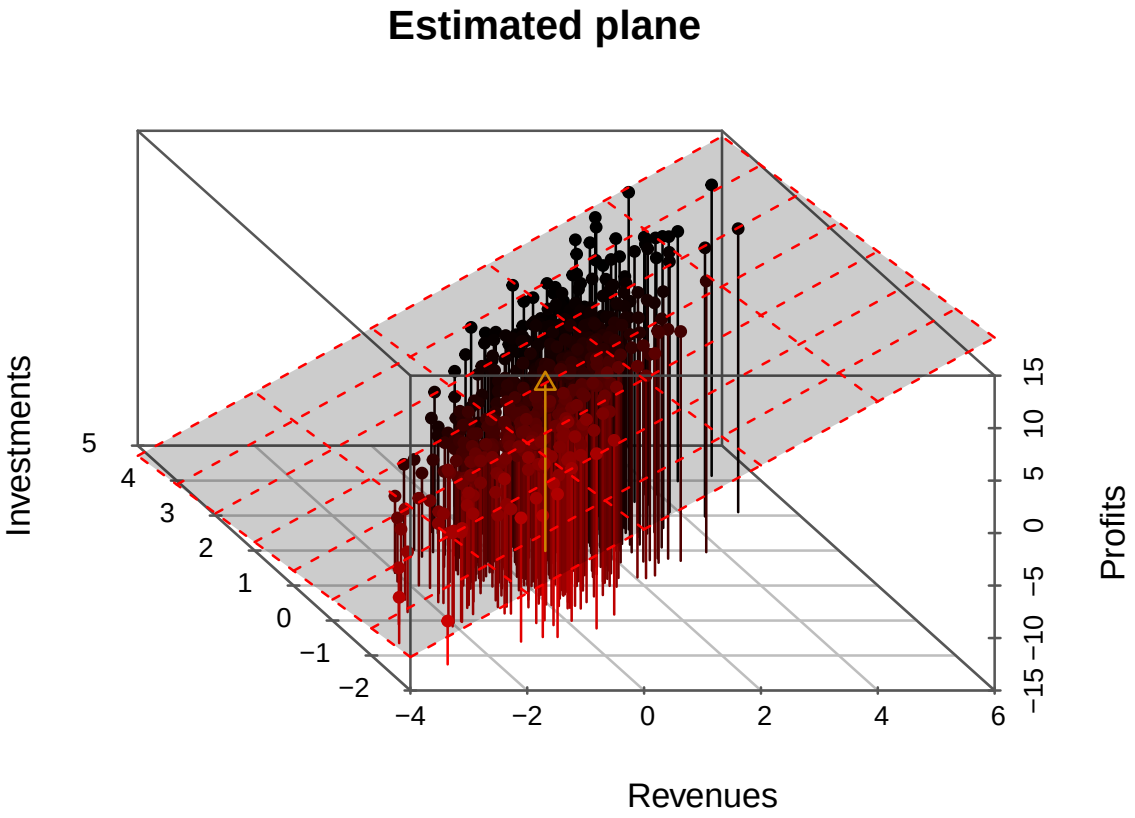
Goodness of fit

- The estimated **Residual Standard Error** is 0.964; it represents the estimated conditional standard deviation s of profits for fixed values of revenues and the share of investments.
- $R^2 = 0.94$ indicates that the fitted model can explains more than 94% of the observed variability in profits. This corresponds to a reduction of about 94% in the sum of squared errors compared to using $\bar{y}$ as a predictor.

The adjusted R^2 is 0.9391, which is very similar to R^2

- These indexes do not allow us to assess whether the model is correctly specified (i.e., whether all relevant covariates are included, whether the assumption of linearity is appropriate, etc.)
- The following scatterplot shows the observed values and the fitted plane.

```r
require(scatterplot3d)
ex1 <- scatterplot3d(profits[,2],
                     profits[,3],
                     profits[,1],
                     type="h",
                     xlab = "Revenues",
                     ylab = "Investments",
                     zlab = "Profits",
                     highlight.3d=TRUE,
                     col.axis="gray34",
                     col.grid="gray",
                     angle=120,
                     main="Estimated plane",
                     pch = 20)
ex1$points3d(mean(profits[,2]),
             mean(profits[,3]),
             mean(profits[,1]),
             col = "Orange",
             type = "h",
             pch =  2)
ex1$plane3d(lm1,col="red",
             draw_lines = TRUE,
            draw_polygon = TRUE)
```


Estimated plane
Investments
Revenues
Profits
5
4
3
2
1
0
−1
−2
−4
−2
0
2
4
6
15
10
5
0
−5
−10
−15

Learning objectives

- Interpret the estimated parameters, statistical tests and confidence intervals for the multiple linear regression
- Use diagnostic plots to examine the model

Estimated multiple linear regression model

We are interested in explaining profits according to revenues (x_1) and investments (x_2)

```
load("profits.Rdata")
lm1 <- lm(y ~ x1 + x2, profits)
```

Fitted values

Fitted values are given by the following equation for example for unit 1 it results

$$\hat{y}_1 = \hat{\beta}_0 + \hat{\beta}_1 x_{11} + \hat{\beta}_2 x_{12}$$

$$1.9023 = -0.8245 + 3.0436 * 1.4499 - 0.5946 * 2.8357$$

for which we observe net revenues equal to 1.4499 and investments 2.8357.

Its expected **profit** according to the estimated model is *positive* $\hat{y}_1 = 1.9023$ even the observed response is instead negative $y_1 = -0.0323$.

For units 10, 21 e 35 observed profits are

```
profits[c(1, 10,21,35),1]
## [1] -0.03229359  3.09526301  3.79689054  0.25586342
```

while predicted profits for such units according to the estimated model can be calculated approximately as

```
-0.8245 + 3.0436*1.4499 -  0.5946*2.8357
## [1] 1.902308
-0.8245 + 3.0436*1.6666  - 0.5946*1.3498
## [1] 3.445373
-0.8245 + 3.0436*2.5548  - 0.5946*2.9820
## [1] 5.178192
-0.8245 + 3.0436*0.4751  - 0.5946*0.7482
## [1] 0.1766346
```

The output object contains fitted values for each unit in the sample

```
lm1$fitted[c(1,10,21,35)]
##         1        10        21        35
## 1.9023377 3.4455196 5.1783496 0.1766872
```

Residuals

Residuals are $z_i = y_i - \hat{y}_i$ and they can be calculated as

```
-0.0323 - 1.9023
## [1] -1.9346
 3.0953  - 3.4454
## [1] -0.3501
 3.7969  - 5.1782
## [1] -1.3813
 0.2559  - 0.1766
## [1] 0.0793
```

The output object of the `lm` function contains the residuals

```
lm1$residuals[c(1,10,21,35)]
##           1          10          21          35
## -1.93463125 -0.35025659 -1.38145906  0.07917624
```

Descriptive statistics of the residuals are the following

```
require(skimr)
skim_without_charts(lm1$res)
```

Table 13: Data summary

Name	lm1$res
Number of rows	500
Number of columns	1
Column type frequency:	
numeric	1
Group variables	None

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100
data	0	1	0	0.96	-3.23	-0.62	0.02	0.65	2.58

As expected (recall the properties of residuals) they on average are around zero.

In this model they show a quite low variability of around 1.

The maximum is less than three showing that their expected range aligns with realizations from a univariate Gaussian distribution.

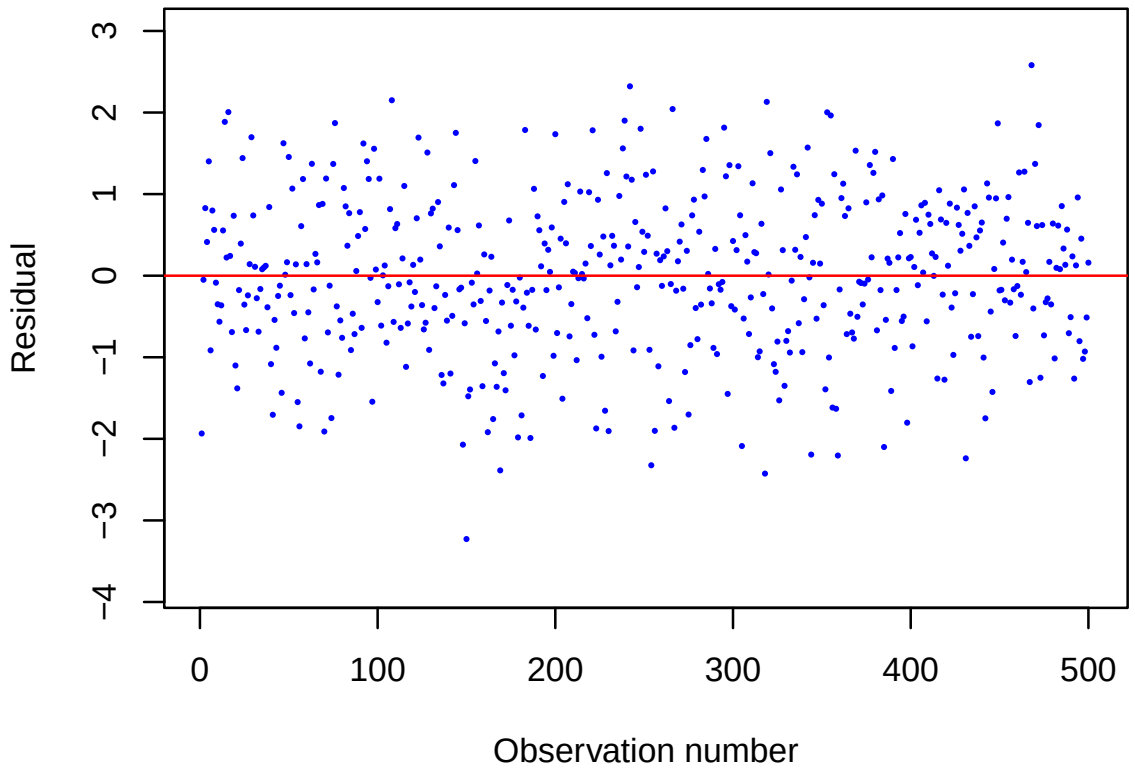
Decting unusual and influential observations

Graphical inspection of residuals is useful to judge the estimated model.

Graphical inspection of the residuals

- Representing units $i = 1, \dots, n$ in the x-axis against the residuals (z_i) in the y-axis

```r
plot(lm1$residuals,
     main = " ",
     xlab = "Observation number",
     ylab = "Residual",
     ylim = c(-3.8, 3),
     xlim = c(0, 502),
     col="blue",
     pch=16,
     cex=0.4)
abline(h=0, col = "red")
```

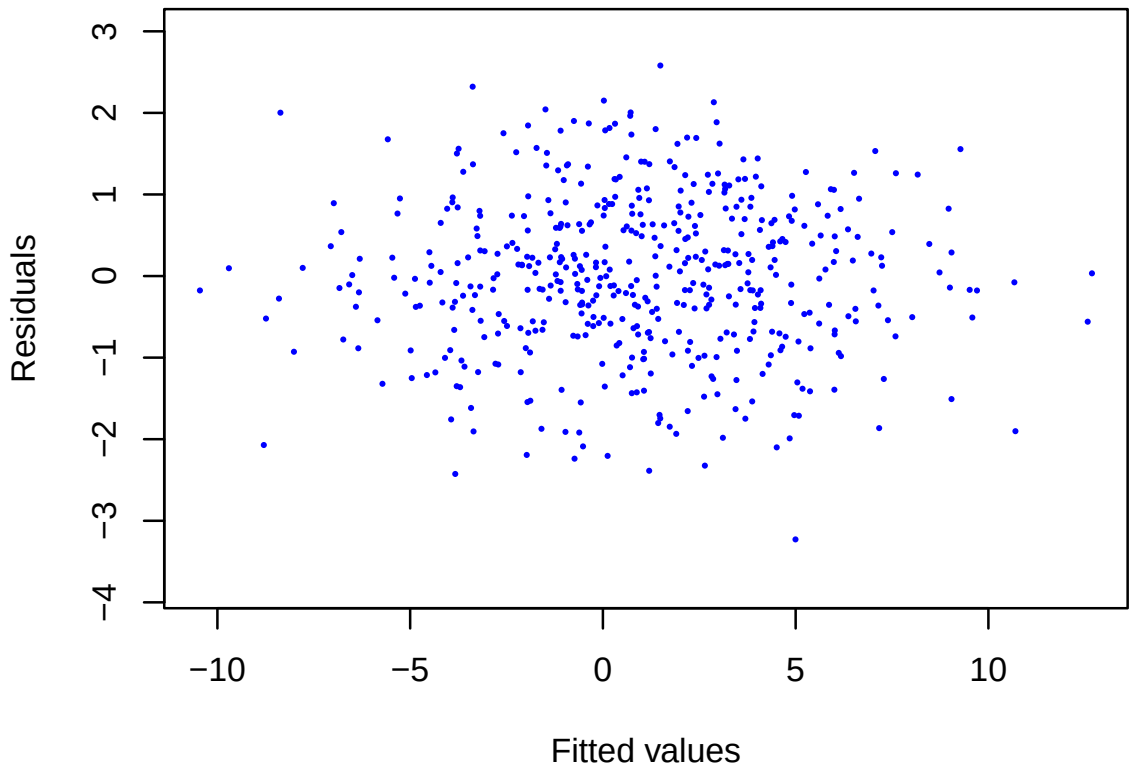


- The range of variation of the residuals is small
- the dispersion of the residuals in the graph does not show any particular pattern
- Deviations from the mean value are balanced: there is a balance between positive and negative residuals.
- There are no particularly distant points from the mean value.

Residual against fitteed values

- The plot of the fitted values ($\widehat{y}_i$) on the x-axis against the residuals (z_i) on the y-axis

```
plot(lm1$fitted,
     lm1$residuals,
     main = " ",
     xlab = "Fitted values",
     ylab = "Residuals",
     ylim = c(-3.8, 3),
     col = "blue",
     pch=16,cex=0.4)
```

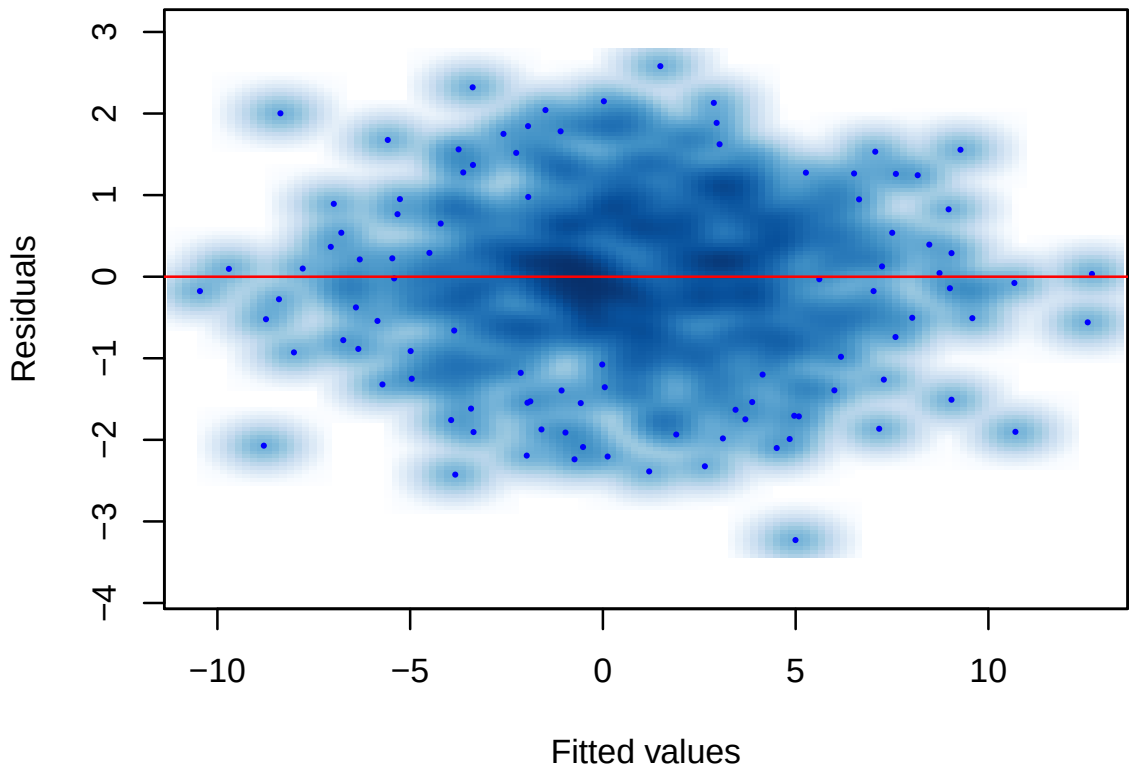


- The points disperse randomly with respect to the centroid.
- There are no patterns and the assumption of constant variance for errors does not appear to be violated.

- There is not a U-shape therefore the assumption of linearity seems to be fulfilled.
- Points form an ellipse which does not have orientation.

The graph can be rendered visually in a different way with, see, among others, the `smoothScatter` function.

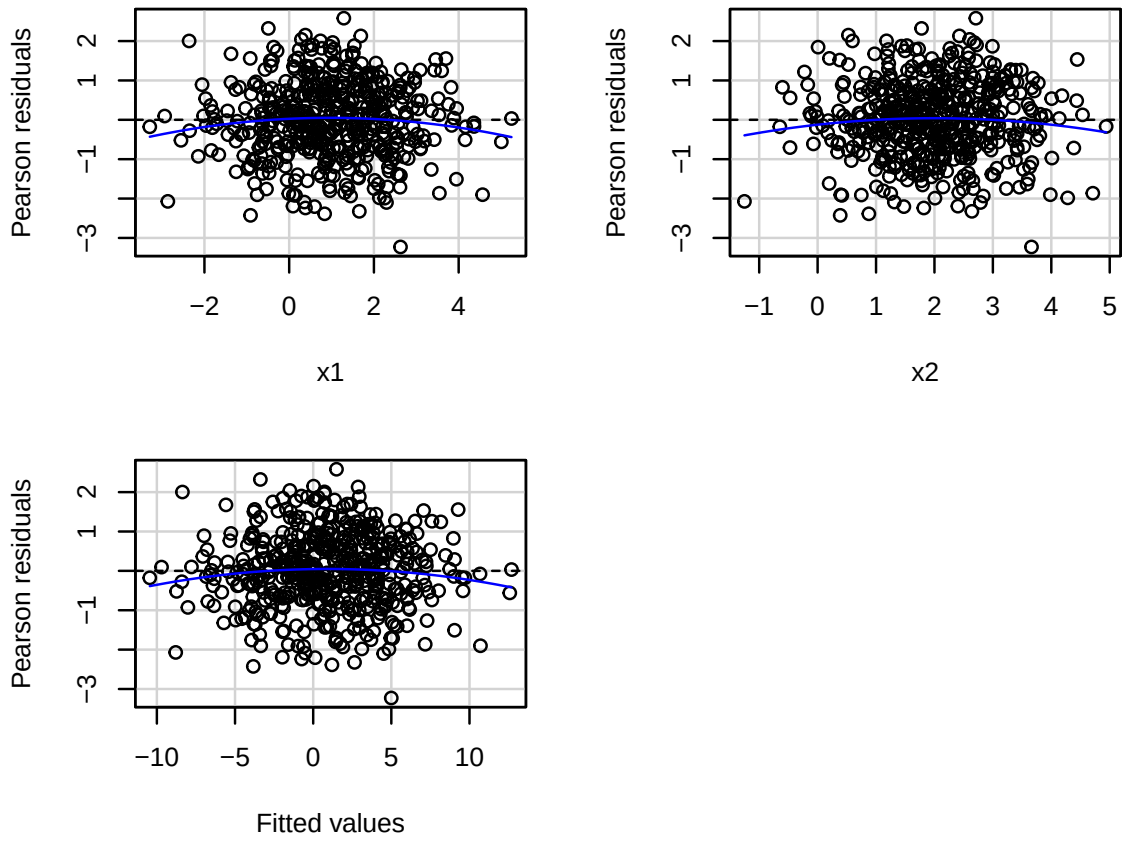
```
smoothScatter(lm1$fitted,
     lm1$residuals,
     main = " ",
     xlab = "Fitted values",
     ylab = "Residuals",
     col="blue",
     ylim = c(-3.8, 3),
     pch=16,cex=0.4)
abline(h=0, col = "red")
```



As can be seen, the shape of this graph is similar to the previous one, but unlike it,

only those points referring to values that are far from the centre of the ellipse formed by the set of points are highlighted.

```
require(car)
residualPlots(lm1)
```

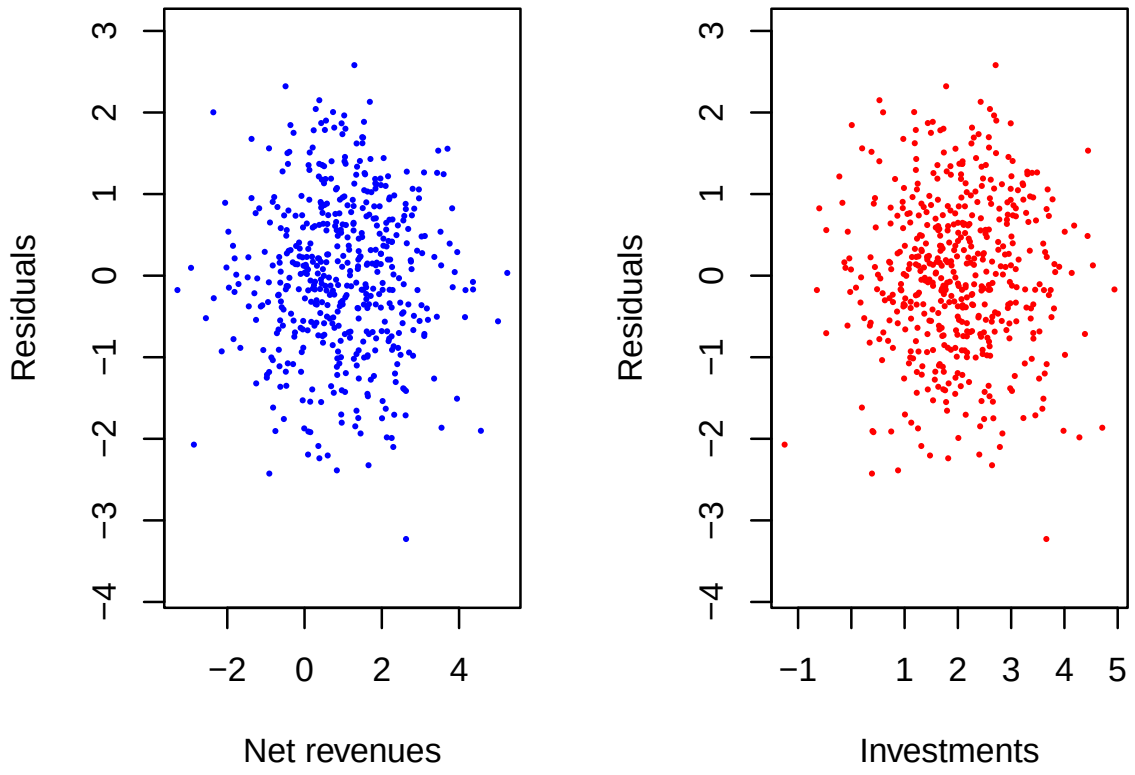



```
##             Test stat Pr(>|Test stat|)
## x1            -1.6977          0.09018 .
## x2            -1.3396          0.18100
## Tukey test    -1.6215          0.10490
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Residuals against covariates

- The plot of the residuals on the y-axis against each explanatory variable on the x-axis

```r
par(mfrow=c(1,2))
plot(profits$x1,
     lm1$residuals,
     main = " ",
     xlab = "Net revenues",
     ylab = "Residuals",
     col="blue",
     ylim = c(-3.8, 3),
     type="p",
     pch=16,cex=0.4)
plot(profits$x2,
     lm1$residuals,
     main = " ",
     xlab = "Investments",
     ylab = "Residuals",
     ylim = c(-3.8, 3),
     col="red",
     type="p",
     pch=16,cex=0.4)
```



- Points are spread randomly around the centre of gravity
- There are no special patterns and the linear association seems plausible.
- There are no values particularly distant from the others.

Empirical cumulative distribution

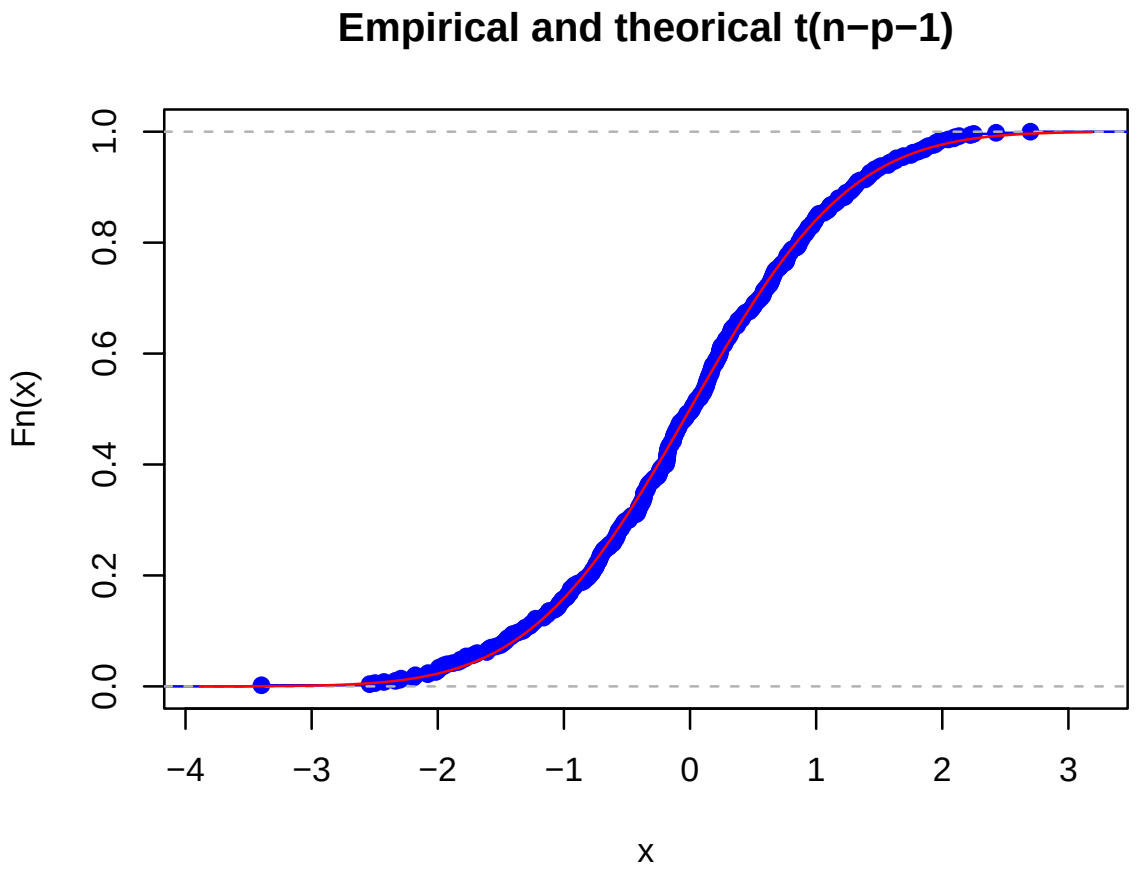
We consider *studentized* residuals (a particular transformation of the residuals) obtained through function `rstudent`

```
rT <- rstudent(lm1)
summary(rT)
##       Min.    1st Qu.     Median       Mean    3rd Qu.       Max.
## -3.3980213 -0.6454384  0.0179223 -0.0002738  0.6795842  2.6989675
```

The expected distribution of this type of residuals is the Student-t distribution with $n-p-2 = 500-2-1 = 497$ degrees of freedom (number of observations minus the number of parameters in the model).

- We compare the empirical distribution with the theoretical one

```
plot(ecdf(rT),
     main ='Empirical and theorical t(n-p-1)',
     col="blue")
curve(pt(x,497),
      col='red',
      add=TRUE)
```



Residuals seem to follow a Student-t distribution; assumptions seem reasonable as the trend in this plot is not far from the red line expected for the residuals.

Remember that the Student-t approaches the standard Normal distribution when the degrees of freedom tend to infinity.

Leverage points

Function `hatvalues` allow us to get the measures of h_{ii} (*hat matrix*) to evaluate observations having a potential leverage effect (high influence on the fit)

```
lev<-hatvalues(lm1)
require(skimr)
```

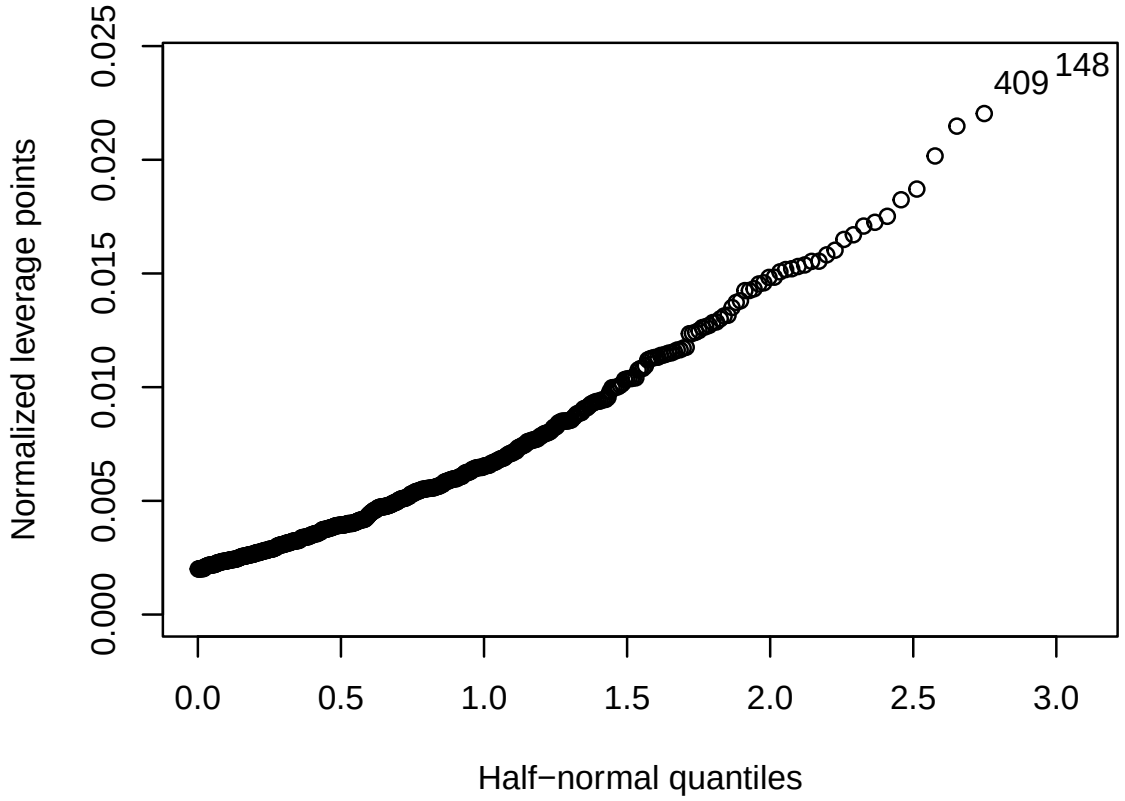
Values are in the range 0.002 e 0.0242. Large values indicate unusual values on the covariates and do not seem to be the case since these values are very low.

Their sum equals the number of parameters $p + 1$

```
sum(lev)
## [1] 3
```

A transformation of them is provided with the function `faraway::halfnorm`, and they are plotted to visualize the id of the units having the largest values.

```
require(faraway)
halfnorm(lev,
         ylab="Normalized leverage points")
```



In this way, we notice that two observations can be potentially influential: those referred to as IDs 148 and 409. However, all the values are very low, and therefore, they are not problematic.

We look at the observed data for such IDs

```
profits[148,]
##              y        x1        x2
## 148 -10.86752 -2.863798 -1.252435
profits[409,]
##             y      x1       x2
## 409 12.01699 5.00802 3.096647
```

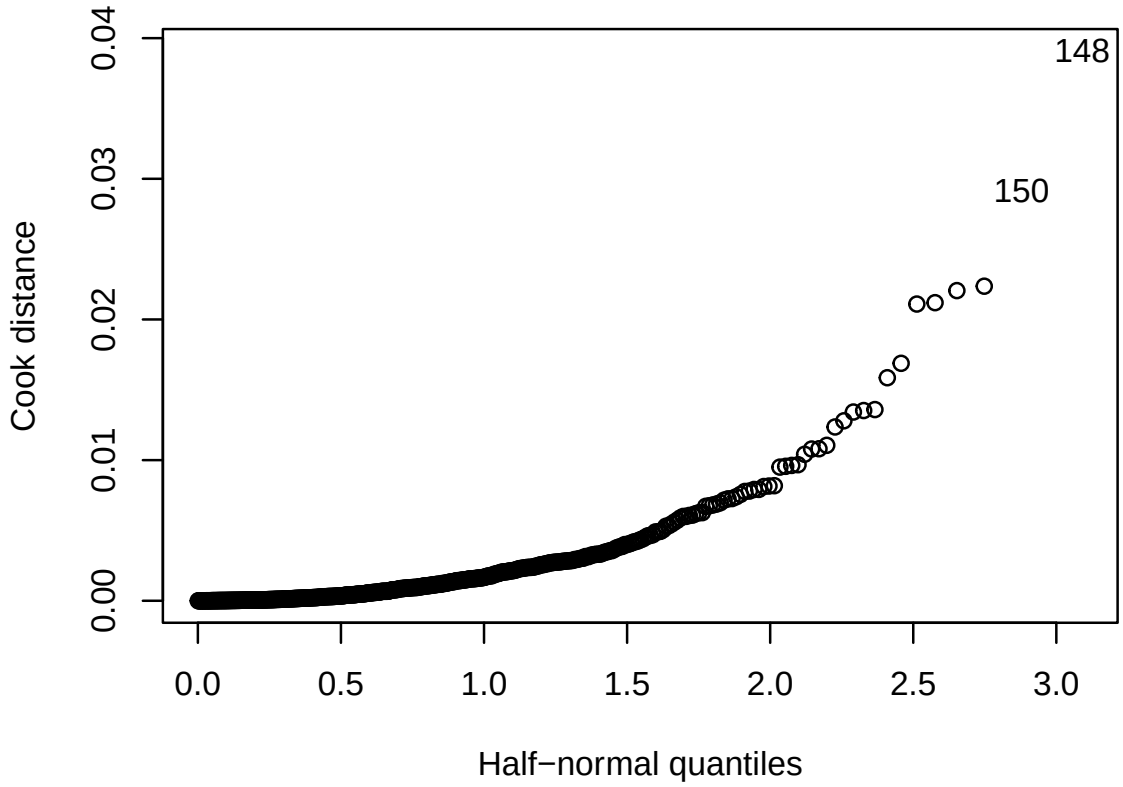
Cook's distance

To calculate this measure we use the function `cook.distance`

```
cook <- cooks.distance(lm1)
summary(cook)
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## 9.000e-09 1.428e-04 7.396e-04 2.018e-03 2.355e-03 3.908e-02
```

Observations that may be influential are those with a Cook's distance above 1. No observation has that large Cook's distance. The plot is the following

```
halfnorm(cook,
         ylab = "Cook distance",  pch=16,
         cex=0.4)
```



Removing one record from the sample

We check if the estimated effects are different when unit 148 is removed from the sample.

`profits[-148,]` exclude line 148

```
lm1a <- lm(y ~ x1 + x2, profits[-148,])
summary(lm1a)
##
## Call:
## lm(formula = y ~ x1 + x2, data = profits[-148, ])
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.2094 -0.6409  0.0159  0.6581  2.5855
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
```

```
## (Intercept) -0.79539    0.10700  -7.433 4.68e-13 ***
## x1           3.04073    0.04491  67.713  < 2e-16 ***
## x2          -0.60569    0.06227  -9.727  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9602 on 496 degrees of freedom
## Multiple R-squared:  0.9386, Adjusted R-squared:  0.9383
## F-statistic:  3791 on 2 and 496 DF,  p-value: < 2.2e-16
summary(lm1)
##
## Call:
## lm(formula = y ~ x1 + x2, data = profits)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.2286 -0.6205  0.0173  0.6531  2.5808
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.82453    0.10657  -7.737 5.72e-14 ***
## x1           3.04361    0.04506  67.551  < 2e-16 ***
## x2          -0.59457    0.06229  -9.545  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9639 on 497 degrees of freedom
## Multiple R-squared:  0.9391, Adjusted R-squared:  0.9389
## F-statistic:  3835 on 2 and 497 DF,  p-value: < 2.2e-16
```

The estimated effects obtained with the full and reduced data are almost identical.

Learning objectives

- Interpret the estimate of the variance of the error terms
- Comment on the accuracy of the estimated regression coefficients
- Calculate confidence intervals for the estimated parameters

Estimation of the variance

The residual deviance, denoted as S^2 is an unbiased estimator of the variance of the error terms σ^2.

It is provided by the summary function

```
load("profits.Rdata")
lm1 <- lm(y ~ x1 + x2, profits)
summary(lm1)
##
## Call:
## lm(formula = y ~ x1 + x2, data = profits)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.2286 -0.6205  0.0173  0.6531  2.5808
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.82453    0.10657  -7.737 5.72e-14 ***
## x1           3.04361    0.04506  67.551  < 2e-16 ***
## x2          -0.59457    0.06229  -9.545  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9639 on 497 degrees of freedom
## Multiple R-squared:  0.9391, Adjusted R-squared:  0.9389
## F-statistic:  3835 on 2 and 497 DF,  p-value: < 2.2e-16
```

The *Residual Standard Error* is S therefore S^2 is given by

```
0.9639^2
## [1] 0.9291032
summ <- summary(lm1)
(summ$sigma)^2
## [1] 0.9290406
```

then $\hat{\sigma}^2 = 0.9291$.

It averages the square residuals with denominator $n-p-1$ (here $p = 2$) so $500-2-1 = 497$.

Note that the sample marginal variance of profits is 3.9 considerably larger than s^2.

In fact, R^2 indicates a reduction of 93.9% in the sum of squared errors from using this prediction equation instead of $\bar{y}$ to predict profits.

Comparison of full and null model

The **null model** (`lm0`) includes only β_0 the intercept, thus explaining the response by a signal plus an error.

Within the F-test, the null model is compared with the full model, including both covariates.

The null model can be estimated as follows

```
lm0 <- lm(y ~ 1, profits)
summary(lm0)
##
## Call:
## lm(formula = y ~ 1, data = profits)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -11.8077  -2.4196   0.0235   2.7543  11.7800
##
```

```
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.9402     0.1744   5.391 1.08e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.899 on 499 degrees of freedom
```

The observed value of the F statistic is equal to 3835 for testing

$$H_0 : \beta_1 = \beta_2 = 0.$$

It is not surprising that there is strong evidence against the null hypothesis, since we have chosen the covariates appropriately and the sample size is large. Therefore, the degrees of freedom of the F statistic are $df_1 = 2$ and $df_2 = 497$. The corresponding p-value is essentially zero.

Thus, there is very strong evidence against the null hypothesis: at least one explanatory variable contributes to explaining the variability in profits.

The test statistic follows a Fisher–Snedecor F distribution with 2 and 497 degrees of freedom.

t-Test

The output provided by the `lm` function reports the estimated standard errors and the observed values of the t statistics.

These standard errors are used both for hypothesis testing and for the construction of confidence intervals.

The test statistic for evaluating

$$H_0 : \beta_k = 0$$

together with the corresponding observed p-value, is reported in the last two columns of the summary output.

```
summary(lm1, digits=3)
##
## Call:
## lm(formula = y ~ x1 + x2, data = profits)
```

```
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.2286 -0.6205  0.0173  0.6531  2.5808
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.82453    0.10657  -7.737 5.72e-14 ***
## x1           3.04361    0.04506  67.551  < 2e-16 ***
## x2          -0.59457    0.06229  -9.545  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9639 on 497 degrees of freedom
## Multiple R-squared:  0.9391, Adjusted R-squared:  0.9389
## F-statistic:  3835 on 2 and 497 DF,  p-value: < 2.2e-16
```

- *Std. Error* is the estimated standard error, $SE(\hat{\beta}_k)$, of each regression coefficient.
- *t-value* is the observed value of the t-test statistic, computed as the estimated coefficient divided by its standard error. For example,

$$t_{\beta_k} = \frac{\hat{\beta}_k}{SE(\hat{\beta}_k)}$$

  when testing

$$H_0 : \beta_k = 0.$$

```
(3.04361-0)/ 0.04506
## [1] 67.54572
```

- *Pr(>|t|)* is the p-value associated with the two-tailed Student-t test under the null hypothesis $H_0 : \beta_k = 0$.
- *Signif. codes* indicate the significance level corresponding to the reported p-value, according to the thresholds shown below the summary output.

- Three stars denote very strong evidence *against* the null hypothesis.
- Note that the symbols (*Signif. codes*) represent significance thresholds, and a highly significant coefficient is commonly indicated by three stars. A dot indicates a p-value in the interval $[0.05, 0.10)$.
- If no symbol appears next to the p-value, then it is greater than 0.1, and the evidence against the null hypothesis is weak or absent.

Confidence intervals for the regression parameters

A 100(1-α)% confidence interval for β_k is obtained considering quantile of Student-t distribution with $(500 - 2 - 1)$ df.

Fixing $\alpha = 0.05$ the quantile is

```
qt(0.975,497)
## [1] 1.964749
```

The lower and upper bounds are calculated as

```
3.04+c(-1,1)*qt(0.975,497)*0.045
## [1] 2.951586 3.128414
```

The expected increase in profits associated with a one-unit increase in revenues lies between 3 and 3.12 at the 95% confidence level (although values outside this interval are still possible).

The `confint` function provides the observed confidence bounds for all the model parameters, either at the default confidence level or at user-specified confidence levels, as follows.

```
confint(lm1)
##                  2.5 %     97.5 %
## (Intercept) -1.0339145 -0.6151505
## x1           2.9550872  3.1321372
## x2          -0.7169618 -0.4721796
```

```
confint(lm1, level=(1 - 0.10))
##                    5 %       95 %
## (Intercept) -1.0001509 -0.6489141
## x1           2.9693622  3.1178622
## x2          -0.6972257 -0.4919157
confint(lm1, level=(1 - 0.01))
##                   0.5 %     99.5 %
## (Intercept) -1.1000951 -0.5489699
## x1           2.9271066  3.1601178
## x2          -0.7556466 -0.4334948
```

The **length** of the interval is

```
0.71-0.47
## [1] 0.24
```

Learning objectives

- Check for multicollinearity using the Variance Inflation Factor (VIF)
- Perform data-driven model selection using the Akaike Information Criterion

Multicollinearity: Variance inflation factor

VIF (Variance Inflation Factor) is a measure that can be computed using the vif function available in the faraway package (a companion to Applied Regression).

First, we check for variance inflation in the linear regression model fitted to the profit data.

```
load("profits.Rdata")
lm1 <- lm(y ~ x1 + x2, profits)
library(car)
faraway::vif(lm1)
##       x1       x2
## 2.056309 2.056309
```

There is little variance inflation associated with each covariate, since all values are relatively low.

Based on these values, the estimated standard error for the coefficient associated with revenues is $\sqrt{2.06}$ times larger than it would be without collinearity between the two covariates. We therefore conclude that the estimates exhibit substantial stability.

A VIF greater **than 10** is often considered an indication of problematic multicollinearity, although the threshold is context-dependent. In such cases, corrective measures may be needed because multicollinearity inflates the standard errors of the regression coefficients, making the estimates less precise

Moreover, perfect multicollinearity implies that the coefficient estimates are not uniquely identified

Model selection: Data-driven approach

The function `stats::step` function allows you to apply an automatic model selection procedure, such as forward, backward, or stepwise selection, using a data-driven

approach based on the **Akaike information criterion (AIC)**.

The `both` option in the `step` function is the default. Starting from the null model, the procedure evaluates models by adding or removing one covariate at a time. At each step, the model with the lowest AIC among the estimated models is selected.

```
step(lm1)
## Start:  AIC=-33.81
## y ~ x1 + x2
##
##        Df Sum of Sq    RSS     AIC
## <none>               461.7  -33.81
## - x2    1      84.6  546.4   48.34
## - x1    1    4239.3 4701.0 1124.47
##
## Call:
## lm(formula = y ~ x1 + x2, data = profits)
##
## Coefficients:
## (Intercept)           x1           x2
##     -0.8245       3.0436      -0.5946
```

None of the estimated models has an AIC value lower than that of the full model.

Example: Percentage of body fat

Data in `body.Rdata` (from `faraway`) are related to 248 men, see, * Johnson, R. (1996), Fitting percentage of body fat to simple body measurements. *Journal of Statistics Education*, **4**: 265-266.

Each man's percentage of body fat was accurately estimated using an underwater weighing technique. The aim of the study is to explain body fat percentage based on a number of body measurements.

The observations relate to the following variables:

- *brozek*: Percent body fat using Brozek's equation,
- *age*: age in years,
- *weight*: weight (lbs),

- *height*: height, (in inches),
- *adipos*: adiposity index (kg/m^2),
- *neck*: neck circumference (in cm),
- *abdom*: abdomen circumference (cm) at the umbilicus and level with the iliac crest (in cm),
- *hip*: hip circumference (in cm),
- *thigh*: thigh circumference (in cm),
- *wrist*: wrist circumference distal to the styloid processes (in cm),

```
load("body.Rdata")
require(skimr)
```

First, we estimate the full model including all the available covariates and check the VIF values.

Descriptive statistics

```
skim_without_charts(body)
```

Table 15: Data summary

Name	body
Number of rows	248
Number of columns	10
Column type frequency:	
numeric	10
Group variables	None

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100
brozek	0	1	18.83	7.69	0.0	12.80	19.00	24.50	45.10
age	0	1	44.85	12.60	22.0	35.75	43.00	54.00	81.00
weight	0	1	178.11	27.13	118.5	158.19	176.12	196.81	262.75
height	0	1	70.30	2.61	64.0	68.25	70.00	72.25	77.75
adipos	0	1	25.33	3.34	18.1	23.10	24.95	27.30	39.10
neck	0	1	37.95	2.30	31.1	36.38	38.00	39.42	43.90
abdom	0	1	92.31	10.24	69.4	84.47	90.95	99.20	126.20
hip	0	1	99.66	6.47	85.0	95.47	99.30	103.28	125.60
thigh	0	1	59.27	4.92	47.2	56.00	59.00	62.30	74.40
wrist	0	1	18.22	0.92	15.8	17.60	18.30	18.80	21.40

The multiple linear regression model is intended to explain the **body fat percentage (Brozek)**, derived using the equation based on body density (Brozek's equation: 457/Density − 414.2).

Main features:

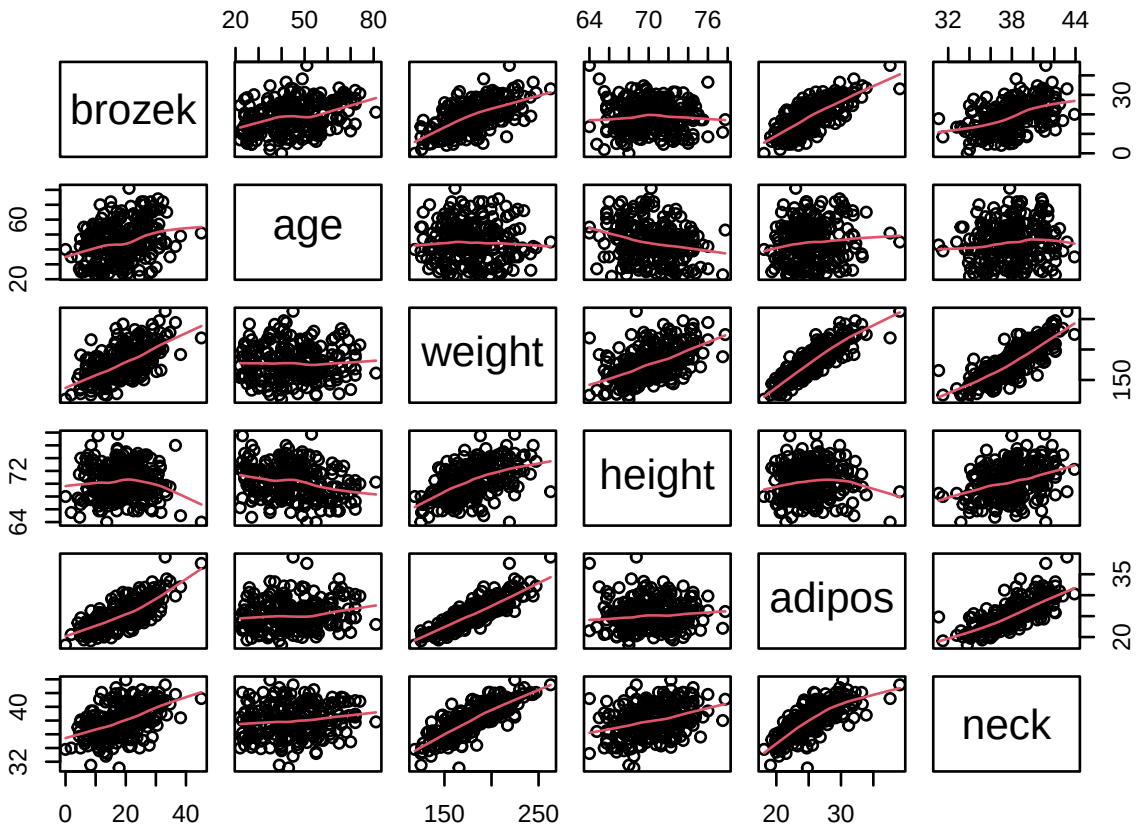
-The observed body fat percentage ranges from 0 to 45, with a mean of 19 and a standard deviation of about 8.

- The men are, on average, 45 years old, and their ages range from 22 to 81. There are 9 individuals over 70 years of age. The standard deviation of age is about 13.
- Body weight shows the greatest variability, with a mean of 178 and a standard deviation of 27.

Provide other comments on the observed data

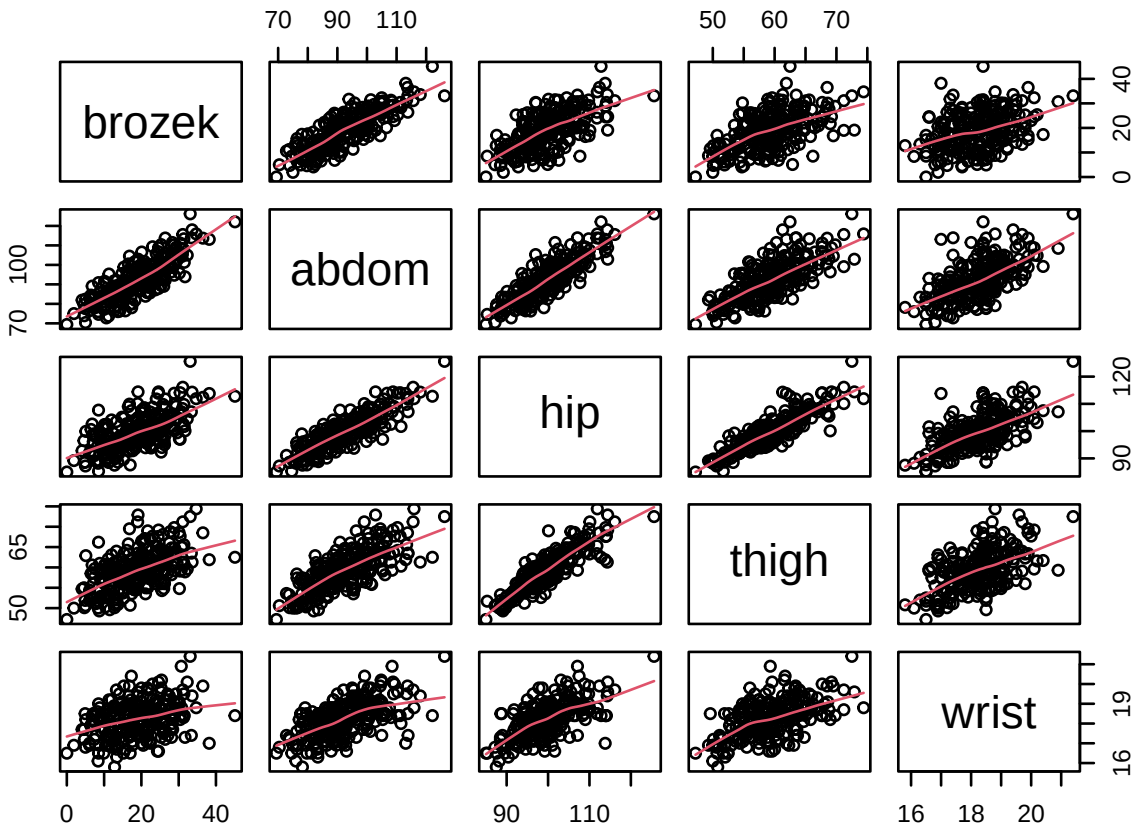
Scatter plot matrices are produced for two groups of variables, with the response variable included in each group.

```
pairs(body[,1:6],
      panel = panel.smooth,
      main = "Body")
```



In particular, there is a strong positive linear association between body fat, weight, and the adiposity index.

```
pairs(body[,c(1,7:10)],
      panel = panel.smooth,
      main = "Body")
```



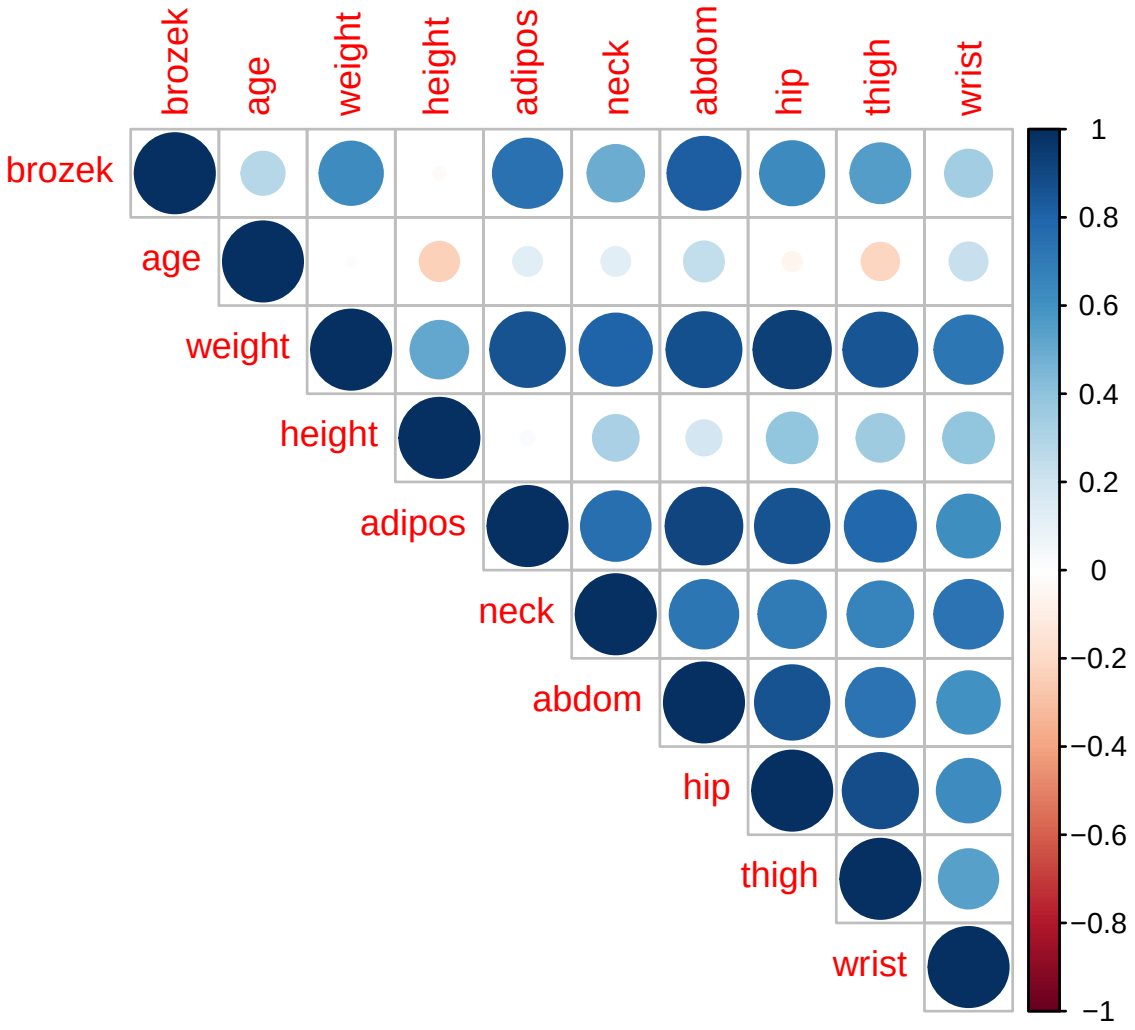
There are several strong pairwise correlations among the covariates and between the covariates and the response.

A particularly noticeable feature is the positive linear association between body fat and all the other variables.

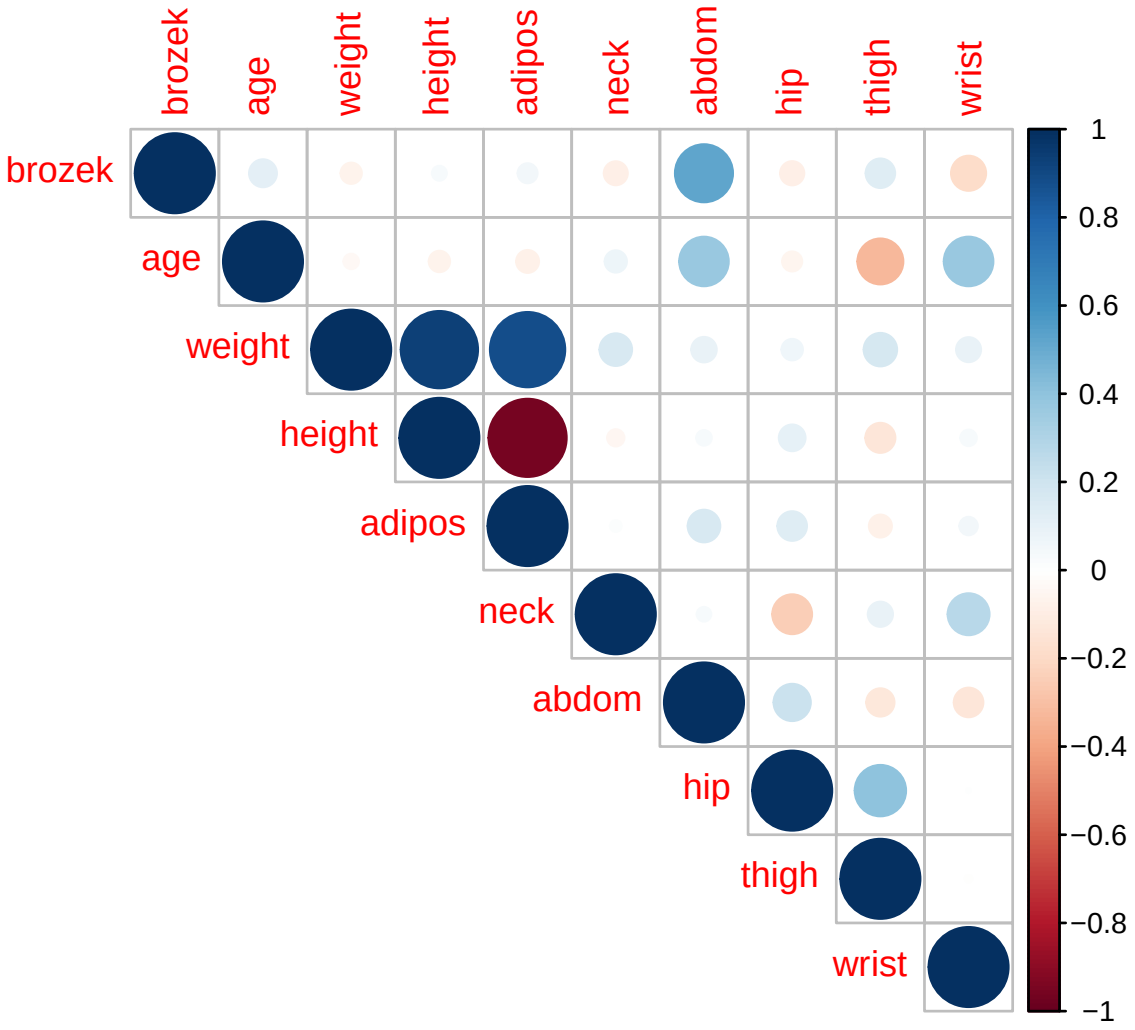
Overall, the explanatory variables also exhibit linear associations with one another.

We depict the raw and partial **correlations**

```
require(ggm)
S <- cov(body)
Rp <- parcor(S)
corrplot::corrplot(cor(body),
                   main=" ", type="upper")
```




```
corrplot::corrplot(Rp,
                   main=" ", type="upper")
```



- The partial correlation between the response and abdom (adipose fat) is high and positive.
- Weight is highly and positively correlated with height and abdom, after controlling for the other variables.
- There is a strong negative partial correlation (-0.95) between adipose fat and height, after controlling for the other variables.
- The partial correlation between weight and height is high and positive.
- The partial correlation between body fat percentage and wrist circumference is negative, although modest (-0.19), indicating a weak inverse linear association between the two variables after controlling for all the others.

Provide other comments on these results

Variance inflation factor

```
require(faraway)
modbody0 <- lm(brozek ~ ., body)
vif(modbody0)
##        age     weight     height     adipos       neck      abdom        hip
##   2.100355 120.312595  27.835462  87.449527   3.743385  11.225091  11.797629
##      thigh      wrist
##   5.822000   2.958093
```

There is much variance inflation. For example, **we can interpret $\sqrt{120.32} = 10.89$ as telling us that the standard error for weight with percentage of fat is around 11 times larger than it would have been without collinearity**.

This measure reflects the magnitude of the effect of covariates that are close to being linearly dependent.

Since the model includes many variables, some of them can probably be omitted without substantially affecting the prediction of the response variable.

We exclude weight and calculate vif

```
modbody1 <- lm(brozek ~ ., body[,-3])
vif(modbody1)
##       age    height    adipos      neck     abdom       hip     thigh      wri
##  2.096498  3.635138 19.231381  3.636931 11.164804 11.731423  5.666029  2.923
```

The values are now more reasonable.

In this case, the highest VIF value is associated with `adipos`. Next, we proceed with variable selection.

Variable selection: data-driven approach

For variable selection, we employ the *step function*, which uses a search procedure that compares models sequentially on the basis of the AIC.

In this case, the default procedure is *backward* selection: starting from the full model, one variable is removed at each step.

```
step(modbody1)
## Start:  AIC=688.8
## brozek ~ age + height + adipos + neck + abdom + hip + thigh +
##     wrist
##
##          Df Sum of Sq    RSS    AIC
## - adipos  1      0.00 3708.1 686.80
## - height  1     26.80 3734.9 688.59
## <none>                3708.1 688.80
## - hip     1     32.15 3740.3 688.94
## - neck    1     36.19 3744.3 689.21
## - age     1     54.56 3762.7 690.43
## - thigh   1     56.56 3764.7 690.56
## - wrist   1    147.30 3855.4 696.46
## - abdom   1   1371.12 5079.2 764.83
##
## Step:  AIC=686.8
## brozek ~ age + height + neck + abdom + hip + thigh + wrist
##
##          Df Sum of Sq    RSS    AIC
## <none>                3708.1 686.80
## - hip     1     39.21 3747.3 687.41
## - neck    1     41.59 3749.7 687.57
## - thigh   1     57.87 3766.0 688.64
## - age     1     57.93 3766.1 688.65
## - height  1     64.03 3772.1 689.05
## - wrist   1    161.36 3869.5 695.37
## - abdom   1   2236.50 5944.6 801.85
##
## Call:
## lm(formula = brozek ~ age + height + neck + abdom + hip + thigh +
##     wrist, data = body[, -3])
##
## Coefficients:
## (Intercept)          age       height         neck        abdom          hip
##     6.06011      0.05411     -0.23857     -0.31873      0.76777     -0.19186
##       thigh        wrist
##     0.23116     -1.44066
```

Start: AIC = 688.8 indicates that all the variables are included in the model.

Step: AIC = 686.8 indicates the AIC value obtained when adipos is removed from the model.

Looking at the second panel, we observe that the AIC values are always higher when one of the variables is removed from the model estimated at the second step.

We now calculate the VIF values for the selected model, which also excludes `adipos`.

```
modbody2 <- lm(brozek ~ ., body[,-c(3,5)])
vif(modbody2)
##      age   height     neck    abdom      hip    thigh    wrist
## 1.981290 1.497954 3.184032 6.832682 9.695463 5.525880 2.675081
```

Only `hip` shows a VIF value close to 10. We can now examine and comment on the estimated parameters of this model.

```
summary(modbody2)
##
## Call:
## lm(formula = brozek ~ ., data = body[, -c(3, 5)])
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -9.6963 -2.6424 -0.3515  3.0084  9.7674
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  6.06011    7.74995   0.782   0.4350
## age          0.05411    0.02794   1.936   0.0540 .
## height      -0.23857    0.11719  -2.036   0.0429 *
## neck        -0.31873    0.19426  -1.641   0.1022
## abdom        0.76777    0.06381  12.031   <2e-16 ***
## hip         -0.19186    0.12044  -1.593   0.1125
## thigh        0.23116    0.11944   1.935   0.0541 .
## wrist       -1.44066    0.44580  -3.232   0.0014 **
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.931 on 240 degrees of freedom
## Multiple R-squared:  0.746,  Adjusted R-squared:  0.7386
## F-statistic: 100.7 on 7 and 240 DF,  p-value: < 2.2e-16
```

- **Residuals** vary between -10 and 10; this range is not excessive.
- The estimated **regression coefficients** for neck, hip, and wrist are negative. The partial regression coefficient for wrist is the largest in absolute value, indicating that body fat decreases by 1.44 percentage points for each additional centimetre of wrist circumference.
- Body fat increases by 0.73 percentage points for a one-unit increase in abdominal circumference, holding the other covariates constant.
- The **estimated standard errors** are quite small, and the values of the t-statistics are not particularly high.
- The **p-values** associated with the individual t-tests indicate that we cannot reject the null hypothesis for neck and hip at the usual significance levels.
- The observed value of the **F-statistic** is around 100, and the corresponding distribution has 7 and 240 degrees of freedom.
- The null hypothesis that all regression coefficients, excluding the intercept, are jointly equal to zero is rejected at the 90%, 95%, and 99% confidence levels. There is strong evidence against the null model.

Prediction

Suppose we want to predict the body fat percentage of an out-of-sample individual (a man) with the following values for the selected covariates:

- age: 38 years
- height: 68
- neck: 35
- abdom: 95
- hip: 101

- thigh: 59
- wrist: 18

First, we place the values of the covariates into a vector called `xc`, in the same order as used to estimate the multiple linear regression model. Then, using `names`, we assign the variable names as labels; these should match the names used the data frame.

```
xc<-c(1,38,68, 35, 95,101,59,18)
names(xc)<-c("int",
             "age",
             "height",
             "neck",
             "abdom",
             "hip",
             "thigh",
             "wrist")
```

Given this set of covariate values, the uncertainty associated with predicting a future observation can be assessed using a prediction interval. We aim to obtain both a *point estimate* of body fat and a 95% prediction interval.

Using the `predict` function, we obtain as output a point estimate together with a 95% prediction interval for a *single future response*.

The function requires its second argument to be a data frame whose variable names match those in the original dataset.

Note that the new values should be consistent with those in the original sample; in particular, they should lie within (or close to) the range of the observed covariates, under the assumption that they come from the same population.

```
predict(modbody2,
        new=data.frame(t(xc)),
        interval = "prediction")
##        fit      lwr      upr
## 1 22.00484 14.13119 29.87849
```

The predicted body fat percentage for a **new** man with these measurements of the body is 22.

The estimated 95% prediction interval is wide, $[14.13, 29.88]$, indicating substantial uncertainty in the prediction. In other words, with 95% confidence, the body fat percentage for this man is expected to lie withing the corresponding interval.

Prediction of mean response

Similarly, the function allows us to obtain an estimate of the mean response using the `confidence` option

We are now considering the question: *what would the average of an individual with characteristics x_c be?*

```
predict(modbody2,
        new=data.frame(t(xc)),
        interval = "confidence")
##        fit      lwr      upr
## 1 22.00484 20.57699 23.43268
```

The point estimate is the same as the previous one, while the interval in the last case is much less wide $[21.0, 23.4]$ at the same confidence level.

Learning objectives

- Estimate and interpret the effect of a binary covariate in a multiple linear model
- Plot the corresponding regression lines
- Assess the presence of interaction effects
- Estimate and interpret the effects of categorical covariate levels

Dummy covariate

The data used in this example come from a 1997 study involving 79 adult females patients treated at a clinic; see the article published in *Journal of Consulting and Clinical Pyschology*. The data are available in the `faraway::sexab` dataset.

The aim to analyze post-traumatic stress disorder (`ptsd`), measured on a continuous scale, in relation to childhood physical abuse (`cpa`), measured on a continuous standardized scale, and sexual abuse (`csa`), coded as a binary yes/no variable.

```
require(faraway)
data(sexab, package="faraway")
require(sexab)
require(skimr)
my_skim<- skim_with(base=sfl(),numeric=sfl(hist=NULL))
my_skim(sexab)
```

Table 17: Data summary

Name	sexab
Number of rows	76
Number of columns	3
Column type frequency:	
factor	1
numeric	2
Group variables	None

Variable type: factor

skim_variable	ordered	n_unique	top_counts
csa	FALSE	2	Abu: 45, Not: 31

Variable type: numeric

skim_variable	mean	sd	p0	p25	p50	p75	p100
cpa	2.35	2.37	-3.12	0.83	2.07	3.74	8.65
ptsd	8.99	4.97	-3.35	6.17	8.91	12.24	18.99

- In the sample average post traumatic stress disorder score `ptsd` is about 9, with a standard deviation of about 5. The minimum observed score is negative -3.35 and the maximum is 19.
- The average score for childhood physical abuse (`cpa`) is 2.35; the lowest observed value in the sample is -3.12 and the highest is 8.64.
- The following output shows the character *labels*, and corresponding numeric *codes* associated with them

```
factor(sexab$csa[1:10])
##  [1] Abused Abused Abused Abused Abused Abused Abused Abused Abused Abused
## Levels: Abused
as.integer(sexab$csa[1:10])
##  [1] 1 1 1 1 1 1 1 1 1 1
```

Note that the variable `csa` is a coded as factor in the data and has `levels` referring to: `Abused` associated with value 1, `NotAbused` with value 2.

There are 45 abused women, while 31 did not suffer sexual violence as children.

The `relevel` function allows you to change the reference level

```
sexab$csa <- relevel(sexab$csa, ref="NotAbused")
as.integer(sexab$csa)
##  [1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
## [39] 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
table(sexab$csa)
##
## NotAbused    Abused
##        31        45
head(sexab$csa)
## [1] Abused Abused Abused Abused Abused Abused
## Levels: NotAbused Abused
```

Summary by the categorical variable

We check for differences among the two groups

```
by(sexab,sexab$csa,summary)
## sexab$csa: NotAbused
##       cpa                ptsd               csa
##  Min.   :-3.1204   Min.   :-3.349   NotAbused:31
##  1st Qu.:-0.2299   1st Qu.: 3.544   Abused   : 0
##  Median : 1.3216   Median : 5.794
##  Mean   : 1.3088   Mean   : 4.696
##  3rd Qu.: 2.8309   3rd Qu.: 6.838
##  Max.   : 5.0497   Max.   :10.914
## ------------------------------------------------------------
## sexab$csa: Abused
##       cpa              ptsd              csa
##  Min.   :-1.115   Min.   : 5.985   NotAbused: 0
##  1st Qu.: 1.415   1st Qu.: 9.374   Abused   :45
##  Median : 2.627   Median :11.313
##  Mean   : 3.075   Mean   :11.941
##  3rd Qu.: 4.317   3rd Qu.:14.901
##  Max.   : 8.647   Max.   :18.993
```

Individuals in the abused group show higher levels of PTSD than those in the non-

abused group.

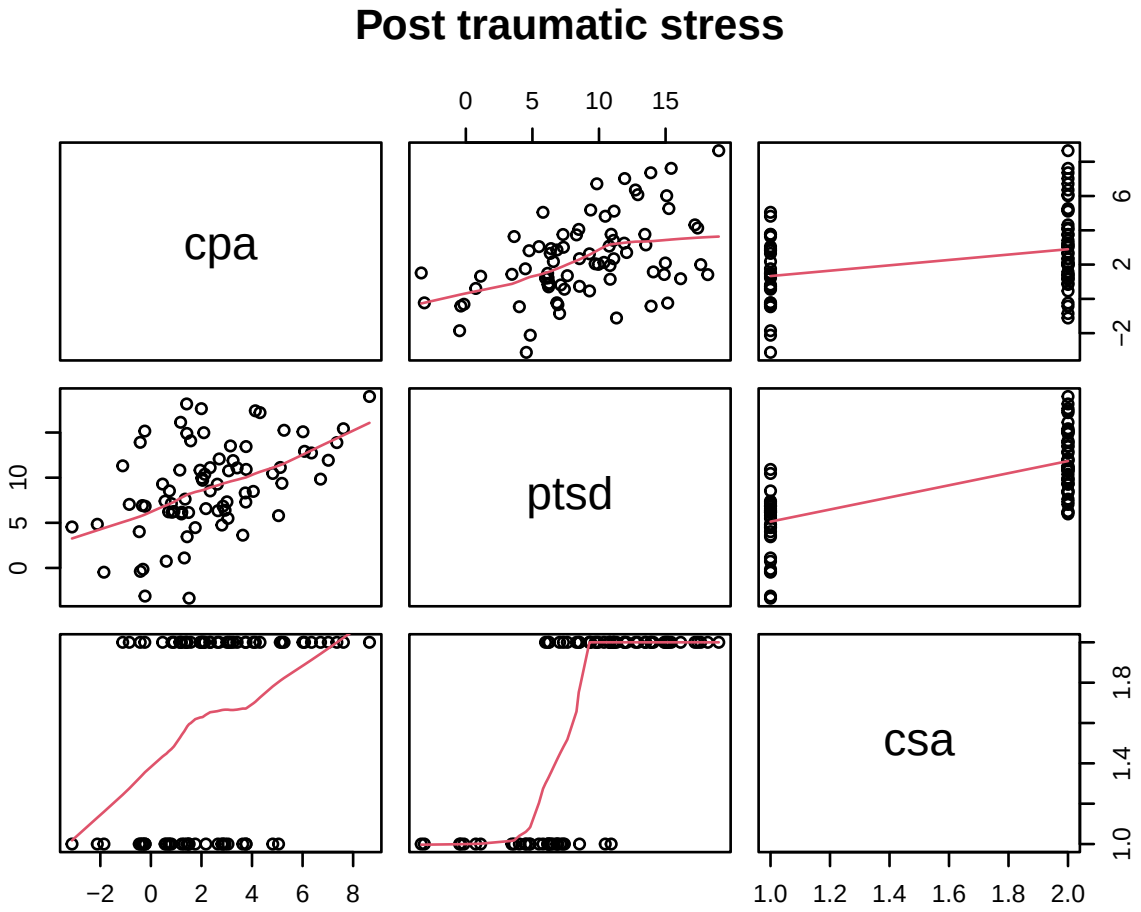
A t-test may be used to test whether the mean PTSD scores differ between the two groups (see the Welch two-sample t-test).

However, childhood physical abuse may be a confounding factor, since it could partly explain the higher stress levels observed among individuals who were also sexually abused.

Plot

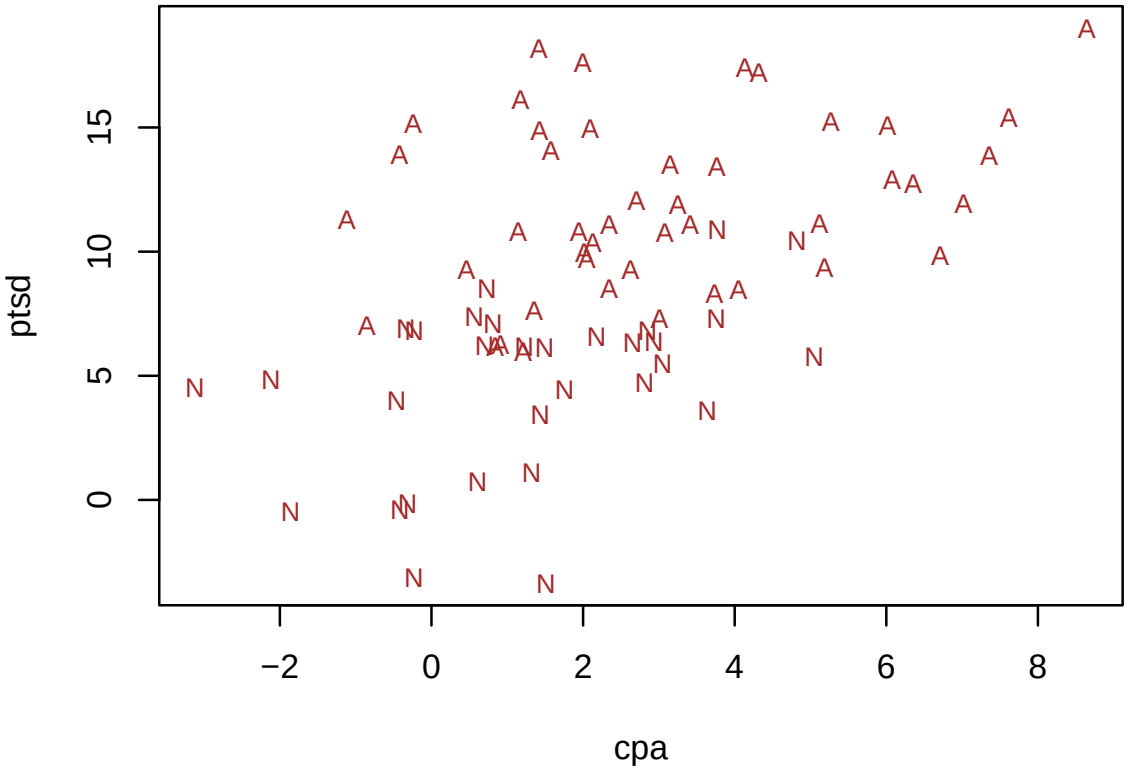
The scatter plot matrix informs that the association of the response with the level of physical abuse `cpa` is linear, and that (second row third column) the values of `cpa` for `csa=2` are on average higher.

```
pairs(sexab,
      panel = panel.smooth,
      main = "Post traumatic stress")
```



The following plot displays all three variables, using the `pch` argument to distinguish the binary variable through different symbols

```
plot (ptsd ~ cpa,
      pch=as.character (csa),
      sexab,
      col= c("brown"),
      cex = 0.8)
```

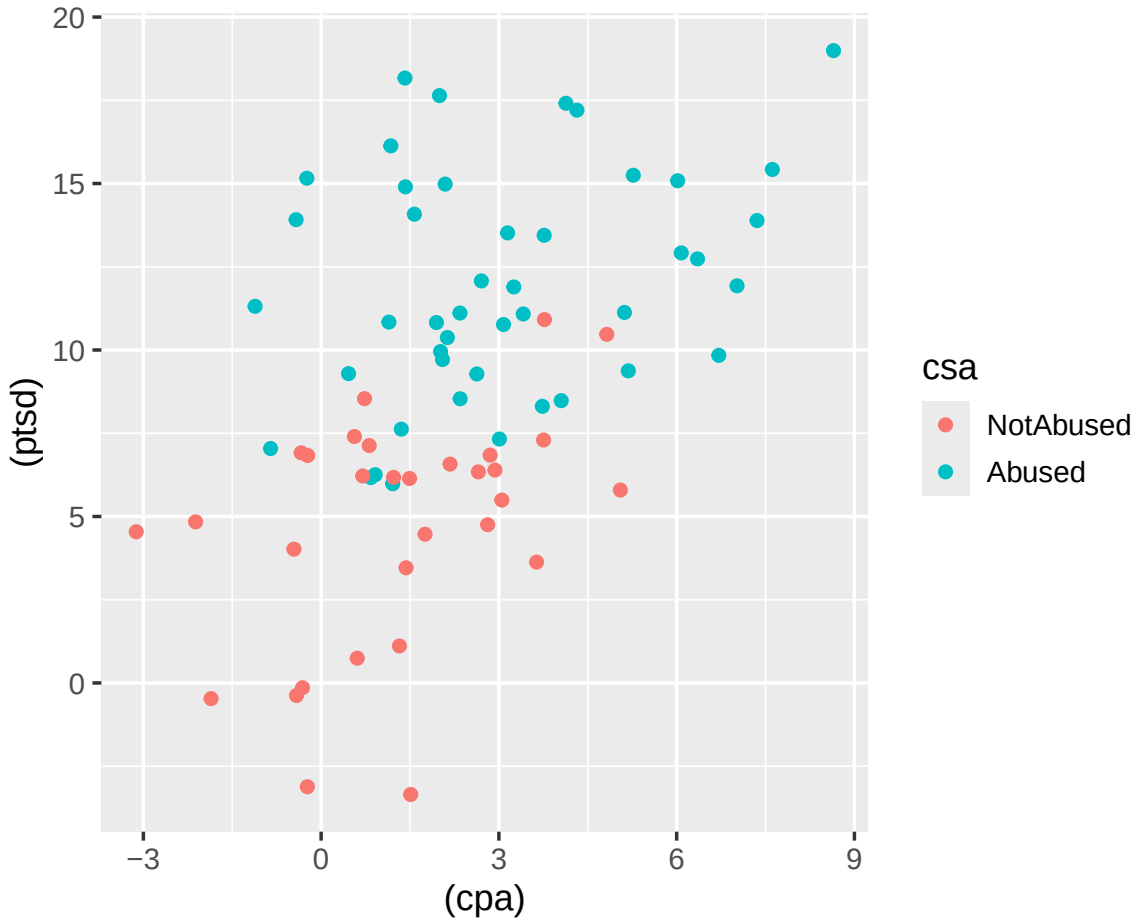


The scatterplot of `ptsd` against the physical abuse score also distinguishes observations according to the third variable, namely sexual abuse status, using different symbols and colours: A for abused (blue) and N for not abused (brown).

The figure clearly shows that the observations labelled A tend to lie above those labelled N. In other words, for the same level of physical abuse, women who were also sexually abused in childhood tend to report higher levels of stress. Furthermore, the highest values of `cpa` are observed among those in the abused group.

A similar plot is shown below.

```
require(ggplot2)
ggplot(data = sexab, aes((cpa), (ptsd),
color = csa)) +
geom_point()
```



The ANCOVA model helps to disentangle the effect of the continuous variable and of the categorical one.

First, we look at the model matrix

```
lm1 <- lm(ptsd ~ cpa+csa,sexab)
head(model.matrix(lm1))
##   (Intercept)      cpa csaAbused
## 1           1  2.04786         1
## 2           1  0.83895         1
## 3           1 -0.24139         1
## 4           1 -1.11461         1
```

```
## 5           1  2.01468         1
## 6           1  6.71131         1
```

notAbused is the reference level coded with 0 and the parameter represent the difference between Abused and the reference level.

```
summary(lm1)
##
## Call:
## lm(formula = ptsd ~ cpa + csa, data = sexab)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -8.1567 -2.3643 -0.1533  2.1466  7.1417
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   3.9753     0.6293   6.317 1.87e-08 ***
## cpa           0.5506     0.1716   3.209  0.00198 **
## csaAbused     6.2728     0.8219   7.632 6.91e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.273 on 73 degrees of freedom
## Multiple R-squared:  0.5786, Adjusted R-squared:  0.5671
## F-statistic: 50.12 on 2 and 73 DF,  p-value: 2.002e-14
```

The slope of both lines is 0.551, but the Abused line is 6.273 higher than the NotAbused.

The regression line for notAbused is the following

$$\hat{Y}_N = 3.97 + 0.55 * cpa$$

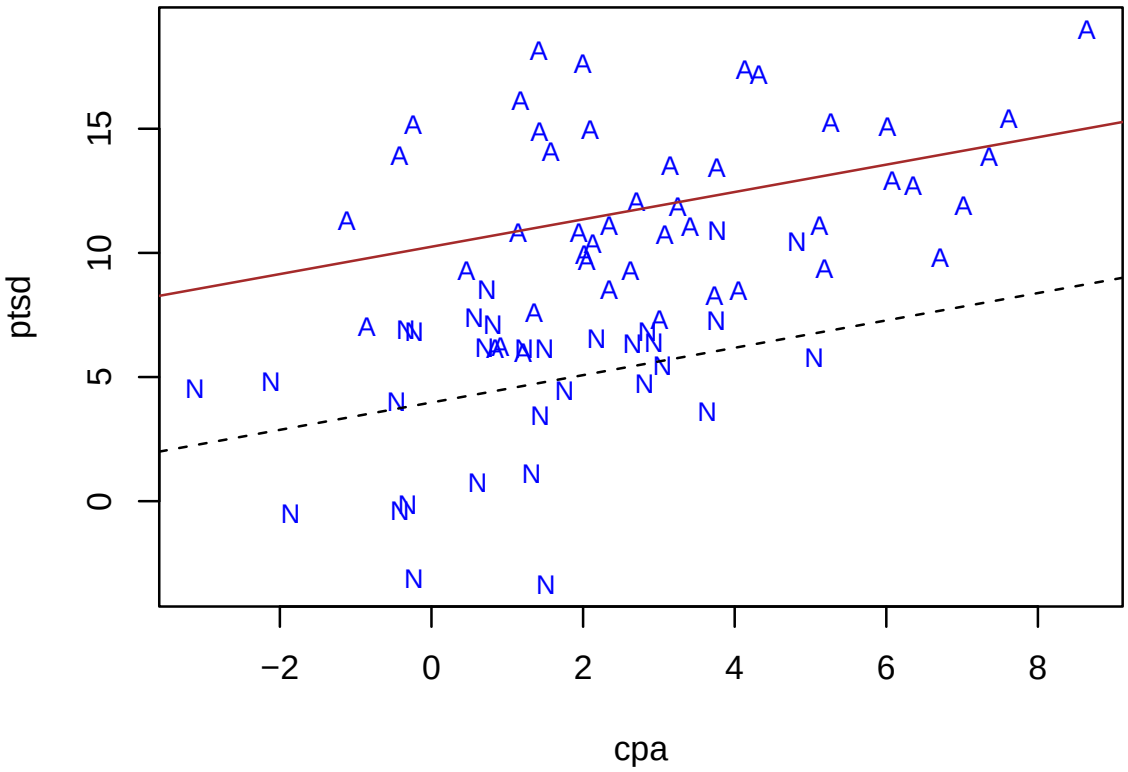
And those of Abused is

$$\hat{Y}_A = (3.97 + 6.27) + 0.55 * cpa$$

$$\hat{Y}_A = 10.24 + 0.55 * cpa$$

We can add the two parallel lines to the plot

```
plot (ptsd ~ cpa,
      pch=as.character (csa),
      sexab,
      col = c("blue"),
      cex = 0.8)
abline (10.248, 0.551, col = "brown")
abline (10.248-6.273, 0.551, lty=2)
```



The intercepts are different, whereas the slopes of the two regression lines are the same.

The vertical distance between the two lines is equal to the estimated coefficient of the dichotomous variable.

Since the lines are parallel, the value 6.27 represents the main effect of the dichotomous variable and corresponds to the constant vertical distance between the two regression lines for any fixed value of the continuous covariate.

The value 0.55 represents the main effect of the continuous covariate (`cpa`), regardless of the level of the dummy variable.

Thus, after controlling for sexual abuse, the association between the childhood physical abuse score (`cpa`) and the post-traumatic stress disorder score (`y`) is positive: the expected `PTSD` score increases by 0.55 points for each one-unit increase in physical abuse, regardless of whether the woman has or has not been sexually abused.

The regression line for women who have been sexually abused has a higher intercept: when physical abuse is zero, the expected value of the response is substantially higher for women who have also experienced sexual abuse.

The confidence intervals for the regression parameters at the 95% confidence level are obtained using the `confint` function

```
confint(lm1)
##                 2.5 %    97.5 %
## (Intercept) 2.7210704 5.2294976
## cpa         0.2085843 0.8925201
## csaAbused   4.6346959 7.9108087
```

The R^2 is 0.567, indicating that 57% of the variability of the post-traumatic stress disorder score is explained by the combined effects of the physical abuse score and the variable indicating whether the woman was sexually abused at school age.

Based on the F-test we reject the null hypothesis at all conventional significance levels.

Interaction terms

We also may consider including interaction terms. In the presence of an interaction between two variables, it is often useful to center the continuous variable around its mean. *This allows for a more meaningful interpretation of the model intercept, which*

can be then understood as the expected mean value corresponding to the reference level of the dichotomous variable.

The model with an interaction term can be estimated by using * instead of +. In this case, a new variable is included in the model, defined as the product of the physical abuse score and the indicator of whether the woman has been sexually abused.

```
lm2 <- lm(ptsd ~ cpa*csa,sexab)
summary(lm2)
##
## Call:
## lm(formula = ptsd ~ cpa * csa, data = sexab)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -8.1999 -2.5313 -0.1807  2.7744  6.9748
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)     3.6959     0.7107   5.201 1.79e-06 ***
## cpa             0.7640     0.3038   2.515   0.0142 *
## csaAbused       6.8612     1.0747   6.384 1.48e-08 ***
## cpa:csaAbused  -0.3140     0.3685  -0.852   0.3970
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.279 on 72 degrees of freedom
## Multiple R-squared:  0.5828, Adjusted R-squared:  0.5654
## F-statistic: 33.53 on 3 and 72 DF,  p-value: 1.133e-13
```

Note that the estimated interaction coefficient is negative. Its interpretation requires comparing the slope of the physical abuse score between the sexsually abused and not sexsually abused conditions.

For $\hat{\beta}_3 = -0.314$, the effect of physical abuse on the expected post-traumatic stress disorder score is reduced in the sexually abused group. In particular, the slope 0.764 for individuals who were not sexually abused, whereas it is 0.450 = 0.764-0.314 for individual whe were sexually abused.

According to the t-test, we reject the null hypothesis that the interaction term co-

efficient is equal to zero at all conventional significance levels. However, for model selection, it is generally preferable to rely on the AIC criterion.

```
step(lm2)
## Start:  AIC=184.41
## ptsd ~ cpa * csa
##
##           Df Sum of Sq    RSS    AIC
## - cpa:csa  1    7.8069 782.08 183.17
## <none>                 774.28 184.41
##
## Step:  AIC=183.17
## ptsd ~ cpa + csa
##
##        Df Sum of Sq     RSS    AIC
## <none>               782.08 183.17
## - cpa   1    110.30  892.38 191.20
## - csa   1    624.03 1406.11 225.76
##
## Call:
## lm(formula = ptsd ~ cpa + csa, data = sexab)
##
## Coefficients:
## (Intercept)          cpa    csaAbused
##      3.9753       0.5506       6.2728
```

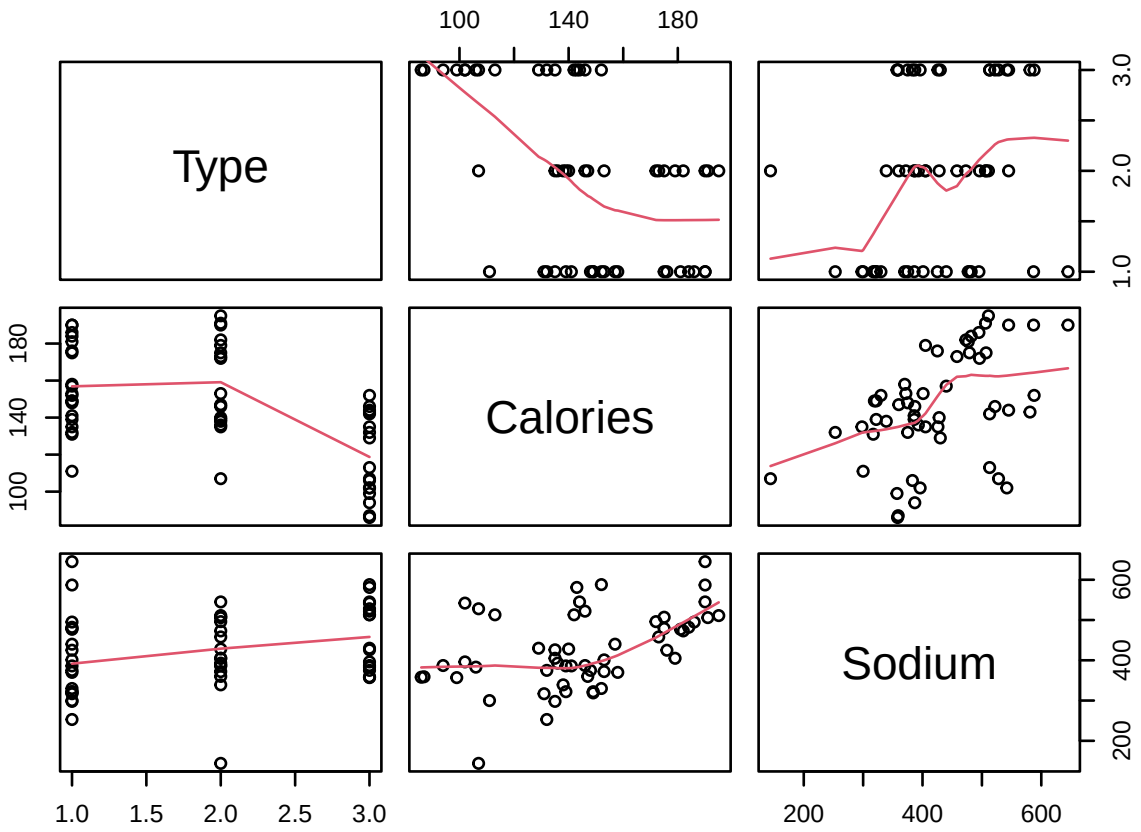
We observe that the additive model is also preferred based on the Akaike Information Criterion (AIC).

Categorical covariate with more than two levels

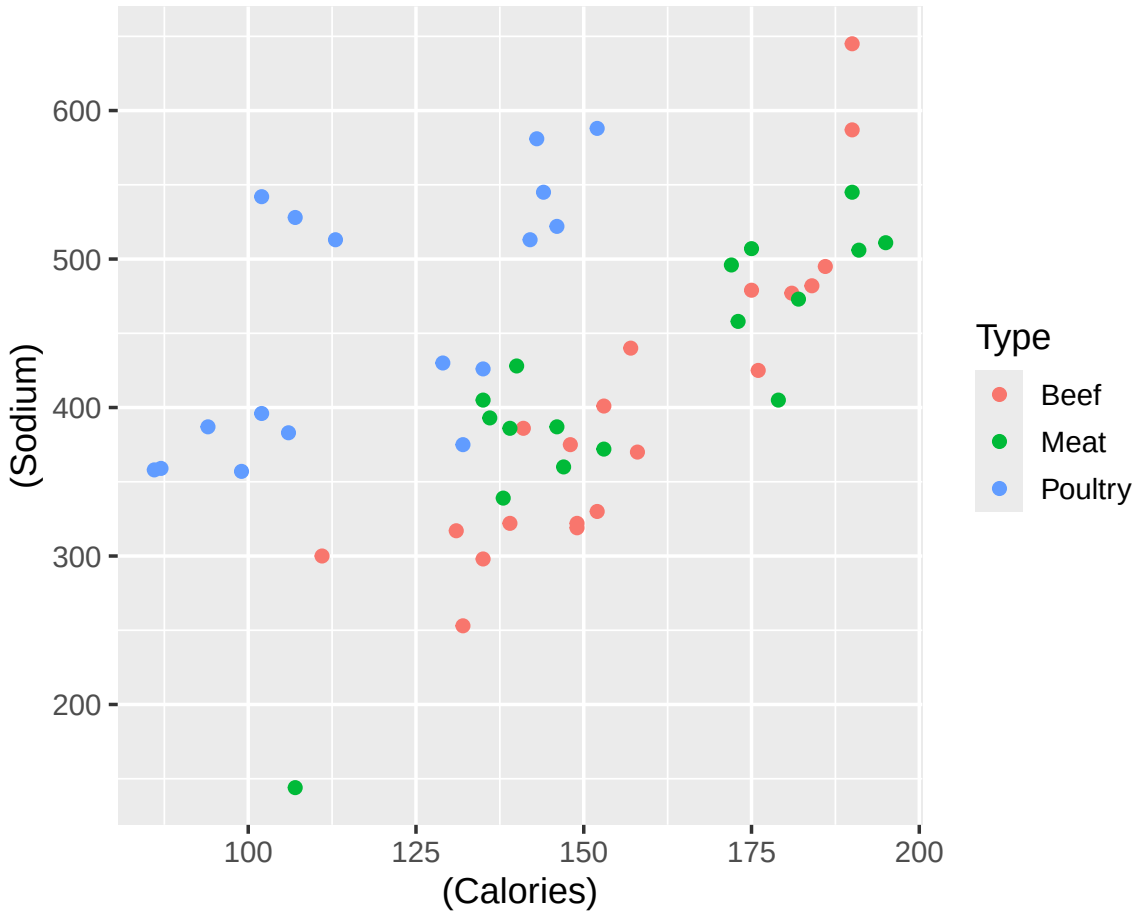
In the following example, we assume a linear association between the amount of sodium (continuous) contained in the meat and the number of calories, distinguishing among different types of meat: beef, other meats, and poultry.

```r
require("HH")
data(hotdog, package="HH")
dim(hotdog)
## [1] 54  3
head(hotdog)
##   Type Calories Sodium
## 1 Beef      186    495
## 2 Beef      181    477
## 3 Beef      176    425
## 4 Beef      149    322
## 5 Beef      184    482
## 6 Beef      190    587
str(hotdog)
## 'data.frame':    54 obs. of  3 variables:
##  $ Type    : Factor w/ 3 levels "Beef","Meat",..: 1 1 1 1 1 1 1 1 1 1 ...
##  $ Calories: int  186 181 176 149 184 190 158 139 175 148 ...
##  $ Sodium  : int  495 477 425 322 482 587 370 322 479 375 ...
```

```r
pairs(hotdog,
      panel = panel.smooth,
      main = "hotdog")
```




```
ggplot(data = hotdog, aes((Calories), (Sodium), color = Type)) +
geom_point()
```



Poultry shows, on average, a similar amount of sodium but lower calories compared to the other types of meat.

The full model includes interaction terms and is specified as follows:

```
lm1<-lm(Sodium ~ Calories*Type, data=hotdog)
step(lm1)
## Start:  AIC=437.58
## Sodium ~ Calories * Type
##
##                 Df Sum of Sq    RSS    AIC
## - Calories:Type  2     10402 153331 437.37
## <none>                       142930 437.58
##
## Step:  AIC=437.37
## Sodium ~ Calories + Type
##
```

```
##             Df Sum of Sq    RSS    AIC
## <none>                    153331 437.37
## - Type       2    227386 380718 482.48
## - Calories   1    301917 455249 494.14
##
## Call:
## lm(formula = Sodium ~ Calories + Type, data = hotdog)
##
## Coefficients:
## (Intercept)     Calories     TypeMeat  TypePoultry
##     -113.28         3.28        11.29       182.76
```

Note that the reference category is the first one. The AIC criterion again suggests selecting an additive model

```
lm2 <-lm(Sodium ~ Calories + Type, data=hotdog)
summary(lm2)
##
## Call:
## lm(formula = Sodium ~ Calories + Type, data = hotdog)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -127.409  -30.569   -2.556   33.427  137.985
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -113.2838    53.3043  -2.125   0.0385 *
## Calories       3.2798     0.3305   9.922 2.09e-13 ***
## TypeMeat      11.2925    18.2783   0.618   0.5395
## TypePoultry  182.7615    22.1856   8.238 7.15e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 55.38 on 50 degrees of freedom
## Multiple R-squared:  0.6851, Adjusted R-squared:  0.6663
## F-statistic: 36.27 on 3 and 50 DF,  p-value: 1.356e-12
```

- The residuals show a fairly wide range and should therefore be examined more carefully.
- For a one-unit increase in calories, sodium increases by about 3.28 units, holding the categorical covariate fixed.
- The fitted regression line for poultry lies below those for beef and other meat types. In fact, we can write separate equations for each type of meat.
- The straight line for the chicken meat is given by

$$\hat{Y}_P = -113.283 + 3.28 * calories + 182.7615$$

$$\hat{Y}_P = 69.478 + 3.28 * calories$$

- The equation for meat

$$\hat{Y}_M = 11.29 - 113.283 + 3.28 * calories$$

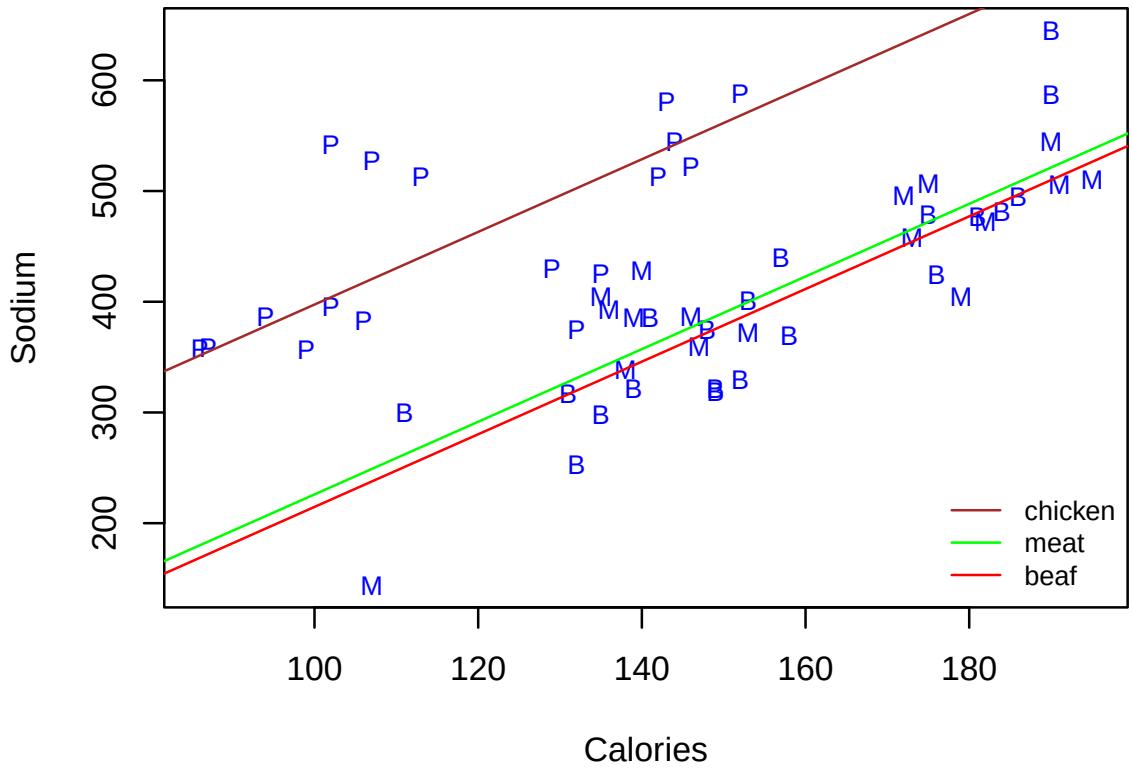
- The equation for beef is

$$\hat{Y}_B = -113.283 + 3.28 * calories$$

These parallel lines can also be compared by depicting them graphically

```
plot(Sodium ~ Calories,
     pch =as.character(Type),
     hotdog,
     col=c("blue"),
     cex=0.8)
abline(69.477, 3.28, col="brown")
abline(-101.993, 3.28, col="green")
abline(-113.283, 3.28, col="red")
legend("bottomright", 2,
       c("chicken", "meat", "beaf"),
```

```
       col = c("brown","green","red"),
       lty= 1,
        bty = "n",
       cex = 0.8)
```



Learning objectives

- Modelling binomial responses to assess how the probability of success, p, can be explained by covariates.
- Interpreting the estimated regression coefficients as changes in the odds, that is, in the ratio between the probability of success and the probability of failure.
- Answering questions such as: does the probability that a woman is employed depend on whether her husband's income is low or on the absence of children?
- Prediction: what is the probability of a given event for each unit, given a set of covariates?

Logistic regression model for a binary response

We consider the following case study based on data collected from the Canadian Women's Labour-Force Participation survey. The dataset `carData::Womenlf` comes from a 1977 survey of the Canadian population and was collected within as part of the *Social Change in Canada Project*.

The sample size is 263, and data refer to the following variables:

- *partic*: a factor indicating labor force participation, with levels no work, part-time work, and full-time work.
- *hincome*: husband's income, measured in $1000s of dollars.
- *children*: presence of children in the household, coded as 2 if children are present and 1 otherwise (`as.numeric(Womenlf$children)`),
- *region*: region of residence, coded from 1 to 5 for Atlantic, British Columbia, Ontario; Prairie, Quebec, respectively.

```
require(carData)
data(Womenlf)
names(Womenlf)
## [1] "partic"   "hincome"  "children" "region"
head(Womenlf)
##     partic hincome children  region
## 1 not.work      15  present Ontario
```

```
## 2 not.work      13  present Ontario
## 3 not.work      45  present Ontario
## 4 not.work      23  present Ontario
## 5 not.work      19  present Ontario
## 6 not.work       7  present Ontario
require(skimr)
my_skim<- skim_with(base=sfl(),numeric=sfl(hist=NULL))
my_skim(Womenlf)
```

Table 20: Data summary

Name	Womenlf
Number of rows	263
Number of columns	4
Column type frequency:	
factor	3
numeric	1
Group variables	None

Variable type: factor

skim_variable	ordered	n_unique	top_counts
partic	FALSE	3	not: 155, ful: 66, par: 42
children	FALSE	2	pre: 184, abs: 79
region	FALSE	5	Ont: 108, Que: 65, Pra: 31, Atl: 30

Variable type: numeric

skim_variable	mean	sd	p0	p25	p50	p75	p100
hincome	14.76	7.23	1	10	14	19	45

We analyze individual-level data with a binary response related to job participation in order to obtain maximum likelihood estimates of the logistic regression parameters. The aim is to examine the probability of female employment as a function of the

husband's income and the presence of children.

Relevel of the response

We aim to model the logit of the probability *of not working*. To this end, we combine the other two categories (part and full time work) into a single group, thereby obtaining a binary response variable.

```
table(Womenlf$partic)
##
## fulltime not.work parttime
##       66      155       42
prop.table(table(Womenlf$partic))
##
##  fulltime  not.work  parttime
## 0.2509506 0.5893536 0.1596958
Y <- as.factor(0 + (Womenlf$partic=='not.work'))
table(Y)
## Y
##   0   1
## 108 155
round(prop.table(table(Y)),3)
## Y
##     0     1
## 0.411 0.589
```

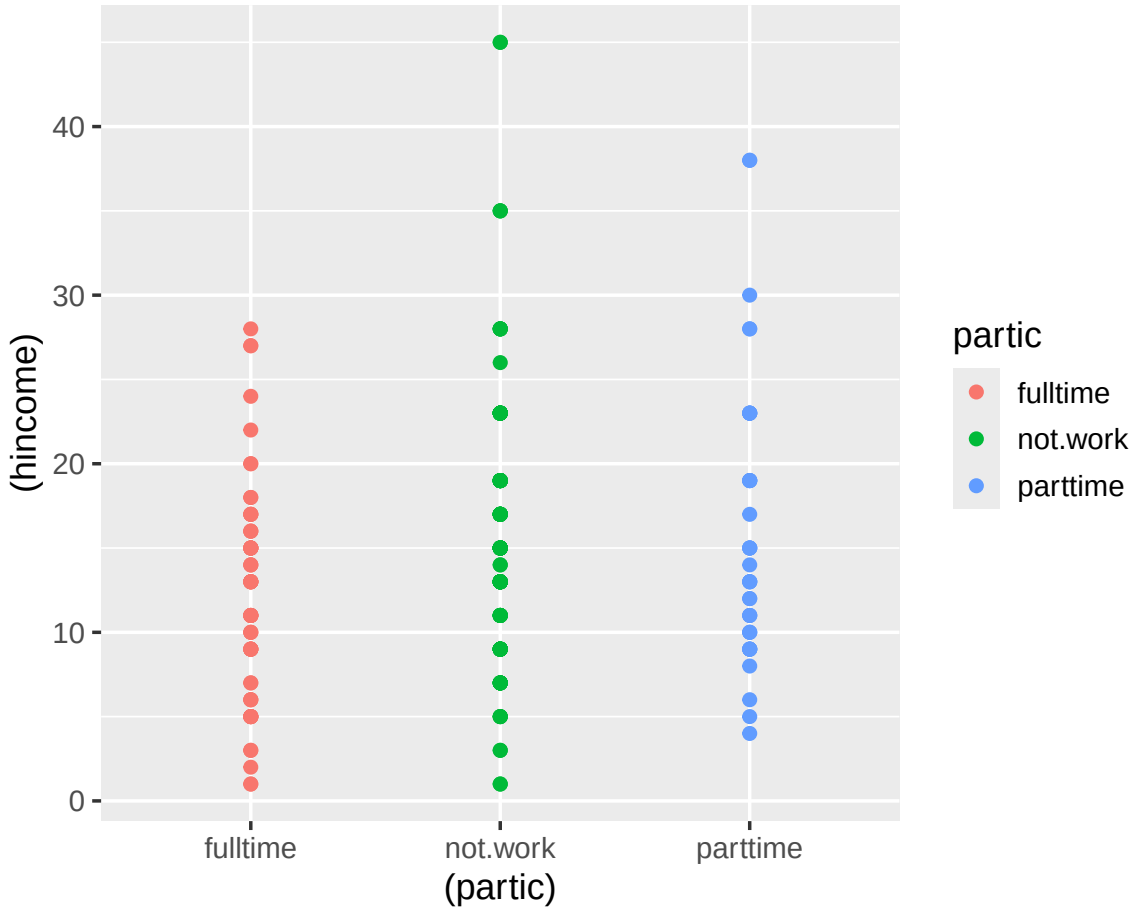
We notice that 59% of women do not work. We add the new vector to the data frame.

```
Womenlf$Y <- Y
rm(Y)
```

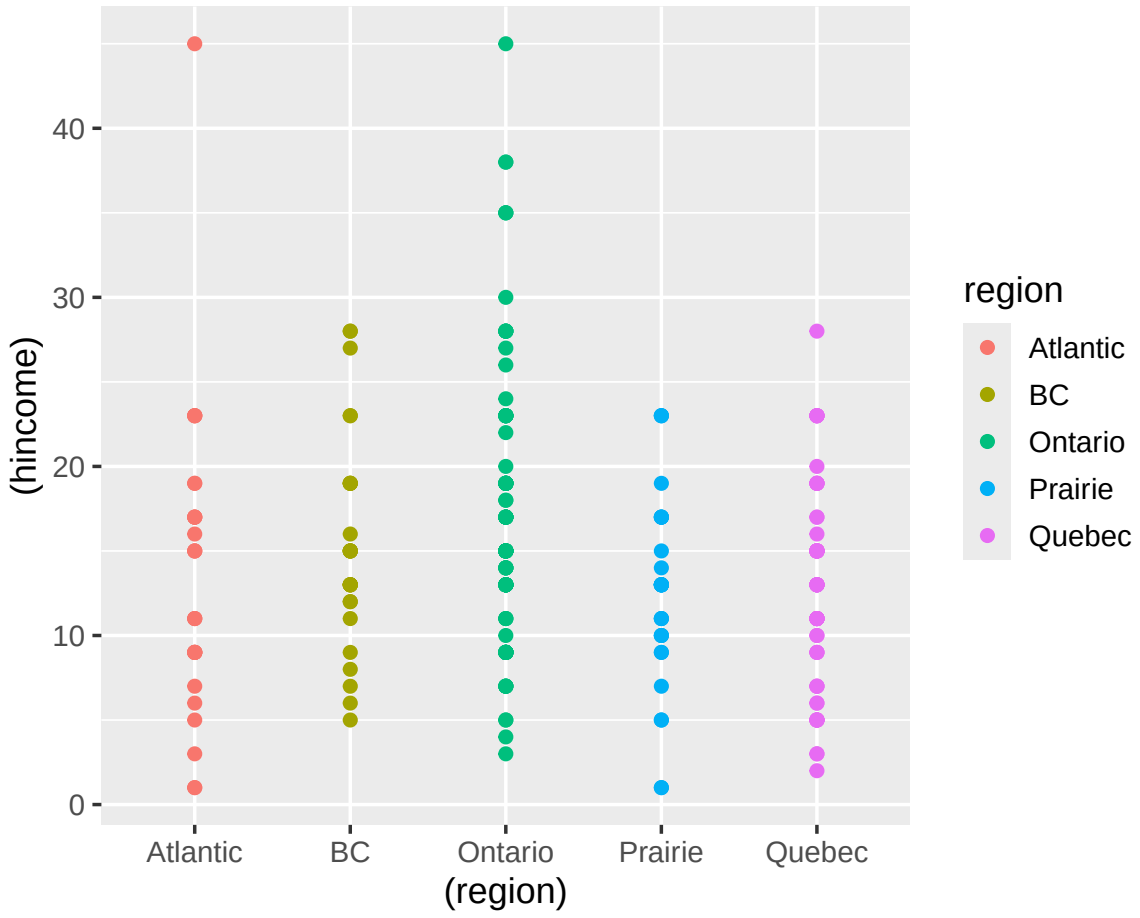
Explanatory analyses

We present the data through some plots:

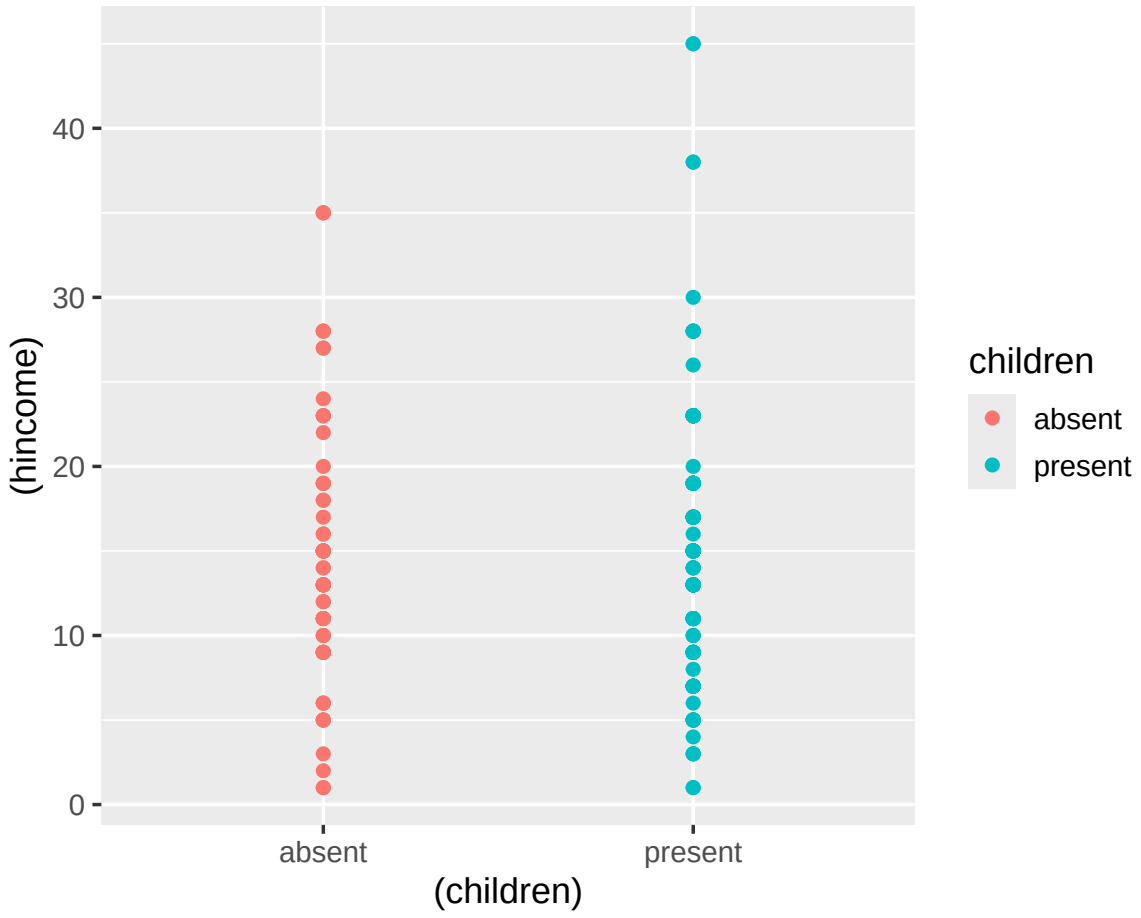
```
require(ggplot2)
ggplot(data = Womenlf, aes((partic), (hincome),
                           color = partic)) + geom_point()
```




```
require(ggplot2)
ggplot(data = Womenlf, aes((region), (hincome),
                           color = region)) + geom_point()
```




```
require(ggplot2)
ggplot(data = Womenlf, aes((children), (hincome),
                           color = children)) + geom_point()
```



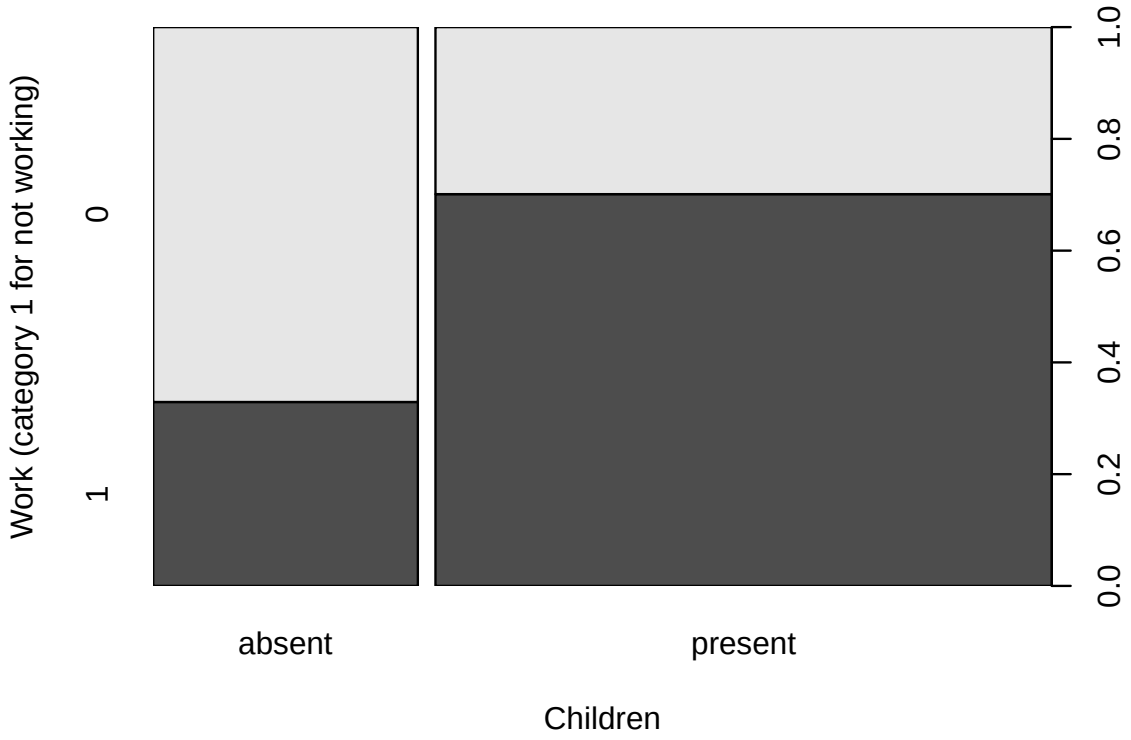
No substantial differences in employment status are observed with respect to the husband's income.

The following plot shows the proportion of employed women according to whether children are present or absent.

We now check the levels and coding of the categorical variable.

```
head(Womenlf$children)
## [1] present present present present present present
## Levels: absent present
as.integer(head(Womenlf$children))
## [1] 2 2 2 2 2 2
```

```
plot(Womenlf$children, Womenlf$Y,
     main = " ",
     xlab = "Children",
     ylab = "Work (category 1 for not working)")
```



Note that in the plot the conditional relative frequencies considering the row totals (children) are depicted that is

```
tab<- table(Womenlf$children, Womenlf$Y);
addmargins(table(Womenlf$children, Womenlf$Y))
##
##              0   1 Sum
##   absent    53  26  79
##   present   55 129 184
##   Sum      108 155 263
round(prop.table(tab, margin = 1),3)
##
##                0     1
##   absent   0.671 0.329
##   present 0.299 0.701
```

The previous figure relates to the response variable to the presence or absence of children in the household. It shows that the proportion of unemployed women (1) is much higher when children are present (0.70) than when they are absent (0.33).

The `prop.table` function takes the output of the `table` function as input and returns relative frequencies.

If the option `margin = 2` is specified, the proportions are computed relative to the columns of totals.

```
tab<- table(Womenlf$children, Womenlf$Y);
tab
##
##              0   1
##   absent    53  26
##   present   55 129
round(prop.table(tab, margin = 1),3)
##
##                0     1
##   absent   0.671 0.329
##   present  0.299 0.701
```

The odds of unemployment (i.e., the odds in favor of not working) for women with children are equal to the ratio between the probability of not working and the probability of working

```
129/55
## [1] 2.345455
0.7011/0.2989
## [1] 2.345601
```

while the odds of unemployment for women without children is

```
26/56
## [1] 0.4642857
 0.3291/0.6709
## [1] 0.4905351
```

We observe that the odds are higher in presence of children.

We can compare these odds by computing the *odds ratio*

```
2.3454/0.4643
## [1] 5.051475
```

this value is greater than 1 and indicates that the odds of not working are approximately 5 times higher for women with children than for those without children.

Estimation of the model with glm function

For the estimation of the logistic model, the `glm` (generalised linear models) function is used. It requires to specify the distribution and the link function by setting `family = binomial`

```
mod1<-glm(Y ~ hincome + children, data = Womenlf,
                  family = binomial)
summary(mod1)
##
## Call:
## glm(formula = Y ~ hincome + children, family = binomial, data = Womenlf)
##
## Coefficients:
##                  Estimate Std. Error z value Pr(>|z|)
## (Intercept)     -1.33583    0.38376  -3.481   0.0005 ***
## hincome          0.04231    0.01978   2.139   0.0324 *
## childrenpresent  1.57565    0.29226   5.391    7e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 356.15  on 262  degrees of freedom
## Residual deviance: 319.73  on 260  degrees of freedom
## AIC: 325.73
##
## Number of Fisher Scoring iterations: 4
mod1$aic
## [1] 325.7325
```

We have

$$logit(\hat{p}_i) = -1.3358 + 0.0423(inc) + 1.5756(chil)$$

where $\hat{p}_i$ is the probability of not working.

Note that the logistic model must be interpreted as a *multiplicative* model for the odds, $p/(1-p)$.

- Therefore, $\exp(\hat{\beta}_1) = \exp(0.0423) = 1.0432$ represents the ratio of the odds for group A (not working) versus group B (working) associated with one-unit increase in the husband's income, holding the presence or absence of children constant.
- It can also be interpreted as follows: for each additional 1000 dollars of the husband's income, the odds of female unemployment increase by 4.32 per cent since 1.0432-1 = 0.0432*100 = 4.32, ceteris paribus with respect to the presence or absence of children.
- The estimated regression coefficient associated with the presence of children shows that $\exp(\hat{\beta}_2) = \exp(1.5756) = 4.8336$. Thus, at fixed level of the husband's income, the odds of not working for mothers are about 4.8 times those for women without children.
- Husband's income is also a risk factor for unemployment, although its effect is much smaller than that of the presence of children.
- Residual do not show excessive variability.
- The **residual deviance** of the estimated model is equal to 319.73. In general, models with smaller residual deviance are preferred.
- The **null deviance** is the deviance of the model without covariates, that is, a model including only the intercept.
- The Wald test for each regression coefficient yields a highly significant p-value for the coefficient associated with the precence of children. The observed value of the test statistic is:
$$Z = \frac{1.5756}{0.2922} = 5.391$$
this indicates that the null hypothesis $H_0 : \beta_2 = 0$ can be rejected at any conventional significant level.
- The Wald test for the regression coefficient associated with the husband's income also leads us to reject the null hypothesis at the significant level of 95%. There is moderate evidence that support unemployment increase with higher husband incomes after adjusting for the presence of children.

The deviance of the null model is greater than the residual deviance, indicating that the covariates contribute to explaining the logit under consideration.

The Akaike Information Criterion (AIC) of the full model is 325.7325.

Fitted values

We can estimate of the probability of not working, $expit(\widehat{\eta}_i)$, for a given value of the husband's income (10000 dollars) and in the presence of children, since, as stated in the teaching notes, the logit model can be also expressed written in terms of **probability**

$$\hat{p}_2 = \frac{\exp(\hat{\beta}_0 + \hat{\beta}_1 * income + \hat{\beta}_2 * chil)}{1 + \exp(\hat{\beta}_0 + \hat{\beta}_1 * income + \hat{\beta}_2 * chil)}$$

```
round(exp(-1.3358 + 0.0423*(10) + 1.5756*2)/
        (1+exp(-1.3358 + 0.0423*(10) + 1.5756*2)),3)
## [1] 0.904
```

The same probability of not working in case of absence of children is

```
round(exp(-1.3358 + 0.0423*(10) + 1.5756*1)/(1+exp(-1.3358 +
                                                  0.0423*(10) + 1.5756*1)),3)
## [1] 0.66
```

Fitted values refer to estimated probabilities for each unit in the sample

```
prob <- mod1$fitted.values
round(head(prob),3)
##     1     2     3     4     5     6
## 0.706 0.688 0.895 0.771 0.740 0.631
Womenlf[1,]
##     partic hincome children  region Y
## 1 not.work      15  present Ontario 1
```

For the first sample unit, for example, the estimated probability of not working based on the full model is 0.71. This woman is unemployed, has children, and has a household income of $15,000 dollars.

We can summarize the estimated probabilities and identify the unit with the highest estimated probability.

```
summary(prob)
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.2153  0.4052  0.6693  0.5894  0.7229  0.8951
which(prob == max(prob))
##  3 15
##  3 15
Womenlf[3,]
##     partic hincome children  region Y
## 3 not.work      45  present Ontario 1
Womenlf[15,]
##      partic hincome children   region Y
## 15 not.work      45  present Atlantic 1
```

The estimated probabilities range from 0.21 to 0.9. Unit 15 has the highest probability, equal to 0.9: this is a woman who does not work, has a high household income, and lives in the Atlantic region.

Note, however, that the place of residence has been omitted from the model for the time being.

Model selection

Information criteria, such as the Akaike Information Criterion (AIC), can also be used with the `step` function, as illustrated previously for model selection

```
step(mod1)
## Start:  AIC=325.73
## Y ~ hincome + children
##
##             Df Deviance    AIC
## <none>           319.73 325.73
## - hincome    1   324.56 328.56
## - children   1   351.06 355.06
##
```

```
## Call:  glm(formula = Y ~ hincome + children, family = binomial, data = Women
##
## Coefficients:
##     (Intercept)          hincome  childrenpresent
##        -1.33583          0.04231          1.57565
##
## Degrees of Freedom: 262 Total (i.e. Null);  260 Residual
## Null Deviance:       356.2
## Residual Deviance: 319.7     AIC: 325.7
```

The first estimated model is the full model, the second excludes income, and the third excludes the presence of children. Since the residual deviance is higher for the last two models, the full model provides the best fit.

The 95% confidence intervals are as follows:

```
round(confint(mod1),5)
##                    2.5 %   97.5 %
## (Intercept)     -2.11534 -0.60591
## hincome          0.00447  0.08226
## childrenpresent  1.01272  2.16144
```

Note that we can also obtain the confidence interval for the estimated odds ratio as follow

```
round(exp(confint(mod1)),5)
##                    2.5 %  97.5 %
## (Intercept)     0.12059 0.54558
## hincome         1.00448 1.08574
## childrenpresent 2.75308 8.68362
```

We note that none of the intervals for the odds ratios include 1

```
round(exp(c(0.00447, 0.08226)),3)
## [1] 1.004 1.086
round(exp(c(1.01272, 2.16144)),3)
## [1] 2.753 8.684
```

To summarize:

- An increase in the husband's income raises the odds of female non-employment by between 0.4% to 8%, holding the number of children fixed.
- For each additional child in the family, the odds are multiplied by a factor ranging from $exp(1.012) = 2.75$ to $exp(2.161) = 8.68$. In other words, the odds in favour of non working are between about 3 and 9 times those for women without children, at the same level of husband's income, with 95% confidence.

Adding the place of residence

We now assess whether the variable *region* can be included in the model. First, we examine its labels and coding

```
table(Womenlf$region)
##
## Atlantic       BC  Ontario  Prairie   Quebec
##       30       29      108       31       65
table(as.integer(Womenlf$region))
##
##   1   2   3   4   5
##  30  29 108  31  65
```

The baseline category is 1, corresponding to the Atlantic region.

We then estimate the model including all covariates and perform variable selection

```
mod3<-glm(Y ~ hincome + children + region,
          data = Womenlf,
          family = binomial)
```

```
step(mod3)
## Start:  AIC=331.3
## Y ~ hincome + children + region
##
##            Df Deviance    AIC
## - region    4   319.73 325.73
## <none>          317.30 331.30
## - hincome   1   322.43 334.43
## - children  1   347.85 359.85
##
## Step:  AIC=325.73
## Y ~ hincome + children
##
##            Df Deviance    AIC
## <none>          319.73 325.73
## - hincome   1   324.56 328.56
## - children  1   351.06 355.06
##
## Call:  glm(formula = Y ~ hincome + children, family = binomial, data = Women
##
## Coefficients:
##     (Intercept)          hincome  childrenpresent
##        -1.33583          0.04231          1.57565
##
## Degrees of Freedom: 262 Total (i.e. Null);  260 Residual
## Null Deviance:       356.2
## Residual Deviance: 319.7     AIC: 325.7
```

The AIC is lower when we the *region* variable is excluded from the model. Therefore, according to this criterion, the performed model is the one estimated above without the region of residence.

However we can assess the significance of the covariates by examining the Wald tests using the `summary` function

```
summary(mod3)
##
## Call:
## glm(formula = Y ~ hincome + children + region, family = binomial,
##     data = Womenlf)
##
## Coefficients:
##                 Estimate Std. Error z value Pr(>|z|)
## (Intercept)     -1.26771    0.55296  -2.293   0.0219 *
## hincome          0.04534    0.02057   2.204   0.0275 *
## childrenpresent  1.60434    0.30187   5.315 1.07e-07 ***
## regionBC        -0.34196    0.58504  -0.585   0.5589
## regionOntario   -0.18778    0.46762  -0.402   0.6880
## regionPrairie   -0.47186    0.55680  -0.847   0.3967
## regionQuebec     0.17310    0.49957   0.347   0.7290
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 356.15  on 262  degrees of freedom
## Residual deviance: 317.30  on 256  degrees of freedom
## AIC: 331.3
##
## Number of Fisher Scoring iterations: 4
```

None of the coefficients associated with the *region* variable are statistically significant, suggesting that the associations between female employment, husband's income, and the presence of children are consistent across all regions of the country.

```
tab1<- table(Womenlf$region, Womenlf$Y);
tab1
##
##              0  1
##   Atlantic 10 20
##   BC       15 14
```

```
##   Ontario  44 64
##   Prairie  14 17
##   Quebec   25 40
prop.table(tab1, margin = 1)
##
##                    0         1
##   Atlantic 0.3333333 0.6666667
##   BC       0.5172414 0.4827586
##   Ontario  0.4074074 0.5925926
##   Prairie  0.4516129 0.5483871
##   Quebec   0.3846154 0.6153846
```

There no substantial differences in the population of employed women across Canada regions.

Prediction: classification table and ROC curve

The model can be used to predict employment status of each women in the sample. We can construct a classification table by cross-classifying the observed and predicted outcomes.

We adopt a cutoff rule whereby $p_i < 0.5$ corresponds to employment and $p_i \geq 0.5$ corresponds to unemployment. First we add the predicted probabilities from the selected model to the dataframe

```
prob <- mod1$fitted.values
Womenlf$prob <- prob
```

We use the chosen cutoff to create the binary variable related to predicted probabilities, we create a new dataframe named `dt`

```
library(dplyr)
dt <- mutate(Womenlf, predout = ifelse(prob < 0.5, "WORK", "NOWORK"))
head(dt)
##     partic hincome children  region Y      prob predout
## 1 not.work      15  present Ontario 1 0.7056698  NOWORK
## 2 not.work      13  present Ontario 1 0.6877944  NOWORK
## 3 not.work      45  present Ontario 1 0.8950784  NOWORK
## 4 not.work      23  present Ontario 1 0.7708139  NOWORK
```

```
## 5 not.work      19  present Ontario 1 0.7395592  NOWORK
## 6 not.work       7  present Ontario 1 0.6308762  NOWORK
```

we print the classification table

```
xtabs(~  predout + Y , dt)
##          Y
## predout    0   1
##   NOWORK  55 132
##   WORK    53  23
```

The correct classification rate is

```
n <- dim(dt)[1]; n
## [1] 263
(53+132)/n
## [1] 0.7034221
```

The classification accuracy is 0.7, while the misclassification rate is approximately 30%.

- **Specificity** of the test, $P(\hat{Y} = 0|y = 0)$: *For a women who is actually employed (55+53), what fraction is correctly predicted by the model to be employed?*

```
53/(55+53)
## [1] 0.4907407
```

- **Sensitivity** of the test, $P(\hat{Y} = 1|y = 1)$: *the proportion of unemployed women who are correctly identified by the model*, that is, the **true positive rate**

```
132/(132+23)
## [1] 0.8516129
```

The model is therefore **quite effective at predicing** which women will be unemployed.

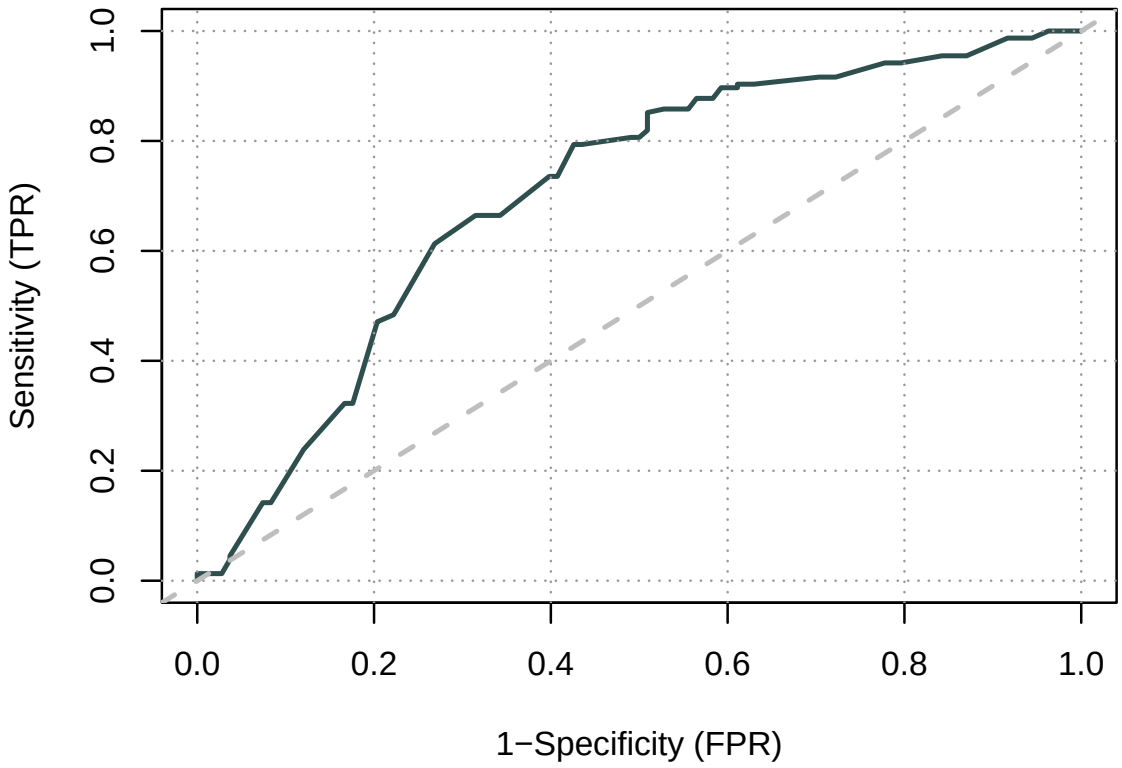
The above calculations are based on a specific threshold value. Therefore, they can be repeated for all possible threshold values. To do this, we use the following code and the `ROCit` package, which allows us to plot the **receiver operating characteristic** (ROC) curve. This curve displays sensitivity, or recall (the true positive rate, TPR),

on the vertical axis against one minus the specificity (the false positive rate, FPR) on the horizontal axis. The ROC curve illustrates the ability of a model to discriminate between classes by plotting the TPR against the FPR for different threshold values.

By varying the decision threshold—for example, classifying an observation as 1 when the predicted probability from the model exceeds 0.6—we obtain different pairs of false positive rate (FPR) and true positive rate (TPR). These pairs form the points on the ROC curve.

```
prob <- predict(mod1, type = "response")
dt$prob <- prob
head(dt)
##     partic hincome children  region Y      prob predout
## 1 not.work      15  present Ontario 1 0.7056698  NOWORK
## 2 not.work      13  present Ontario 1 0.6877944  NOWORK
## 3 not.work      45  present Ontario 1 0.8950784  NOWORK
## 4 not.work      23  present Ontario 1 0.7708139  NOWORK
## 5 not.work      19  present Ontario 1 0.7395592  NOWORK
## 6 not.work       7  present Ontario 1 0.6308762  NOWORK
```

```
require(ROCit)
roc1 <- rocit(score = dt$prob, class =dt$Y )
summary(roc1)
##
##  Method used: empirical
##  Number of positive(s): 155
##  Number of negative(s): 108
##  Area under curve: 0.7055
plot(roc1, legend = FALSE, YIndex = FALSE,
     AUC = TRUE)
```



Better predictive performance corresponds to higher sensitivity and, more generally, to an ROC curve that lies closer to the upper-left corner. Note that the grey dotted diagonal line corresponds to the *null model*, a model with intercept only.

The are under the ROC curve (AUC) is used as a measure of the performance of the test: the larger the area, the better the predictive ability. In this case, the AUC is 0.71, indicating that the model performs reasonably well even with only two covariates.

In general:

- a curve closer to the upper-left corner indicates better performance;
- the grey dashed diagonal line (from (0,0) to (1,1)) represents a model with no discrimination ability, that is a model equivalent to random guessing;
- The Area Under the Curve (AUC) summarizes the ROC curve by means of a single number:
  - AUC = 1: Perfect classifier.
  - AUC = 0.5: No better than random.
  - AUC $<$ 0.5: Worse than random (model is reversing predictions).

Generally, leave-one-out cross-validation is used to obtain less biased estimates of misclassification probabilities.

−> −>

−>

Learning objectives

- Estimation of finite mixture models of Gaussian distributions using the package `Mclust`.
- Apply model-based cluster analysis to identify heterogenous subgroups of units in the reference population.
- Apply supervised and unsupervised learning techniques

Estimation of a Gaussian mixture model for cholesterol data

`datacol.Rdata` contains measurements related to a sample of individuals selected for a study related to the effects of cholesterol on hearth diseases.

In this case we consider univariate data represented by the measurement of the cholesterol (in mg on dl of plasma). Values are referred to both men and women and the variable gender (sex) is also reported.

```
#load('datacol.Rdata')
#datacol1 <- datacol[,-3]
#save(datacol1, file= "datacol1.Rdata")
```

```
load('datacol1.Rdata')
dim(datacol1)
## [1] 200   2
head(datacol1)
##     ID cholst
## 1 1244    175
## 2  835    299
## 3  176    250
## 4  901    243
## 5 1972    150
## 6 1994    234
```

```
require(skimr)
skim_without_charts(datacol1)
```

Table 23: Data summary

Name	datacol1
Number of rows	200
Number of columns	2
Column type frequency:	
numeric	2
Group variables	None

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100
ID	0	1	1344.43	774.26	1	682.5	1367.5	2012.25	2616
cholst	0	1	220.49	43.92	133	191.0	214.0	249.00	357

The mean cholesterol level observed in the sample is 220.5, with a relatively high variability around the mean (standard deviation of 44). The mean is greater than the median, indicating positive skewness: a few high values pull the mean upward, while the empirical distribution is concentrated at lower cholesterol levels.

The library called `mclust` (Gaussian Mixture Modelling for Model-Based Clustering) allows the estimation of mixture model parameters with Gaussian components.

```
require('mclust')
```

Function Mclust

The main function of the `mclust::Mclust` library requires as arguments:

- the dataset
- the number of components of the model (`G`),

- the `modelNames = "E"` option to specify whether model components are assumed to have the same variance, *homoscedastic* model, or `modelNames = "V"` if assumed to have component-specific variance, *heteroscedastic* model

The following code estimate the mixture distribution for the reference population and perform cluster analysis assuming a variance-covariance matrix which is equal across components (model label "E") with two mixture components (label G = 2):

```
mod1 <- Mclust(datacol1$cholst,
       G = 2,
       modelNames = "E")
```

Tollerance level for the convergence criterion of the EM algorithm are fixed internally and can be changed.

A summary of the model results is printed

```
summary(mod1)
## ----------------------------------------------------
## Gaussian finite mixture model fitted by EM algorithm
## ----------------------------------------------------
##
## Mclust E (univariate, equal variance) model with 2 components:
##
##  log-likelihood   n df       BIC       ICL
##       -1032.055 200  4 -2085.302 -2126.187
##
## Clustering table:
##   1   2
## 158  42
```

The `summary` returns:

- value of the **log-likelihood function** at convergence after the parameters have been estimated with the EM algorithm

- **number of parameters** of the model free to vary: in the two-component model they are given by the two averages, one variance (for model E) and two weights of each component but one is found as a difference from 1 due to the constraint to sum to 1.

If we add an additional argument `parameters = TRUE` the estimated parameters are also printed

```
summary(mod1,parameters = TRUE)
## ----------------------------------------------------
## Gaussian finite mixture model fitted by EM algorithm
## ----------------------------------------------------
##
## Mclust E (univariate, equal variance) model with 2 components:
##
##  log-likelihood   n df       BIC       ICL
##       -1032.055 200  4 -2085.302 -2126.187
##
## Clustering table:
##   1   2
## 158  42
##
## Mixing probabilities:
##         1         2
## 0.7777503 0.2222497
##
## Means:
##        1        2
## 203.7208 279.1730
##
## Variances:
##        1        2
## 935.6102 935.6102
```

where:

- **Mixing prob.** are the estimated weights of each component $\hat{\pi}$. We see that $\hat{\pi}_1 = 0.78$ is the weight of the first component and $\hat{\pi}_2 = 0.22$ of the second one.

- **means** of each cluster: the first cluster is represented by a random variable with a Normal distribution having mean $\hat{\mu}_1$=204, and the second cluster by another Normal random variable with mean $\hat{\mu}_2 = 279$. The first cluster is related to individuals with normal levels of cholesterol, while the second cluster with those suffering from overt cholesterol.
- **Variances**: the estimates of the variances of each components: $\hat{\sigma}^2 = 936$ since by assumption under the model E we assume identical variance of the two components. The average variability around each mean of the two clusters is 30 mg per dl of plasma.

The population of patients is characterised by a mixture of two subpopulations of patients: those having values of cholesterol which are within the normal expected range for healthy patients. Such a subpopulation is represented by a random variable $X_1 \sim N(204, 936)$. Through the model we identify a second subpopulation of patients: those at risk with potentially high and dangerous cholesterol levels. Such a subpopulation is represented by a random variable $X_2 \sim N(279, 936)$.

Classification

- **clustering table** returns the total number of patients classified in the first or second component of the mixture.

42 patients out of 152 are classified in the first group and all the others in the second.

Note that this assignment is made with respect to the **posterior probabilities** found in the `mod1$z` object

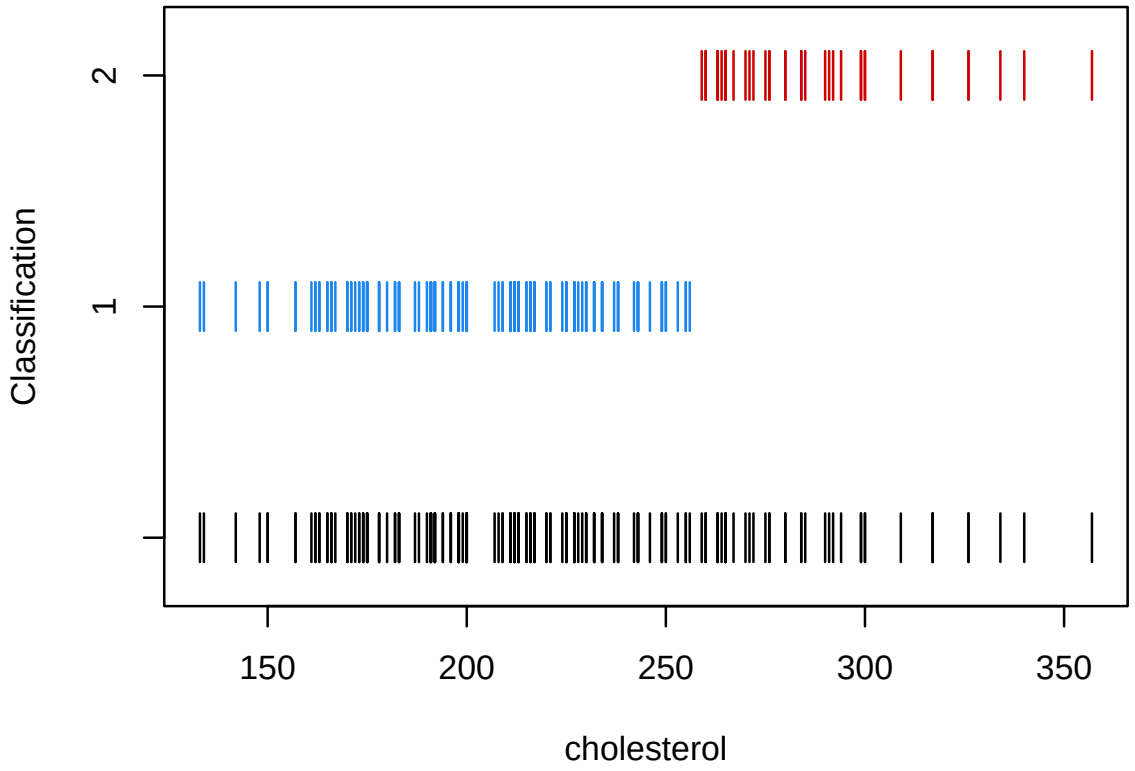
```
head(round(mod1$z, 2))
##      [,1] [,2]
## [1,] 1.00 0.00
## [2,] 0.03 0.97
## [3,] 0.63 0.37
## [4,] 0.75 0.25
## [5,] 1.00 0.00
## [6,] 0.86 0.14
```

According to this model with two components the first unit is almost surely (with high probability) assigned to the first cluster, while the second patient is assigned to the second cluster with equal high probability. Note that the third patient instead is assigned of the first cluster but with more uncertainty: the posterior probability for this unit is 0.63 for cluster 1 and 0.37 for cluster 2.

The association with gender and the two clusters estimated with the model can be visualized as

The following graph, shows all the observed increasingly ordered values of cholesterol in cl **black**, and the clustering in group 2 (**red**) and group 1 (**blue**).

```
plot(mod1,
     what='classification', xlab = "cholesterol")
```



The graph shows that the cholesterol value discriminating between the two subpopulations is slightly above 250.

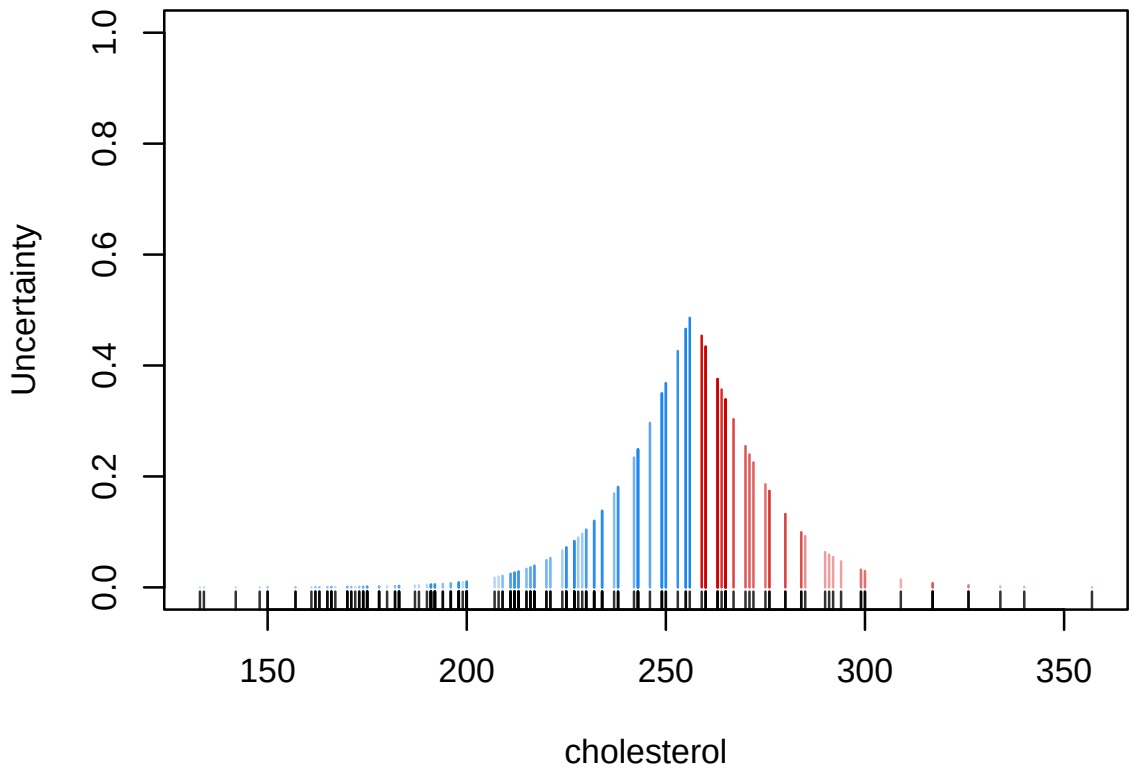
A measure of uncertainty associated with the MAP classification for each unit is computed as

$$u_i = 1 - max_k \hat{z}_{ik}$$

which falls within the interval [0,1]. Values close to zero indicate a low level of uncertainty in the classification of the corresponding observations.

This can also be represented graphically as follows:

```
plot(mod1,
     what='uncertainty', xlab = "cholesterol")
```



We observe that the maximum uncertainty is around 0.6. The filled plot shows the uncertainty for each individual; in particular, the units classified with higher uncertainty are those located near the point corresponding to the change point between cluster 1 and 2.

Model selection

Selection of the number of mixture components as well as of the variance-covariance structure is performed with the `mclus:mclustBIC` function.

Bayesian Information Criterion (BIC) is reported for mixture models with an increasing number of components and for two types of variances: equal or different.

The function accepts as input the data as a vector

```
bayesinf <- mclustBIC(datacol1$cholst)
bayesinf
## Bayesian Information Criterion (BIC):
##           E         V
## 1 -2090.155 -2090.155
## 2 -2085.302 -2089.269
## 3 -2096.304 -2103.554
## 4 -2104.550 -2118.857
## 5 -2114.649 -2124.799
## 6 -2123.391 -2149.002
## 7 -2134.099 -2134.237
## 8 -2144.769 -2142.100
## 9 -2155.475 -2132.422
##
## Top 3 models based on the BIC criterion:
##       E,2       V,2       E,1
## -2085.302 -2089.269 -2090.155
```

The table lists the realised BIC values for each mixture model with a number of components ranging from 1 to 9 (rows). The first row refers to the model without heterogeneity, i.e. without components assuming a homogeneous population.

The first column refers to models in which equal variability is assumed for the components of the mixture and the second to models in which specific variability is assumed for each component.

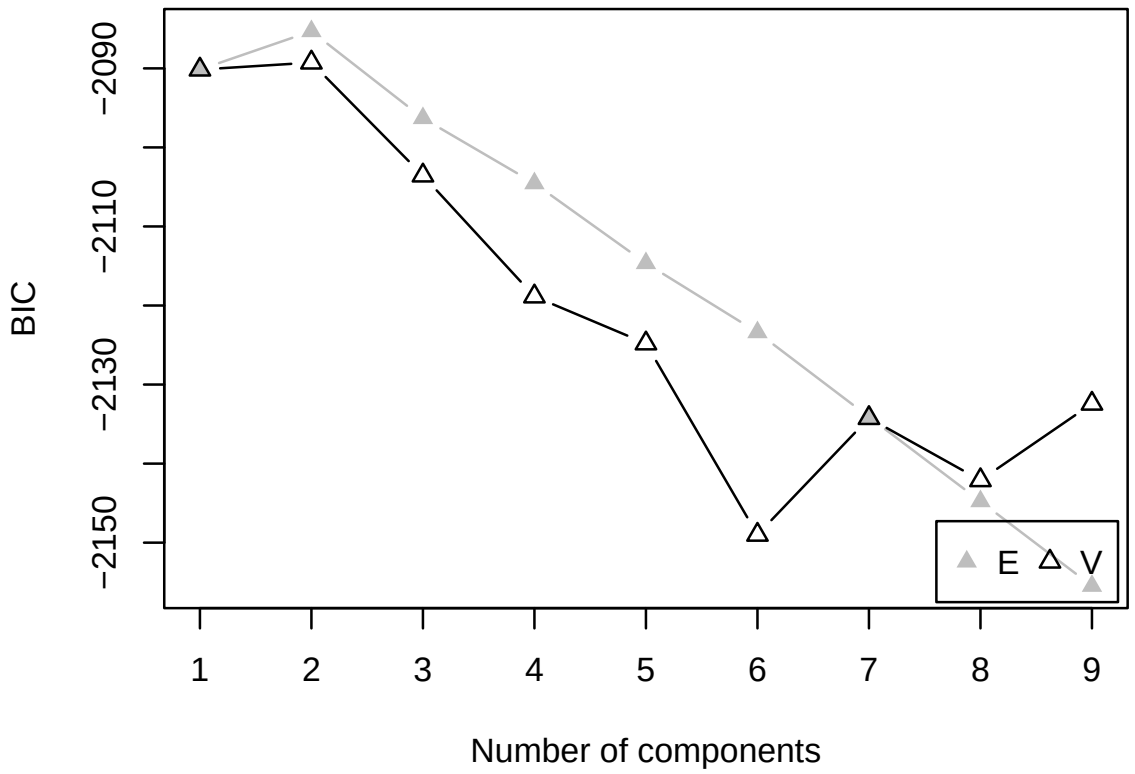
The best model is the one with the lowest BIC value, i.e. with the highest log-likelihood relative to the number of parameters and the number of observations.

The three best models with respect to this criterion are displayed at the bottom as **top 3 models**.

The most parsimonious model that accounts for the variability in the data is the one with 2 components and the same variance, which is precisely the model estimated above.

Values in the table can also be represented graphically as follows

```
plot(bayesinf)
```



The selected model as the best is (E,2): a two component Gaussian mixture with equal variance.

Note that the BIC and AIC are reported to have a negative sign. Therefore, the best model has values of such indexes closer to 0.

Bootstrap standard errors and confidence interval

The confidence intervals for the model parameters are obtained with the bootstrap by applying the percentile method.

In the following we illustrate the application of the bootstrap for the best model that assumes equal variance of the two components (`E`).

The function `mclust::MclustBootstrap` allows to perform nonparametric bootstrap, resampling with replacement from the original data, and the mixture model is re-estimated for each bootstrap sample.

By default the function uses $B = 999$ bootstrap samples, however this number can be changed with the option `nboot`. It requires to input an object as returned by the function estimating the model. The seed can be fixed for reproducibility

```r
set.seed(134)
bootClust <- MclustBootstrap(mod1)
```

The summary function returns the results by reporting the estimated standard errors for each parameter

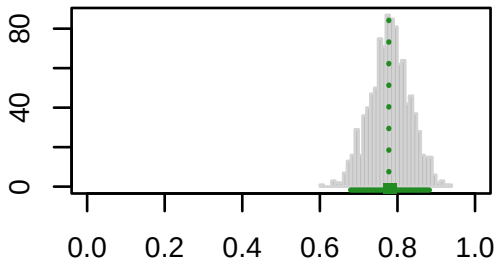
```r
summary(bootClust, what = "se")
## ---------------------------------------------------------
## Resampling standard errors
## ---------------------------------------------------------
## Model                        = E
## Num. of mixture components = 2
## Replications                 = 999
## Type                         = nonparametric bootstrap
##
## Mixing probabilities:
##          1          2
## 0.05117175 0.05117175
##
## Means:
##        1        2
## 3.694618 8.375127
##
## Variances:
##        1        2
## 122.0477 122.0477
```

and the percentile confidence intervals calculated at the 95% confidence level are also available by specifying `what = "ci"`

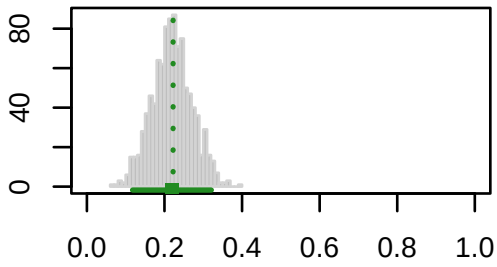
```
summary(bootClust, what = "ci")
## ------------------------------------------------------------
## Resampling confidence intervals
## ------------------------------------------------------------
## Model                        = E
## Num. of mixture components = 2
## Replications                 = 999
## Type                         = nonparametric bootstrap
## Confidence level             = 0.95
##
## Mixing probabilities:
##                1         2
## 2.5%  0.6791669 0.1176816
## 97.5% 0.8823184 0.3208331
##
## Means:
##               1        2
## 2.5%  196.4379 264.8307
## 97.5% 211.4407 298.6920
##
## Variances:
##                1         2
## 2.5%   704.1918  704.1918
## 97.5% 1173.5503 1173.5503
```

The bootstrap distribution for each parameter can be represented graphically, where the vertical dotted lines indicate the maximum likelihood and the bottom line indicate the percentile confidence intervals

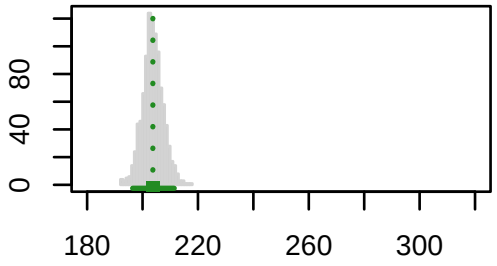
```
par(mfrow = c(2,2))
plot(bootClust, what = "pro")
plot(bootClust, what = "mean")
```



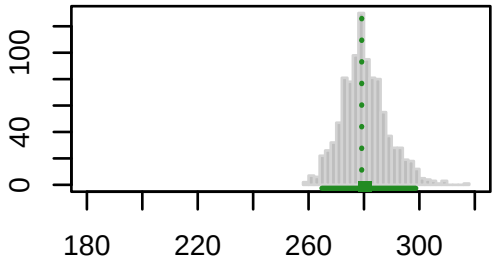
Mix. prop. for comp. 1



Mix. prop. for comp. 2



mean for comp. 1



mean for comp. 2